

Phenix Tutorials

26 documentation pages

Generated from local Phenix documentation

Contents

1. Phenix Examples and Tutorials
2. Phenix Video Tutorials
3. Tutorial: Iterative model-building, density modification and refinement starting from experimental phases
4. Tutorial: Solving a structure with cryo-EM data using docking
5. Tutorial: Solving a structure with cryo-EM data using model-building
6. Tutorial: Cryo_fit2: Fit Biomolecules into Cryo-EM Maps using dynamics (commandline)
7. Install Cryo_fit2
8. How to generate and record movie by cryo_fit2?
9. Tutorial: Fitting Biomolecules into Cryo-EM Maps using MD Simulation (command-line)
10. Tutorial: Fitting Biomolecules into Cryo-EM Maps using MD Simulation (GUI)
11. Install Cryo_fit
12. How to generate and record movie by cryo_fit1?
13. Tutorial: Generating ligand coordinates and restraints for structure refinement
14. Tutorial: Fitting a flexible ligand into a difference electron density map
15. Tutorial: Fitting a flexible ligand into a difference electron density map
16. Tutorial: Solving a structure with MAD data
17. Tutorial: Solving a structure with MIR data
18. Tutorial: Validation with MolProbity
19. Tutorial: solving a structure with Phaser-MR

20. Tutorial: Molecular replacement, refinement, and rebuilding (RanGDP mutant)
21. Tutorial: Automatically rebuilding a model solved by molecular replacement
22. Tutorial: RNase-S Refinement
23. Tutorial: Structure refinement
24. Tutorial: Solving a structure with SAD data
25. Tutorial: Refinement against twinned data
26. Phenix Video Tutorials

Phenix Examples and Tutorials

tutorials.html

Video tutorials with data

The data for each of these tutorials are available in the GUI by selecting 'New Project' and then 'Set up Tutorial Data' and selecting a tutorial. Each video tutorial provides some background and then carries out a demonstration using the tutorial data.

Help with tutorials from the Phenix chatbot

You can use the Phenix chatbot to get help on using any Phenix program. Just ask it any question about using Phenix.

AlphaFold video tutorials

- Overview of structure determination with AlphaFold in Phenix (Introduction to the AlphaFold video tutorials)
- How to run AlphaFold prediction in Phenix (Uses sequence from Fab heavy chain tutorial)
- X-ray structure determination with AlphaFold using PredictAndBuild (Bromodomain tutorial)
- Cryo-EM structure determination with AlphaFold using PredictAndBuild (Fab heavy chain tutorial)

Crystallography video tutorials

- How to run Phaser-MR
- Using an AlphaFold model for molecular replacement
- Scaling and merging anomalous data
- Automated model building with Phenix AutoBuild
- SAD phasing using AutoSol
- Fitting ligands into density with phenix.ligandfit
- Calculating polder maps
- Simulate a SAD experiment with phenix.plan_sad_experiment
- Using Xtricator and Xtriage from HKLviewer for inspecting TNCS, anisotropy and twinning.
- HKLViewer demonstration
- Checking data quality with phenix.xtriage
- Advanced Tutorial phenix.xtriage - Space Group Identification, Ice rings & Wilson Plot
- Advanced Tutorial phenix.xtriage - translational NCS (tNCS)
- Advanced Tutorial phenix.xtriage - Checking for twinning

Validation video tutorials

- How to run MolProbity (web interface and Phenix GUI)
- MolProbity: All atom contacts tutorial

Refinement video tutorials

- How to run real-space-refine
- Phenix tutorial: phenix.refine using default values (GUI)
- Changing custom parameters in phenix.refine
- How to use secondary structure restraints
- How to use multiple refinement strategies and apply them to selected parts of a model

Model manipulation and restraints video tutorials

- Basic Phenix eLBOW tutorial
- How to run PDB tools via the GUI
- Explaining the atom selection syntax
- How to run Structure Comparison

Cryo-EM video tutorials

- Automated map sharpening with phenix.autosharpen
- Density modification for cryo-EM maps
- Automatic map interpretation with map_to_model

ChimeraX/Isolde with Phenix

- Running Phenix via ChimeraX: setup
- Running Emplace_local via ChimeraX
- Introduction to interactive low-resolution model building with ISOLDE

Sample data and instructions supplied with the GUI

Step-by-step Tutorials

- Tutorial 1: Solving a structure using SAD data
- Tutorial 2: Solving a structure using MAD data
- Tutorial 3: Solving a structure using MIR data
- Tutorial 4: Iterative model-building, density modification and refinement starting from experimental phases
- Tutorial 5: Introduction to molecular replacement, refinement, and rebuilding for novice users
- Tutorial 6: Solving a structure using Molecular Replacement
- Tutorial 7: Automatically rebuilding a structure solved by Molecular Replacement
- Tutorial 8: Fitting a flexible ligand into a difference electron density map
- Tutorial 9: Structure refinement
- Tutorial 10: Refining a structure in the presence of merohedral twinning
- Tutorial 11: Generating ligand coordinates and restraints for structure refinement

- Tutorial 12: Structure validation using MolProbity
- Tutorial 13: Running refinement examples on the command line
- Tutorial 14: Solving a cryo-EM structure by density modification, docking, refinement and loop fitting
- Tutorial 15: Solving a cryo-EM structure by density modification, model-building, refinement and loop fitting

Phenix Video Tutorials

tutorial_channel_raw.html

The Phenix video tutorials explain how to run Phenix programs and how to interpret the results. All tutorial videos are available on the Phenix tutorials YouTube channel: www.youtube.com/c/phenixtutorials. A list of all videos is also below (click on the panels to expand the view).

Overview

Map-tools

Refinement

Reflection data

Data analysis

Molecular replacement

Experimental Phasing

Other tools

Ligands

Validation

Mixed Tools

Mini-lectures

Other

Full length lectures (45 min - 1 hour)

Tutorial: Iterative model-building, density modification and refinement starting from experimental phases

autobuild.html

Introduction

This tutorial will start with experimental SAD data and density-modified phases, and carry out the process of iterative model-building, density modification and refinement with AutoBuild. It is designed to be read all the way through, giving pointers for you along the way. Once you have read it all and run the example data and looked at the output files, you will be in a good position to run your own data through AutoBuild.

Setting up to run PHENIX

If PHENIX is already installed and your environment is all set, then if you type:

```
echo $PHENIX
```

then you should get back something like this:

```
/xtal/phenix-1.3
```

If instead you get:

```
PHENIX: undefined variable
```

then you need to set up your PHENIX environment. See the PHENIX installation page for details of how to do this. If you are using the C-shell environment (csh) then all you will need to do is add one line to your .cshrc (or equivalent) file that looks like this:

```
source /xtal/phenix-1.3/phenix_env
```

(except that the path in this statement will be where your PHENIX is installed). Then the next time you log in \$PHENIX will be defined.

Running the demo p9-build data with AutoBuild

To run AutoBuild on the demo p9-build data, make yourself a tutorials directory and cd into that directory:

```
mkdir tutorials
cd tutorials
```

Now type the phenix command:

```
phenix.run_example --help
```

to list the available examples. Choosing p9-build for this tutorial, you can now use the phenix command:

```
phenix.run_example p9-build
```

to build the p9-build structure with AutoBuild. This command will copy the directory \$PHENIX/examples/p9-build to your current directory (tutorials) and call it tutorials/p9-build/. Then it will run AutoBuild using the command file run.sh that is present in this tutorials/p9-build/ directory.

This command file run.sh is simple. It says:

```
#!/bin/sh
echo "Running AutoBuild on P9 data..."
phenix.autobuild seq_file=p9.seq data=p9-solve.mtz \
```

```
input_map_file=p9-resolve.mtz resolution=2.4 \  
ncs_copies=1
```

The first line (`#!/bin/sh`) tells the system to interpret the remainder of the text in the file using the `sh` (or `bash`) -shell (`sh`).

The command `phenix.autobuild` runs the command-line version of AutoBuild (see Automated Structure Solution using AutoBuild for all the details about AutoBuild including a full list of keywords).

The arguments on the command line tell AutoBuild about the sequence file (`seq_file=p9.seq`), the data file (`data=p9-solve.mtz`), the map file with density-modified phases (`input_map_file=p9-resolve.mtz`), and the resolution `resolution=2.4` and number of ncs copies to look for (`ncs_copies=1`). (Note that each of these is specified with an `=` sign, and that there are no spaces around the `=` sign.)

Note the backslash "`\`" at the end of some of the lines in the `phenix.autobuild` command. This tells the C-shell (which interprets everything in this file) that the next line is a continuation of the current line. There must be no characters (not even a space) after the backslash for this to work.

The structure factor amplitudes and experimental phase information are in the datafile `p9-solve.mtz`. This is an `mtz` file which is a binary file that contains summary information about the dataset as well as the reflection data.

Although the `phenix.run_example p9-build` command has just run AutoBuild from a script (`run.sh`), you can run AutoBuild yourself from the command line with the same `phenix.autobuild seq_file= ...` command. You can also run AutoBuild from a GUI, or by putting commands in another type of script file. All these possibilities are described in Using the PHENIX Wizards.

Where are my files?

Once you have started AutoBuild or another Wizard, an output directory will be created in your current (working) directory. The first time you run AutoBuild in this directory, this output directory will be called `AutoBuild_run_1_` (or `AutoBuild_run_1_`, where the slash at the end just indicates that this is a directory). All of the output from run 1 of AutoBuild will be in this directory. If you run AutoBuild again, a new subdirectory called `AutoBuild_run_2_` will be created.

Inside the directory `AutoBuild_run_1_` there will be one or more temporary directories such as `TEMP0` created while the Wizard is running. The files in this temporary directory may be useful sometimes in figuring out what the Wizard is doing (or not doing!). By default these directories are emptied when the Wizard finishes (but you can keep their contents with the command `clean_up=False` if you want.)

What parameters did I use?

Once the AutoBuild wizard has started (when run from the command line), a parameters file called `autobuild.eff` will be created in your output directory (e.g., `AutoBuild_run_1_/autobuild.eff`). This parameters file has a header that says what command you used to run AutoBuild, and it contains all the starting values of all parameters for this run (including the defaults for all the parameters that you did not set).

The `autobuild.eff` file is good for more than just looking at the values of parameters, though. If you copy this file to a new one (for example `autobuild_lores.eff`) and edit it to change the values of some of the parameters (`resolution=3.0`) then you can re-run AutoBuild with the new values of your parameters like this:

```
phenix.autobuild autobuild_lores.eff
```

This command will do everything just the same as in your first run but use only the data to 3.0 Å.

Reading the log files for your AutoBuild run file

While the AutoBuild wizard is running, there are several places you can look to see what is going on. The most important one is the overall log file for the AutoBuild run. This log file is located in:

```
AutoBuild_run_1_/AutoBuild_run_1_1.log
```

for run 1 of AutoBuild. (The second 1 in this log file name will be incremented if you stop this run in the middle and restart it with a command like `phenix.autobuild run=1`).

The `AutoBuild_run_1_1.log` file is a running summary of what the AutoBuild Wizard is doing. Here are a few of the key sections of the log files produced for the p9-build SAD dataset.

Summary of the command-line arguments

Near the top of the log file you will find:

```
-----  
Starting AutoBuild with the command:
```

```
phenix.autobuild seq_file=p9.seq data=p9-solve.mtz \  
input_map_file=p9-resolve.mtz resolution=2.4 ncs_copies=1
```

This is just a repeat of how you ran AutoBuild; you can copy it and paste it into the command line to repeat this run.

Guessing the chain type

The AutoBuild Wizard will read in your sequence file and guess whether this is PROTEIN, DNA, or RNA from the sequence:

```
Guessing chain type from p9.seq  
Setting chain type to PROTEIN
```

If you want to tell the Wizard what the chain type is, you can say, `chain_type=PROTEIN`.

Guessing column labels

The AutoBuild Wizard will need to know which columns in your input data file and your input map file to use. It guesses which column labels to use and lists them out:

```
Getting column labels from p9-solve.mtz for input data file  
SG: I 4  
Cell: [113.94899749755859, 113.94899749755859, 32.4739990234375, 90.0, 90.0, 90.0]  
Input labels: ['FP', 'SIGFP', 'PHIB', 'FOM', 'HLA', 'HLB', 'HLC', 'HLD', 'None']  
  
Getting column labels from p9-resolve.mtz for input map file  
SG: I 4  
Cell: [113.94899749755859, 113.94899749755859, 32.4739990234375, 90.0, 90.0, 90.0]  
Map input labels: ['FP', 'PHIM', 'FOMM']
```

These are indeed the appropriate columns to use for experimental phases and for map coefficients, respectively. Note the "None" in the input labels for p9-solve.mtz. The last input label in this list corresponds to `FreeR_flag` and there is no Free R data in the input data file. All the data that is expected for each input file in AutoBuild can be seen in the AutoBuild web page under "Specifying which columns of data to use from input data files".

Guessing cell contents

The AutoBuild Wizard uses the sequence information in your sequence file (sequence.dat) and the cell parameters and space group to guess the number of NCS copies and the solvent fraction:

```
Number of residues in unique chains in seq file: 136
Unit cell: (113.949, 113.949, 32.474, 90, 90, 90)
Space group: I 4 (No. 79)
CELL VOLUME :421654.549593
N_EQUIV:8
GUESS OF NCS COPIES: 1
SOLVENT FRACTION ESTIMATE: 0.65
Data file (for everything including refinement): p9-solve.mtz
```

Running phenix.xtriage

The AutoBuild Wizard automatically runs phenix.xtriage on your input datafile to analyze it for twinning, outliers, translational symmetry, and other special conditions that you should be aware of. You can read more about xtriage in Data quality assessment with phenix.xtriage. The xtriage output is in the file p9-solve.mtz_xtriage.log. Part of the summary output from xtriage for this dataset looks like this:

```
The largest off-origin peak in the Patterson function is 6.40% of the
height of the origin peak. No significant pseudotranslation is detected.
```

```
The results of the L-test indicate that the intensity statistics
behave as expected. No twinning is suspected.
```

Generation of FreeR flags

The AutoBuild Wizard will create a set of free R flags indicating which reflections are not to be used in refinement. By default 5% of reflections (up to a maximum of 2000) are reserved for this test set. If you supply a reflection file with free R flags already set, then they will be used. If you want to supply a file ref.mtz specifically for refinement, you can do that with input_refinement_file=ref.mtz. Also if you want to supply a high-resolution datafile hires.mtz that has then you can do this with the keywords input_hires_file=hires.mtz. After generation of free R flags if necessary, and any merging of data files, the file to be used for refinement is called exptl_fobs_phases_freeR_flags.mtz.

Model-building with RESOLVE

The AutoBuild Wizard by default uses RESOLVE to build an atomic model of your structure.

In each cycle of model-building, the AutoBuild Wizard breaks up the building process into separate steps that can be run in parallel and then combines the results.

In the first model-building cycle, the AutoBuild Wizard builds 4 separate models by running 4 subprocesses, each of which runs the AutoBuild Wizard to just build a single model and refine it and return:

```
Build cycle 1 of 17 method:build

Build cycle 1 of 17 method:build

Running 3 parallel build jobs

Standard build in parallel
This is the first try at building this model
Setting background=False as nproc=1
Try: 1 building 1 model
Try: 2 building 1 model
Try: 3 building 1 model
Running up to 1 jobs in parallel... with total of 3 jobs
Splitting work into 3 jobs and running 1 at a time with sh in
/net/idle/scratch1/terwill/run_072908a/p9-build/AutoBuild_run_1_/TEMP0
```

```

Starting job 1...
Starting job 2...Starting job 3...
Collecting models....

Solution for try : 1 cycle_best_1.pdb
Solution 1 from build cycle 1 R= 0.24
Saving /net/idle/scratch1/terwill/run_072908a/p9-build/AutoBuild_run_1/TEMP0/
AutoBuild_run_1/cycle_best_1.pdb as MODEL_1.pdb in AutoBuild_run_1/TEMP0

Solution for try : 2 cycle_best_1.pdb
Solution 1 from build cycle 1 R= 0.24
Saving /net/idle/scratch1/terwill/run_072908a/p9-build/AutoBuild_run_1/TEMP0/
AutoBuild_run_2/cycle_best_1.pdb as MODEL_2.pdb in AutoBuild_run_1/TEMP0

Solution for try : 3 cycle_best_1.pdb
Solution 1 from build cycle 1 R= 0.24
Saving /net/idle/scratch1/terwill/run_072908a/p9-build/AutoBuild_run_1/TEMP0/
AutoBuild_run_3/cycle_best_1.pdb as MODEL_3.pdb in AutoBuild_run_1/TEMP0
Done with 3 parallel build jobs...
Running standard build to merge and extend these models now.

```

If you want to look at the log files for these individual model-building steps, you can look in the directory listed above:

```
/net/cci-filer1/vol1/tmp/terwill/phenix_examples/p9-build/AutoBuild_run_1/TEMP0
```

This will contain subdirectories with the model-building runs:

```

AutoBuild_run_1/TEMP0/AutoBuild_run_1/AutoBuild_run_1_1.log
AutoBuild_run_1/TEMP0/AutoBuild_run_2/AutoBuild_run_2_1.log
AutoBuild_run_1/TEMP0/AutoBuild_run_3/AutoBuild_run_3_1.log

```

In this case model 1 is the best. It is then used to start a merging process in which the best parts of each model are kept to create a composite model. This model is then refined, extended (by building off all the ends of chains) and saved:

```

Model completion cycle 1
Models to combine and extend: ['MODEL_1.pdb', 'MODEL_2.pdb', 'MODEL_3.pdb']
Model 1: Residues built=126 placed=118 Chains=2 Model-map CC=0.82
This is new best model with score = 240
Refining model: Build_combine_extend_1.pdb
Model: AutoBuild_run_1/TEMP0/refine_1.pdb R/Rfree=0.23/0.28

...

New best overall: AutoBuild_run_1/overall_best.pdb

```

```

Model obtained on cycle 1
R (work): 0.229221223063
R (free): 0.278562331997
Residues built: 126
Residues placed: 118
Model-map CC: 0.82
Chains: 2

```

In the second overall cycle of model building, the AutoBuild Wizard carries out several density modification steps to obtain an improved map:

```
Using coordinates of model from previous cycle in building
```

This is the key aspect of iterative model-building, density modification and refinement. The model from each cycle is used to improve the density (even in places where the model was not built) in the map for the next cycle. This is done by using density calculated from the model as part of a real-space target for statistical density modification.

The first stage of density modification uses the identification of local patterns of density unique to macromolecules (such as characteristic distances between atoms), and the presence of helices and strands in the map to improve map quality. The pattern and fragment information comes from an analysis of the map from the previous cycle and is combined into a pseudo-map combine.map. This real-space information is then merged with the experimental phase information in exptl_fobs_phases_freeR_flags.mtz:

```
Density modifying with patterns/fragments and model
Adding pattern/fragment phase information from combine.map to exptl_fobs_phases_freeR_flags.mtz
to create image.mtz
```

The composite phase information is then used in density modification. In this step the model density is used as part of the real-space target for statistical density modification. An omit map is also created that does not include the model-based information.

```
Density modifying image.mtz including model information from refine.pdb_1
to make resolve_work.mtz. Then building model
Creating omit map from image.mtz and previous models
```

A new model is then built, using the best model available so far (in this case from cycle 1), combined with pieces of a model built in four ways. The first is to "fit_loops", in which case all gaps in the model (places where the sequence file says there are residues but for which there is no model yet, and for which the residues on either side of the gap are present in the model) are systematically rebuilt. The method used is to try to build from either end of the gap and if the two chains connect with the correct number of residues then the gap is considered filled. The second method is to "connect" ends of chains. This is the same as the gap-filling procedure except that it is used in cases where the model has not been assigned to sequence so that the ends to be connected are not known in advance and the number of residues in the gap is also not known. The third method is to "build_outside", in which case the current model is used to mask out the density in the region of the model, and a model is built into the remaining density. The fourth method is simply to build a new model from scratch.

```
Standard build in parallel starting with refine.pdb_1 and ['overall_best.pdb']
```

```
Setting background=False as nproc=1
Try: 1 fit_loops=True
Try: 2 connect=True
Try: 3 build_outside=True
```

Then the current best model and the models built from each of these tries are combined together to make a composite model. As in cycle 1 it is then refined, extended, and saved (if it is an improvement):

```
Solution for try : 1 cycle_best_1.pdb
Solution 1 from build cycle 1 R= 999.9
Saving /net/idle/scratch1/terwill/run_072908a/p9-build/
AutoBuild_run_1_/TEMP0/AutoBuild_run_4_/cycle_best_1.pdb as
MODEL_1.pdb in AutoBuild_run_1_/TEMP0
```

```
Solution for try : 2 cycle_best_1.pdb
Solution 1 from build cycle 1 R= 999.9
Saving /net/idle/scratch1/terwill/run_072908a/p9-build/
AutoBuild_run_1_/TEMP0/AutoBuild_run_5_/cycle_best_1.pdb as
MODEL_2.pdb in AutoBuild_run_1_/TEMP0
```

```
Solution for try : 3 cycle_best_1.pdb
Solution 1 from build cycle 1 R= 0.23
Saving /net/idle/scratch1/terwill/run_072908a/p9-build/
AutoBuild_run_1_/TEMP0/AutoBuild_run_6_/cycle_best_1.pdb as
MODEL_3.pdb in AutoBuild_run_1_/TEMP0
Done with 3 parallel build jobs...
Running standard build to merge and extend these models now.
```

In this case the building outside-model gave a good model but the gap-filling and connecting did not fill any gaps or loops. (they gave an R of 999. meaning nothing was refined).

Model completion with these models and the current best model from cycle 1 gave:

```
Model completion cycle 1
Models to combine and extend: ['overall_best.pdb', 'starting_model.pdb',
'MODEL_1.pdb', 'MODEL_2.pdb', 'MODEL_3.pdb']
Model 1: Residues built=123 placed=107 Chains=2 Model-map CC=0.80
This is new best model with score = 226
Refining model: Build_combine_extend_1.pdb
Model: AutoBuild_run_1/TEMP0/refine_1.pdb R/Rfree=0.23/0.27
```

This process of model-building iterated with generation of real-space targets for density modification based on local patterns, fragments of structure, and the model is repeated until the R-factor does not decrease for several cycles. In this example, the best model using this procedure is obtained on cycle 2 with an R/Rfree of 0.22/0.27:

```
New best overall: AutoBuild_run_1/overall_best.pdb
Model obtained on cycle 2
R (work): 0.224655082668
R (free): 0.266763231999
Residues built: 120
Residues placed: 112
Model-map CC: 0.79
Chains: 1
```

Model-rebuilding with RESOLVE

Once the model-building procedure has converged, the AutoBuild Wizard carries out cycles of rebuilding using a slightly different protocol. The main differences in this set of cycles are that the local patterns and fragments approaches are no longer used (the maps by this time look so much like a macromolecule that these procedures do not add anything), and that the starting point for density modification is a model-based map, not the experimental map.

In this example, the rebuilding steps improve the model just a little, and the process ends after 2 cycles of rebuilding:

```
New best overall: AutoBuild_run_1/overall_best.pdb
Model obtained on cycle 2
R (work): 0.224655082668
R (free): 0.266763231999
Residues built: 120
Residues placed: 112
Model-map CC: 0.79
Chains: 1
```

The AutoBuild_summary.dat summary file

A quick summary of the results of your AutoBuild run is in the AutoBuild_summary.dat file in your output directory. This file lists the key files that were produced in your run of AutoBuild (all these are in the output directory) and some of the key statistics for the run. Here is the summary for this p9-build model-building run:

```
Summary of model-building for run 1 Sun Jun 29 12:02:56 2008
Files are in the directory: /net/idle/scratch1/terwill/run_072908a/p9-build/AutoBuild_run_1/

Starting mtz file: /net/idle/scratch1/terwill/run_072908a/p9-build/p9-solve.mtz
Sequence file: /net/idle/scratch1/terwill/run_072908a/p9-build/p9.seq

Best solution on cycle: 4 R/Rfree=0.22/0.27

Summary of output files for Solution 3 from rebuild cycle 4
```

```

--- Model (PDB file) ---
pdb_file: AutoBuild_run_1_/cycle_best_4.pdb

--- Refinement log file ---
log_refine: AutoBuild_run_1_/cycle_best_4.log_refine

--- Model-building log file ---
log: AutoBuild_run_1_/cycle_best_4.log

--- Model-map correlation log file ---
log_eval: AutoBuild_run_1_/cycle_best_4.log_eval

--- 2FoFc and FoFc map coefficients from refinement 2FOFCWT PH2FOFCWT FOFCWT PHFOFCWT ---
refine_map_coeffs: AutoBuild_run_1_/cycle_best_refine_map_coeffs_4.mtz

--- Data for refinement FP SIGFP PHIM FOMM HLAM HLBM HLCM HLDM FreeR_flag ---
hklout_ref: AutoBuild_run_1_/exptl_fobs_phases_freeR_flags.mtz

--- Density-modification log file ---
log_denmod: AutoBuild_run_1_/cycle_best_4.log_denmod

--- Density-modified map coefficients FP PHIM FOM ---
hklout_denmod: AutoBuild_run_1_/cycle_best_4.mtz

```

You might consider making one very good model now with:

```

phenix.autobuild \
  data=AutoBuild_run_1_/exptl_fobs_phases_freeR_flags.mtz \
  model=AutoBuild_run_1_/cycle_best_4.pdb \
  rebuild_in_place=True \
  seq_file=/net/idle/scratch1/terwill/run_072908a/p9-build/p9.seq

```

```

SOLUTION CYCLE R RFREE BUILT PLACED
1 1 0.23 0.28 126 118
2 2 0.22 0.27 120 112
3 4 0.22 0.27 123 111
4 5 0.22 0.26 121 113

```

Note that the file AutoBuild_run_1_/cycle_best_4.log_eval in this example has a complete analysis of the the fit of the model in AutoBuild_run_1_/cycle_best_4.pdb to the best map in AutoBuild_run_1_/cycle_best_4.log_denmod. This is useful in identifying places where additional rebuilding needs to be done.

Creating an improved model after AutoBuild

In our example, the summary file had this phrase in it:

You might consider making one very good model now with:

```

phenix.autobuild \
  data=AutoBuild_run_1_/exptl_fobs_phases_freeR_flags.mtz \
  model=AutoBuild_run_1_/cycle_best_4.pdb \
  rebuild_in_place=True \
  seq_file=/net/cci-filer1/vol1/tmp/terwill/phenix_examples/p9-build/p9.seq

```

This is a command-line command to take the final model from this AutoBuild run, rebuild it 4 times using the "rebuild_in_place" algorithm, combine the resulting models to make a composite model, refine it, and write out the final model. This method is very effective at improving models from AutoBuild (or from any other source). You can see more details of this in Tutorial 6: Automatically rebuilding a structure solved by Molecular Replacement

How do I know if iterative model-building, density modification and refinement worked?

Here are some of the things to look for to tell if you have obtained a good model:

- How much of the model was built? For the AutoBuild Wizard, you should expect more than 90% of your model to be built in favorable cases when the resolution of the data is 3 Å or better. Have a look at the model. It should have all the appropriate features of a protein or nucleic acid.
- What is the R-factor and the free R-factor of the model? For a solution at moderate to high resolution (2.5 Å or better) the R-factor should be in the low 20's and the free R-factor about 0.05 higher to be very good. For lower-resolution data, an R-factor in the low 30's is probably largely correct but the model is not very good.
- Does the model show NCS if there are more than one copies in the asymmetric unit? You can look at your log file which will tell you about any NCS in your model.

What to do next

Once you have run AutoBuild and have obtained a good model, you will want to inspect and touch up the model carefully, rebuilding any parts of the model that do not agree well with the final map. You should also have a close look at all the solvent molecules in your model, making sure that they all have reasonable relationships to the macromolecule and to each other, and that they are not simply filling up density where a ligand or the macromolecule really goes.

The next thing to do is to add in any ligands (metals, cofactors) if there is density for them. You can use the LigandFit Wizard (see Automated Ligand Fitting using LigandFit) to help you fit ligands into your map automatically.

If you do not obtain a good model, then it's not quite time to give up yet. There are a number of standard things to try that may improve the model building. Here are a few that you should try:

- Have a careful look at all the output files. Work your way through the main log file (e.g., AutoBuild_run_1_1.log). Is there anything strange or unusual in any of them that may give you a clue as to what to try next? What does the xtriage output say? Is there twinning or strong translational symmetry? Are there problems with reflections near ice rings? Are there many outlier reflections?
- Have a look at the electron density maps at different stages in model building. How does the starting map look? How about the maps produced during the iterative building process (cycle_best_xx.mtz)? If the maps look poor, then the rebuilding process will not work well. However if the maps look good and the rebuilding is not working well then it is worth trying to identify where the problem is.
- Try a different resolution cutoff. You may be able to use higher-resolution data in AutoBuild than you used in structure solution.

Additional information

For details about the AutoBuild Wizard, see Automated Model building and Rebuilding using AutoBuild. For help on running Wizards, see Using the PHENIX Wizards.

Tutorial: Solving a structure with cryo-EM data using docking

[cryo_em_structure_solution_docking.html](#)

This example is density modification of a cryo-EM map, followed by optional structure determination by docking a homologous structure

The data are half-maps for apoferritin (EMD entry 20026, 1.8 Å)

The files are:

emd_20026_half_map_1_box.ccp4 emd_20026_half_map_2_box.ccp4 -- half maps
seq.dat -- sequence file
20026_auto_sharpen_A.ccp4 -- deposited map, half-map sharpened
apoferritin_chainA_rsr.pdb -- model refined against sharpened map
1aew_A.pdb -- chain A from X-ray structure of horse apoferritin (53% identical to human apoferritin)
run_denmod_dock.csh -- command file that will run all these things for you if you want

- emd_20026_half_map_1_box.ccp4 emd_20026_half_map_2_box.ccp4 -- half maps
- seq.dat -- sequence file
- 20026_auto_sharpen_A.ccp4 -- deposited map, half-map sharpened
- apoferritin_chainA_rsr.pdb -- model refined against sharpened map
- 1aew_A.pdb -- chain A from X-ray structure of horse apoferritin (53% identical to human apoferritin)
- run_denmod_dock.csh -- command file that will run all these things for you if you want

The maps are all cut out (boxed) from the deposited maps (just to make the files smaller for this demo).

- DENSITY MODIFICATION OF HALF-MAPS

To run one cycle of density modification from the command line you can paste this in to a terminal window. From the GUI just set all the same parameters and no need to set the output_directory (it will go in resolve_cryo_em_0 or a similar directory name):

```
phenix.resolve_cryo_em emd_20026_half_map_1_box.ccp4  
emd_20026_half_map_2_box.ccp4 resolution=2.2 dm_resolution=1.8 seq_file=seq.dat cycles=1 nproc=4  
box_before_analysis=False output_directory=denmod  
density_modify_unsharpened_maps=True initial_map_file_name=initial_map.ccp4
```

In about 15 minutes this will produce (in a resolve_cryo_em_xx directory) the file:

```
denmod/denmod_map.ccp4 -- density-modified map
```

You may want to cut out the parts of your map that match apoferritin_chainA_rsr.pdb:

```
phenix.map_box ignore_symmetry_conflicts=True  
apoferritin_chainA_rsr.pdb denmod/denmod_map.ccp4 prefix=denmod_map_A
```

We are saying "ignore_symmetry_conflicts=True" because the map in denmod/denmod_map.ccp4 may have a different "unit cell" than apoferritin_chainA_rsr.pdb. Here the "unit cell" is just a description of the box of points that are in the map. It can sometimes refer to the size of the original map and sometimes to the size of the working part of the map and this command says to ignore these differences.

In Coot or Chimera you can load in the model (apoferritin_chainA_rsr.pdb) and compare the sharpened deposited map (20026_auto_sharpen_A.ccp4) with the density modified map (denmod_map_A.ccp4). Look at residues 13 and 81 and the main chain near residue 161 for examples of substantial improvement in the map.

- STRUCTURE SOLUTION BY DOCKING HOMOLOGOUS MODEL (horse apoferritin, 53% identical)

The file 1aew_A.pdb contains the X-ray structure of 1aew, a homologous protein from horse (53% identical, rmsd 0.5 Å). We are going to dock this structure into our density-modified map, refine it, fix the sequence, refine again, and density modify using the model to get a final map.

1. Getting map symmetry We will want a symmetry file describing the symmetry operators that have been used in our map. We can get them automatically in about 30 seconds with:

```
phenix.map_symmetry denmod/denmod_map.ccp4 resolution=2
```

```
symmetry_out=denmod_map.ncs_spec
```

Now denmod_map.ncs_spec has the 24 symmetry operators for octahedral symmetry in the appropriate location of this map.

- Docking into density-modified map

We can find where 1aew_A.pdb fits in our map with dock_in_map:

```
phenix.dock_in_map denmod/denmod_map.ccp4 1aew_A.pdb resolution=2 nproc=4
```

```
symmetry_file=denmod_map.ncs_spec pdb_out=1aew_A_docked_in_denmod_map.pdb
```

In a minute or two you should get the model 1aew_A_docked_in_denmod_map.pdb.

- Cut out a piece of density-modified map and compare to docked model

You can cut out a piece of your density modified map and look at it along with this model this way:

```
phenix.map_box denmod/denmod_map.ccp4 1aew_A_docked_in_denmod_map.pdb
ignore_symmetry_conflicts=true
```

This will produce 1aew_A_docked_in_denmod_map_box.ccp4 and

1aew_A_docked_in_denmod_map_box.pdb which you can look at in Coot or Chimera. Notice that a lot of the side chains do not match the density because the model is from a homologous structure.

- Refine the model against the density-modified map

We can refine the docked model either against the original or cut-out map. Let's use the cut-out map. We can also refine just chain A or the whole 24-mer. The difference is if you use the whole 24-mer you are going to include inter-chain contacts and if you use a single chain you will ignore them. Let's use just one chain for now:

```
phenix.real_space_refine 1aew_A_docked_in_denmod_map.pdb
1aew_A_docked_in_denmod_map_box.ccp4 resolution=2 ignore_symmetry_conflicts=True
```

After a couple minutes we have the output model:

1aew_A_docked_in_denmod_map_real_space_refined.pdb. If you look at this model in Coot or Chimera you'll see it has moved some side chains into density, but that they still don't match the density because the sequence of this docked model is not the sequence of human apoferritin (look at R29 for example which should be a tyrosine).

- Fix the sequence

We can redo the sequence of our docked model with sequence_from_map:

```
phenix.sequence_from_map 1aew_A_docked_in_denmod_map_real_space_refined.pdb
```

```
1aew_A_docked_in_denmod_map_box.ccp4 resolution=2 seq_unique.dat nproc=4
pdb_out=1aew_A_docked_rsr_seq.pdb
```

```
1aew_A_docked_in_denmod_map_box.ccp4 resolution=2 seq_unique.dat nproc=4  
pdb_out=1aew_A_docked_rsr_seq.pdb
```

In a couple minutes you should get the file: 1aew_A_docked_rsr_seq.pdb with the sequence of human apoferritin mapped onto the real-space-refined model. Have a look at this with the map now in Coot or Chimera. You can see that some of the side chains need refinement and that a loop by P157 needs to be moved. Let's first just refine our new model.

- Refine model with new sequence

We can refine just the same as before but with our new sequence:

```
phenix.real_space_refine 1aew_A_docked_rsr_seq.pdb 1aew_A_docked_in_denmod_map_box.ccp4  
resolution=2 ignore_symmetry_conflicts=True
```

In a couple minutes we have the refined model: 1aew_A_docked_rsr_seq_real_space_refined.pdb. This model fits the map pretty well, but the loop at P157 is still way off. We need to re-fit this loop.

- Fit a loop that is not in density

We can fix this loop using the fit_loops tool. We can look at the model and map and notice that the part of the model that doesn't fit is residues 155-159. Let's fix these with:

```
phenix.fit_loops 1aew_A_docked_rsr_seq_real_space_refined.pdb  
map_in=1aew_A_docked_in_denmod_map_box.ccp4 resolution=2 seq_unique.dat remove_loops=True  
start=155 end=159 pdb_out=1aew_A_docked_rsr_seq_rsr_loop157.pdb
```

In a minute this should fit this part of the chain into the density.

- Refine model with fitted loop

We can refine one more time after fitting our loop at P157:

```
phenix.real_space_refine 1aew_A_docked_rsr_seq_rsr_loop157.pdb  
1aew_A_docked_in_denmod_map_box.ccp4 resolution=2 ignore_symmetry_conflicts=True
```

Now we have 1aew_A_docked_rsr_seq_rsr_loop157_real_space_refined.pdb. Have a look at it in Coot or Chimera.

Tutorial: Solving a structure with cryo-EM data using model-building

[cryo_em_structure_solution_model_building.html](#)

This example is density modification of a cryo-EM map, followed by structure determination by automatic model-building, fixing a mis-traced segment, sequence assignment and refinement

NOTE: the details of what needs to be fixed after automatic model-building (steps 5-11) may change with platform, number of processors or version of Phenix used

The data are half-maps for apoferritin (EMD entry 20026, 1.8 Å)

The files are (NOTE: the map files are in the apoferritin_denmod directory):

emd_20026_half_map_1_box.ccp4 emd_20026_half_map_2_box.ccp4 -- half maps seq.dat -- sequence file
run_denmod_model_building.csh -- script to run all these commands

- emd_20026_half_map_1_box.ccp4 emd_20026_half_map_2_box.ccp4 -- half maps
- seq.dat -- sequence file
- run_denmod_model_building.csh -- script to run all these commands

The maps are all cut out (boxed) from the deposited maps (just to make the files smaller for this demo).

- DENSITY MODIFICATION OF HALF-MAPS

To run one cycle of density modification from the command line you can paste this in to a terminal window. From the GUI just set all the same parameters and no need to set the output_directory (it will go in resolve_cryo_em_0 or a similar directory name):

```
phenix.resolve_cryo_em emd_20026_half_map_1_box.ccp4
```

```
emd_20026_half_map_2_box.ccp4 resolution=2.2 dm_resolution=1.8 seq_file=seq.dat cycles=1 nproc=4  
box_before_analysis=False density_modify_unsharpened_maps=True output_directory=denmod  
output_files.denmod_map_file_name=denmod_map.ccp4 initial_map_file_name=initial_map.ccp4
```

In about 15 minutes this will produce (in a denmod/ or

resolve_cryo_em_xx directory) the file: denmod_map.ccp4

- STRUCTURE SOLUTION BY AUTOMATED MODEL-BUILDING

We are going to build a model into our density-modified map,

refine it, fix the sequence and refine again.

1. Getting map symmetry We will want a symmetry file describing the symmetry operators that have been used in our map. We can get them automatically in about 30 seconds with:

```
phenix.map_symmetry denmod/denmod_map.ccp4 resolution=2  
symmetry_out=denmod_map.ncs_spec
```

Now denmod_map.ncs_spec has the 24 symmetry operators for octahedral symmetry in the appropriate location of this map.

- Automated model-building using a density-modified map

We can build a model into our map with map_to_model:

```
phenix.map_to_model denmod/denmod_map.ccp4 resolution=2 nproc=4
```

```
ncs_file=denmod_map.ncs_spec pdb_out=map_to_model.pdb seq_file=seq.dat
```

In a half hour or so you should get the model 1aew_A_built_in_denmod_map.pdb.

3. Cut out a piece of density-modified map and compare to build_in model You can cut out a piece of your density modified map and look at it along with your model this way:

```
phenix.map_box denmod/denmod_map.ccp4 map_to_model.pdb
```

```
ignore_symmetry_conflicts=true prefix=denmod_map_A
```

This will produce denmod_map_A.ccp4 and denmod_map_A.pdb which you can look at in Coot or Chimera. The model is mostly pretty good but may need some fixing, depending on your version of Phenix

- Refine the model against the density-modified map

We can refine the build_in model either against the original or cut-out map.

Let's use the cut-out map. We can also refine just chain A or the whole 24-mer. The difference is if you use the whole 24-mer you are

going to include inter-chain contacts and if you use a single chain you

will ignore them. Let's use just one chain for now:

```
phenix.real_space_refine map_to_model.pdb denmod_map_A.ccp4 resolution=2
```

```
ignore_symmetry_conflicts=True
```

After a couple minutes we have the output model: map_to_model_real_space_refined.pdb. If you look at this model in Coot or Chimera you'll see it has moved some side chains into density, but that the model still doesn't match the density for residues 154-162.

5. If your model has parts that do not fit, try something like the following.

NOTE: the results for these steps may depend on platform, version, and number of processors so you may have to adjust them based on what your model looks like.

First remove the bad part (here the bad part is residues 154 through 162):

```
phenix.pdbtools map_to_model_real_space_refined.pdb
```

```
remove="resseq 154:162" output.file_name=map_to_model_real_space_refined_del.pdb
```

6. Redo the sequence assignment and see if we can fill in the gap. Here the keyword run_assign_sequence=True is used to reassign the sequence and then fill in the gap:

```
phenix.sequence_from_map run_assign_sequence=True
```

```
model_file=map_to_model_real_space_refined_del.pdb seq_file=seq_unique.dat
```

```
map_file=denmod_map_A.ccp4 pdb_out=sequence_from_map.pdb resolution=2
```

This should produce the file sequence_from_map.pdb in a couple minutes.

- We can fix one other part of the model that does not fit well (residues 6-9):

```
phenix.pdbtools sequence_from_map.pdb
```

```
remove="resseq 6:9" output.file_name=sequence_from_map_del_6-9.pdb
```

- Redo the sequence assignment and see if we can fill in the gap as in 6 above:

```
phenix.sequence_from_map run_assign_sequence=True
```

```
model_file=sequence_from_map_del_6-9.pdb seq_file=seq_unique.dat map_file=denmod_map_A.ccp4  
pdb_out=sequence_from_map_2.pdb resolution=2
```

This should produce the file sequence_from_map_2.pdb in a couple minutes.

- Let's refine one more time:

```
phenix.real_space_refine sequence_from_map_2.pdb denmod_map_A.ccp4
```

```
resolution=2 ignore_symmetry_conflicts=True
```

Now have a look at sequence_from_map_2_real_space_refined.pdb and denmod_map_A.ccp4 in Coot or Chimera. The model should fit the map quite well. If there are missing side chains, you can fill them in with:

- Run sequence_from_map

```
phenix.sequence_from_map sequence_from_map_2_real_space_refined.pdb
```

```
denmod_map_A.ccp4 seq_file=seq_unique.dat resolution=2 pdb_out=sequence_from_map_3.pdb
```

and now sequence_from_map_3.pdb should have more side chains built. You could refine once more too:

- Final refinement:

```
phenix.real_space_refine sequence_from_map_3.pdb denmod_map_A.ccp4
```

```
resolution=2 ignore_symmetry_conflicts=True
```

and sequence_from_map_3_real_space_refined.pdb is the refined model. Next you would go over the model carefully using validation tools and remove any parts that are not ok and fix parts that are not quite right. You can also create the full 24-mer using your map symmetry and the model

Tutorial: Cryo_fit2: Fit Biomolecules into Cryo-EM Maps using dynamics (commandline)

[cryo_fit2_cmdline.html](#)

Contents

- Note
- Overview
- Theory for cryo_fit2
- Input
- How to run
- How to add user specified geometries
- Output as a static structure
- How to show map contour as mesh
- How to generate and record movie?
- FAQ
- Options
- List of most useful options

Note

cryo_fit2 is under active development.

Therefore, consider to use cryo_fit1 instead (especially for RNA modeling).

Overview

This tutorial will show you how to fit biomolecule atomic structures into cryo-EM maps using dynamics simulation with PHENIX commandline

Theory for cryo_fit2

See the theory notes for cryo_fit2

Input

- <initial_model>, <target_map> and <map resolution>

The <initial_model> is a guide or template structure (.cif/mmCIF/.pdb) that is close to a <target_map> (.ccp4/.map) structurally. A user can use map_to_model to build the atomic model. A user can use either dock_in_map or UCSF chimera (Tools->Volume Data->Fit in Map) to roughly place the model into a map. The <target_map> should have decent density. Most mrc type cryo-EM maps have decent density. However, some maps (like from UCSF Chimera's map segmentation) have low density. Therefore, cryo_fit2 will have just little incentive to fit to it. For example, left yellow maps are fine. However, right gray/white maps lack enough density to run. To fit into these low density maps, Doo Nam tried very high map_weights (like 10^8). However, these efforts either broke atomistic model geometries or did not budge the atomistic model.

- The <initial_model> is a guide or template structure (.cif/mmCIF/.pdb) that is close to a <target_map> (.ccp4/.map) structurally.

A user can use map_to_model to build the atomic model. A user can use either dock_in_map or UCSF chimera (Tools->Volume Data->Fit in Map) to roughly place the model into a map.

- A user can use map_to_model to build the atomic model.
- A user can use either dock_in_map or UCSF chimera (Tools->Volume Data->Fit in Map) to roughly place the model into a map.
- The <target_map> should have decent density.

Most mrc type cryo-EM maps have decent density. However, some maps (like from UCSF Chimera's map segmentation) have low density. Therefore, cryo_fit2 will have just little incentive to fit to it. For example, left yellow maps are fine. However, right gray/white maps lack enough density to run. To fit into these low density maps, Doo Nam tried very high map_weights (like 10^8). However, these efforts either broke atomistic model geometries or did not budge the atomistic model.

- Most mrc type cryo-EM maps have decent density.
- However, some maps (like from UCSF Chimera's map segmentation) have low density. Therefore, cryo_fit2 will have just little incentive to fit to it.

For example, left yellow maps are fine. However, right gray/white maps lack enough density to run.

- For example, left yellow maps are fine. However, right gray/white maps lack enough density to run.

For example, left yellow maps are fine. However, right gray/white maps lack enough density to run.

- To fit into these low density maps, Doo Nam tried very high map_weights (like 10^8). However, these efforts either broke atomistic model geometries or did not budge the atomistic model.

How to run

At command line:

```
% phenix.cryo_fit2 <initial_model> <target_map> <map resolution in Angstrom>
```

For example:

```
% phenix.cryo_fit2 model.pdb map.ccp4 3.5
```

Tutorial command:

```
% Go to <user phenix>/modules/cryo_fit2/tutorial
```

```
% source run_me.sh
```

```
%% (run_me.sh is phenix.cryo_fit2 input/tutorial_cryo_fit2_model.pdb input/tutorial_cryo_fit2_map.ccp4 resolution)
```

How to add user specified geometries

- There are two methods of generating a user specified geometry file (.eff file).

If a user specified geometry file is small, then simply copy/modify <user phenix>/modules/cryo_fit2/tutorial/eff_file_template/user_custom_geom.eff, and modify it for a user's need. If a user specified geometry file is large, then run: % phenix.cryo_fit2 <initial_model> <target_map> <map resolution in Angstrom> <write_custom_geom_only=True> <HE_angle_sigma_scale> <HE_sigma> <parallelity_sigma> For example: % phenix.cryo_fit2 model.pdb map.ccp4 3.5 write_custom_geom_only=True HE_angle_sigma_scale=0.04 HE_sigma=0.2 parallelity_sigma=0.02 Then,

automatically generated geometry file should be generated at the same folder. Modify it for a user's need. Other than write_custom_geom_only, all parameters are optional (if a user didn't specify these, cryo_fit2 will use default values).

- If a user specified geometry file is small, then simply copy/modify <user phenix>/modules/cryo_fit2/tutorial/eff_file_template/user_custom_geom.eff, and modify it for a user's need.

If a user specified geometry file is small, then simply copy/modify <user phenix>/modules/cryo_fit2/tutorial/eff_file_template/user_custom_geom.eff, and modify it for a user's need.

- If a user specified geometry file is large, then run: % phenix.cryo_fit2 <initial_model> <target_map> <map resolution in Angstrom> <write_custom_geom_only=True> <HE_angle_sigma_scale> <HE_sigma> <parallelity_sigma>

If a user specified geometry file is large, then run:

```
% phenix.cryo_fit2 &lt;initial_model&gt; &lt;target_map&gt; &lt;map resolution in Angstrom&gt; &lt;write_custom_geom_only=True&gt;
```

For example: % phenix.cryo_fit2 model.pdb map.ccp4 3.5 write_custom_geom_only=True HE_angle_sigma_scale=0.04 HE_sigma=0.2 parallelity_sigma=0.02 Then, automatically generated geometry file should be generated at the same folder. Modify it for a user's need. Other than write_custom_geom_only, all parameters are optional (if a user didn't specify these, cryo_fit2 will use default values).

For example:

```
% phenix.cryo_fit2 model.pdb map.ccp4 3.5 write_custom_geom_only=True HE_angle_sigma_scale=0.04 HE_sigma=0.2 parallelity_sigma=0.02
```

Then, automatically generated geometry file should be generated at the same folder.

Modify it for a user's need.

Other than write_custom_geom_only, all parameters are optional (if a user didn't specify these, cryo_fit2 will use default values).

- Once a user generated an eff file (by any method), then provide it to cryo_fit2 like: % phenix.cryo_fit2 <initial_model> <target_map> <map resolution in Angstrom> <eff file>

Once a user generated an eff file (by any method), then provide it to cryo_fit2 like:

```
% phenix.cryo_fit2 &lt;initial_model&gt; &lt;target_map&gt; &lt;map resolution in Angstrom&gt; &lt;eff file&gt;
```

For example: % phenix.cryo_fit2 model.pdb map.ccp4 3.5 user_custom_geom.eff

- For example: % phenix.cryo_fit2 model.pdb map.ccp4 3.5 user_custom_geom.eff

For example:

```
% phenix.cryo_fit2 model.pdb map.ccp4 3.5 user_custom_geom.eff
```

Output as a static structure

A final cryo_fitted structure: output/cryo_fit2_fitted.pdb

How to show map contour as mesh

In pymol commandline:

```
% load ./user_map.ccp4, map1, 1 , ccp4
% isomesh mesh1, map1, 1.5, (all), 0, 1, 3.0
```



```
% cmd.color("blue", "mesh1")
```

Otherwise, use pymol GUI:

```
% A (action) -&gt; Mesh -&gt; @ Level 3.0
```

How to generate and record movie?

Generate_record_movie_by_cryo_fit2

FAQ

See the cryo_fit2 FAQ

Options

All options will be used as default if unspecified. e.g. secondary_structure_enabled = True

List of most useful options

Install Cryo_fit2

cryo_fit2_install.html

Contents

- Note
- 1. Install Phenix
- 2. Link cryo_fit2 to phenix
- 3. Troubleshooting
- 4. Test run

Note

cryo_fit2 is under active development.

Please consider to use cryo_fit1 instead.

1. Install Phenix

See the installation notes for Phenix

2. Link cryo_fit2 to phenix

At <user phenix>/modules

```
git clone git@github.com:cryoFIT/cryo_fit2.git
```

At any folder,

```
libtbx.configure cryo_fit2
```

3. Troubleshooting

- If user's OS doesn't have git install git
- install git
- If a user sees the error message during git clone, "Permission denied (publickey).fatal: Could not read from remote repository. Please make sure you have the correct access rights and the repository exists.", user's computer needs to be registered to his/her github account. Therefore, make github account (free), Generate a new SSH key and Add a new SSH key to your GitHub account
- user's computer needs to be registered to his/her github account. Therefore, make github account (free), Generate a new SSH key and Add a new SSH key to your GitHub account
- If user's macOS shows, "xcode-select: note: no developer tools were found at '/Applications/Xcode.app', requesting install. Choose an option in the dialog to download the command line developer tools." download commandline as instructed (this may take ~7 minutes)
- download commandline as instructed (this may take ~7 minutes)
- "-bash: libtbx.configure: command not found" (in commandline) export
PATH="/Users/.../phenix-xxxx/build/bin":\$PATH source /Users/.../phenix-xxxx/phenix_env.sh
libtbx.configure cryo_fit2

- (in commandline)
- export PATH="/Users/.../phenix-xxxx/build/bin":\$PATH
- source /Users/.../phenix-xxxx/phenix_env.sh
- libtbx.configure cryo_fit2

4. Test run

See the cryo_fit2 tutorial

How to generate and record movie by cryo_fit2?

cryo_fit2_movie.html

Contents

- Output as a movie
- How to save a movie file (confirmed with MacPymol 2.2.2)
- Note

Output as a movie

- When an record_states option is True, structures of intermediate steps that can be animated with pymol play button are save as output/all_states.pdb
- Open this pdb file with pymol, then click play button (triangle at right bottom) plays the movie (confirmed with MacPymol 2.2.2).

(note. A below difference is observed with MacPymol ver. 2.2.2) If a user dragged all_states.pdb into pymol after opening it, the biomolecule structure looks simplified and movie playing is fast. If a user dragged all_states.pdb into pymol icon, pymol opens along with all_states.pdb, the biomolecule structure is more detailed, and movie playing is slow.

- (note. A below difference is observed with MacPymol ver. 2.2.2)

If a user dragged all_states.pdb into pymol after opening it, the biomolecule structure looks simplified and movie playing is fast. If a user dragged all_states.pdb into pymol icon, pymol opens along with all_states.pdb, the biomolecule structure is more detailed, and movie playing is slow.

- If a user dragged all_states.pdb into pymol after opening it, the biomolecule structure looks simplified and movie playing is fast.
- If a user dragged all_states.pdb into pymol icon, pymol opens along with all_states.pdb, the biomolecule structure is more detailed, and movie playing is slow.

How to save a movie file (confirmed with MacPymol 2.2.2)

As a user is playing a movie (by clicking a bottom right triangle movie playing button),

File -> Export Movie As -> MPEG

Note

- If a number MD steps is too many, memory requirement is too much.

For example, 3k atoms molecules (like L1 stalk in a ribosome) required more than 160 GB of memory.

- For example, 3k atoms molecules (like L1 stalk in a ribosome) required more than 160 GB of memory.

Tutorial: Fitting Biomolecules into Cryo-EM Maps using MD Simulation (command-line)

[cryo_fit_cmdline.html](#)

Contents

- Overview
- Input files
- Basic command-line syntax
- Running the tutorial with provided structure and map
- Output
- Example
- Limitation
- How to generate and record movie?
- Options
- List of the most useful options
- List of all available keywords

Overview

This tutorial will show you how to fit biomolecular structures into cryo-EM maps using molecular dynamics simulation with Cryo_fit under the PHENIX command-line environment.

For GUI tutorial, please see the [cryo_fit_gui_tutorial](#).

Theoretical explanation of Cryo_fit is [here](#).

For Cryo_fit installation, please see installation notes for [cryo_fit](#).

Input files

<initial_model> and <target_map>

Initial Model: .cif or .pdb

The initial model is a guide or template structure (CIF/mmCIF/pdb) that is close to the model representing the target cryo EM map structurally.

Target Map: .ccp4, .map/.mrc (MRC style binary file), or .sit (Situx style text file)

Basic command-line syntax

- `phenix.cryo_fit <initial_model> <target_map>`

For example, `phenix.cryo_fit model.pdb map.ccp4`

- For example, `phenix.cryo_fit model.pdb map.ccp4`

Running the tutorial with provided structure and map

- Use the map and model files in \$PHENIX/modules/cryo_fit/tutorial_input_files

Command:

```
phenix.cryo_fit GTPase_activation_center_tutorial.pdb GTPase_activation_center_tutorial_gaussian_1p5.mrc
```

Output

The "Output" folder has all the useful output files

Cryo_fit saves 3 structures with the highest correlation coefficients (CC):

cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb extracted_<x>_ps.pdb
extracted_<y>_ps.pdb

- cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb
- extracted_<x>_ps.pdb
- extracted_<y>_ps.pdb

These are

The final cryo_fitted structure (The PDB file with the highest CC):

cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb A <x>.gro file is for vmd visualization. A geometry extracted PDB file at certain (x) pico second during MD simulation:
extracted_<x>_ps.pdb

- The final cryo_fitted structure (The PDB file with the highest CC):

cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb

- cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb
- A <x>.gro file is for vmd visualization.
- A geometry extracted PDB file at certain (x) pico second during MD simulation:

extracted_<x>_ps.pdb

- extracted_<x>_ps.pdb

If Cryo_fit cannot find a structure with higher CC than that of the starting model, the input PDB file will exist in the output folder.

Also, the history of the CCs between cryo_fitted structures and the cryo-EM map is stored in the "cc_record" file.

- In summary, the PDB file with the highest structure-map CC after a successful run is

cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb

- cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobit.pdb
- For structure samples among those with top-ranked CCs, use

extracted_<x>_ps.pdb

- extracted_<x>_ps.pdb

Example

Limitation

Gromacs4.5.5 does not handle water molecules in structure files. Cryo_fit will remove any water from the input coordinate before starting.

Cryo_fit doesn't handle less common monomers such as 7C4, BMA, GDP, ILX, NAG, SEP, TRX. It will simply exclude those monomers and report what are excluded.

How to generate and record movie?

Generate_record_movie_by_cryo_fit1

Options

Default options will be used if unspecified. Expert users of Gromacs may want to customize some options if needed.

List of the most useful options

List of all available keywords

{{phil:cryo_fit.command_line.run}}

Tutorial: Fitting Biomolecules into Cryo-EM Maps using MD Simulation (GUI)

cryo_fit_gui.html

Contents

- Overview
- Input files
- Launch cryo_fit
- Enter Input Files
- Specify cryo_fit executable location
- Enter Options
- Run
- Result
- Output Example
- Limitation
- Reference
- How to generate and record movie?
- List of most useful options

Overview

This tutorial will show you how to fit biomolecular structures into cryo-EM maps with molecular dynamics (MD) simulation using Cryo_fit in the PHENIX graphical user interface (GUI).

For command-line tutorial, please see the [cryo_fit_commandline_tutorial](#)

Theoretical explanation of cryo_fit is [here](#)

For Cryo_fit installation, please see the [installation notes for cryo_fit](#)

Input files

<initial_model> and <target_map>

Initial Model: .cif or .pdb

The initial model is a guide or template structure (CIF/mmCIF/pdb) that is close to the model representing the target cryo EM map structurally.

Target Map: .ccp4, .map/.mrc (MRC style binary file), or .sit (Situs style text file)

Launch cryo_fit

Enter Input Files

Click browse buttons.

(Tutorial input files are in \$PHENIX/modules/cryo_fit/tutorial_input_files)

Specify cryo_fit executable location

(This step is optional if you've already setup your Cryo_fit environment in your .cshrc/.bashrc as instructed during Cryo_fit installation.)

Click browse buttons.

(select for example, \$PHENIX/modules/cryo_fit/gromacs/bin)

Enter Options

All options can be left blank (cryo_fit will figure out all options automatically).

Run

It should start something like this (total steps are 1~8).

Running time (With 2.7 GHz CPU, macOS)

All default values, it took ~2 minutes. With 10k steps, it took ~9 minutes

Result

Output files are in steps/8_cryo_fit folder. These include --

The final fitted atomic model: cryo_fitted.pdb

The .gro and .pdb files from the structures with the highest 3 map-model correlation coefficients (CC):
extracted_x_steps_x_ps.gro/pdb (.gro files are for visualization with vmd)

The history of CC values: steps/8_cryo_fit/cc_record

Output Example

Limitation

Gromacs4.5.5 does not handle water molecules in structure files. Cryo_fit will remove any water from the input coordinate before starting.

Cryo_fit doesn't handle less common monomers such as 7C4, BMA, GDP, ILX, NAG, SEP, TRX. It will simply exclude those monomers and report what are excluded.

Reference

S. Kirmizialtin, J. Loerke, E. Behrmann, C. MT. Spahn, K. Y Sanbonmatsu, Using Molecular Simulation to Model High-Resolution Cryo-EM Reconstructions, Methods Enzymol., 558, 2015, 497-514

How to generate and record movie?

Generate_record_movie_by_cryo_fit1

List of most useful options

Install Cryo_fit

cryo_fit_install.html

Contents

- [prerequisite] Install Phenix
- 1. Download Cryo_fit installer
- 2. Install Cryo_fit
- 4. Run Cryo_fit
- 6. Test run (optional)

[prerequisite] Install Phenix

Cryo_fit will be installed in your Phenix environment. Make sure you have Phenix installed and sourced. For Phenix installation, see installation notes for Phenix

1. Download Cryo_fit installer

Cryo_fit can be run on Linux or macOS computers. Please download Cryo_fit installer for Linux or Cryo_fit installer for macOS

2. Install Cryo_fit

Unzip the installer file, change to the extracted directory, and type `./install<enter>`. That's it. Please ensure that your account has 'write' privilege in your Phenix directory, or try `'sudo -E ./install'` if you have sudo access.

As a sequence of commands to execute (assuming the installer file is "Cryo_fit-Linux.zip"):

```
unzip Cryo_fit-Linux.zip
cd Cryo_fit-Linux
./install
```

At the end of installation, please follow the prompt to setup the path before starting Cryo_fit.

4. Run Cryo_fit

If you are in a Phenix session, please close and re-launch Phenix. You are now ready to run Cryo_fit in GUI or in command line.

6. Test run (optional)

You may want to know more about Cryo_fit. Try a test run! See the cryo_fit tutorial

How to generate and record movie by cryo_fit1?

cryo_fit_movie.html

Contents

- Pymol (confirmed with MacPymol 2.2.2)
- UCSF Chimera (confirmed with version 1.13 and 1.14)
- UCSF ChimeraX (confirmed with version 0.7 and 0.8 (2018-08-14))
- VMD (confirmed with version 1.9.3)
- Movie examples that are generated by cryo_fit
- Note

Pymol (confirmed with MacPymol 2.2.2)

- [command] cd output/trajectory
- [command] load trajectory.gro, mytraj
- [command] load_traj traj.xtc, mytraj
- Click salmon color play button (triangle) in right bottom to play

reference

UCSF Chimera (confirmed with version 1.13 and 1.14)

- [menu] Tools -> MD/Ensemble analysis -> MD Movie -> Trajectory format = GROMACS
- Run input (.tpr) = output/trajectory/for_cryo_fit.tpr
- Trajectory (.xtc) = output/trajectory/traj.xtc
- OK
- (optional)

[menu] Tools -> Depiction -> Rainbow -> OK [menu] File -> Open -> .../user.mrc (set opacity into 50%) [menu] Presets -> Publication 1

- [menu] Tools -> Depiction -> Rainbow -> OK
- [menu] File -> Open -> .../user.mrc (set opacity into 50%)
- [menu] Presets -> Publication 1
- (in MD Movie) click black play button to play
- (in MD Movie) File -> Record Movie, to record as .mp4 file

UCSF ChimeraX (confirmed with version 0.7 and 0.8 (2018-08-14))

- [menu] File -> Open -> .../output/cryo_fitted_chain_recovered_cleaned_for_real_space_refine_molprobability.pdb
- [command] rainbow
- [menu] File -> Open -> .../user.map (set opacity into 50%)
- [command] cd <user_path>/output/trajectory

- [command] open traj.xtc structureModel #1

(note) Specified structure and coordinates should have the same number of atoms, if these are mismatched, Doo Nam recommends to use UCSF Chimera instead

- [command] coordset slider #1
- Click black play button to play
- Click red record button to record movie as .mp4 file

VMD (confirmed with version 1.9.3)

- [menu] File -> New Molecule -> Browse -> (.../user.map) -> Load
- [menu] Graphics -> Representation -> Draw -> Solid Surface
- [menu] Graphics -> Representation -> Material -> Transparent
- [menu] Graphics -> Representation -> Show -> Isosurface
- [menu] File -> New Molecule -> Browse -> (output/trajectory/trajectory.gro) -> Load

Movie examples that are generated by cryo_fit

- Watch supplementary videos in Journal of Structural Biology, Vol 208, 2019, p 1-6, Cryo_fit: Democratization of flexible fitting for cryo-EM

Note

- If the cryo_fit has been automatically repeated to improve cc, the trajectory file shows only the last cryo_fit run.
- Therefore, to watch a full change of conformations, "phenix.cryo_fit user.mrc user.pdb no_rerun=True number_of_steps_for_cryo_fit=<enough step>" is recommended.

Tutorial: Generating ligand coordinates and restraints for structure refinement

elbow.html

Introduction

This tutorial will demonstrate how to perform a structure refinement including a ligand.

Setup

You can create a subdirectory, 'elbow_tutorial', and copy the files required for this tutorial using the command:

```
elbow.setup_tutorial
```

When do we need restraints for a ligand?

A structure refinement requires that all atoms have energy types associated and ideally restraints to guide the geometry optimisation. PHENIX refinement uses a monomer library to resolve some ligands but not all.

Running on a structure containing an unknown ligand:

```
phenix.refine 1utn.pdb 1utn.mtz
```

results in the following error message.

```
Number of atoms with unknown nonbonded energy type symbols: 16
" N ABN A 246 "
" C ABN A 246 "
" C1 ABN A 246 "
" C2 ABN A 246 "
" C3 ABN A 246 "
" C4 ABN A 246 "
" C5 ABN A 246 "
" C6 ABN A 246 "
" N ABN A 247 "
" C ABN A 247 "
... (remaining 6 not shown)
```

Generating ligand restraints using PDB file

Clearly, we need to generate restraints for the ligand ABN which is in the PDB file. We commence by using the following command:

```
phenix.elbow 1utn.pdb
```

which prints out a list of the HETATM molecules in the PDB file.

```
Running eLBOW in expert mode
Defaults changed by /net/cci/nigel/.elbow/elbow_defaults.py
xyz : True (--no-xyz to reverse)
overwrite : True (--no-overwrite to reverse)
Random number seed: 578305348
0:00 Parsing Parsing Parsing Parsing Parsing Parsing Parsing Parsing P

Input format is PDB

MoleculeClass : C:11 N: 2 O: 3 (PDB format)
16 atoms
15 bonds
0 angles
```

```
0 dihedrals
```

```
Multiple residue names
resSeq resName #atoms type
246 'ABN' 8 HETATM
256 'TRS' 8 HETATM
```

Use `--residue` to specify residue

Use `--do-all` to process all residues not in internal library

As the print out says, all of the residues not in the internal library can be processed using

```
phenix.elbow 1utn.pdb --do-all
```

Because the residue TRS is in the internal library, only ABN is processed. The resulting files are `elbow.ADB.1utn_pdb.cif` and `elbow.ABN.1utn_pdb.pdb`. After checking the geometry in the PDB file, we repeat the calculation using the `--opt` to optimise the geometry using the semi-empirical AM1 method, thus:

```
phenix.elbow 1utn.pdb --do-all
```

The resulting file, `elbow.1utn_pdb.all.cif`, is ready for use with `phenix.refine`.

Generating ligand restraints using SMILES

To demonstrate the use of a SMILES string, we can get the SMILES string for ABN from the MSDChem website. We can now run using the SMILES string:

```
phenix.elbow --smiles "NCC1CCCCC1"
```

The resulting output, `elbow.cif` and `elbow.pdb`, have the default residue name, `LIG`, and atom names. In order to match the atoms in the refinement, we can use the PDB to obtain the atom names via the `--template` option and the `--residue` option:

```
phenix.elbow --smiles "NCC1CCCCC1" --template 1utn.pdb --residue ABN --opt
```

The resulting output files have the AM1 optimised geometry (`--opt`), the atom names from the PDB file and the correct connectivity from the SMILES string.

Generating link edits

The input PDB file, `1utn.pdb`, has a number of LINK records. These can be converted to "edits" for input into `phenix.refine`. The LINK records:

```
LINK CA CA A6000 OE1 GLU A 70 1555 1555
LINK CA CA A6000 O ASN A 72 1555 1555
LINK CA CA A6000 O VAL A 75 1555 1555
LINK CA CA A6000 O HOH Z 89 1555 1555
LINK CA CA A6000 O HOH Z 104 1555 1555
LINK CA CA A6000 OE2 GLU A 80 1555 1555
```

Running:

```
phenix.link_edits 1utn.pdb
```

results in the "edits" being written to `1utn_pdb.edits`. The "edits" are a one-to-one correspondence of the LINK records and a bond "edit".

Metal coordination

Conveniently, the LINK records were contained in the PDB file used in this example. More likely, the linking information is not available. The "edits" can be obtained using:

```
phenix.metal_coordination 1utn.pdb
```

The "edits" are written to elbow.edits and contain both bond and angle "edits".

Putting it all together

We can put all the files together to perform a refinement. Adding the option --dry-run reads the data. Remove to perform complete refinement.

```
phenix.refine 1utn.mtz 1utn.pdb elbow.1utn_pdb.all.cif 1utn_pdb.edits --dry-run
```

Tutorial: Fitting a flexible ligand into a difference electron density map

ligandfit.html

Overview

This tutorial describes how to use the LigandFit wizard to automatically place an ATP molecule in a difference map. An older tutorial covering use of LigandFit on the command line is still available.

Setup

The dataset for this tutorial is "N-ethylmaleimide sensitive factor + ATP", which can be automatically created as a project in the Phenix GUI. Three files are included: a ligand-free model, experimental data (MTZ file), and the ligand itself. Once you have created the project, open the LigandFit GUI under the "Ligands" category. The files can be added to the input list by dragging them from the desktop, or by clicking the "Add file" button and selecting them from the browser window.

The space group and unit cell will be automatically loaded from the data file, and suitable column labels added to the "Input labels" menu. In this case, we only have amplitudes available, so these will be used as input, and LigandFit will calculate an Fo-Fc map using model phases. However, in many cases you will obtain optimal results by using an mFo-DFc map such as those output by phenix.refine or phenix.maps.

For this example, the rest of the parameters can be left on default settings; consult the full documentation for explanation of what these mean. The only exception is "Number of processors"; if you have multiple processor cores available, you can set this higher. Note that since the ligand structure has already been characterized in the PDB, we could simply enter the three-letter residue name ("ATP") under "Ligand code", instead of supplying the ATP as a PDB file. However, this will increase the runtime, since eLBOW will be run to generate coordinates.

Running the program

Click "Run" on the toolbar to launch LigandFit. A new tab will appear showing the log output:

LigandFit will start several child processes to sample a wide variety of ligand conformations. The "Parallel jobs" tab will show the output of the individual runs as they progress; the best result will be selected for real-space refinement and be copied to the main output directory.

At the end of the run, the summary tab will list the essential output files and the statistics for the final result, and activate the buttons to launch other programs.

Clicking "Open in Coot" will start Coot and load the difference map, starting model, and ligand structure. Click on the result in the summary tab to zoom in on the ligand in Coot. Further refinement is essential, but we can see that the fit is already very good. (The excess difference density near the phosphate atoms belongs to an unmodeled magnesium ion and coordinating waters.)

References

Automated ligand fitting by core-fragment fitting and extension into density. T.C. Terwilliger, H. Klei, P.D. Adams, N.W. Moriarty, and J.D. Cohn. Acta Crystallogr D Biol Crystallogr 62, 915-22 (2006).

- Automated ligand fitting by core-fragment fitting and extension into density. T.C. Terwilliger, H. Klei, P.D. Adams, N.W. Moriarty, and J.D. Cohn. *Acta Crystallogr D Biol Crystallogr* 62, 915-22 (2006).

Tutorial: Fitting a flexible ligand into a difference electron density map

ligandfit_cmdline.html

Introduction

This is a tutorial for running LigandFit on the command line; for GUI usage, see the separate tutorial. This tutorial will start with experimental data and a model (1J4R.pdb) with ligand removed (1J4R_no_ligand.pdb) and a randomized ligand conformation (1J4R_random.pdb), and will fit the ligand into difference density calculated from the experimental data and partial model. The tutorial is designed to be read all the way through, giving pointers for you along the way. Once you have read it all and run the example data and looked at the output files, you will be in a good position to run your own data through LigandFit.

Setting up to run PHENIX

If PHENIX is already installed and your environment is all set, then if you type:

```
echo $PHENIX
```

then you should get back something like this:

```
/xtal/phenix-1.3
```

If instead you get:

```
PHENIX: undefined variable
```

then you need to set up your PHENIX environment. See the PHENIX installation page for details of how to do this. If you are using the C-shell environment (csh) then all you will need to do is add one line to your .cshrc (or equivalent) file that looks like this:

```
source /xtal/phenix-1.3/phenix_env
```

(except that the path in this statement will be where your PHENIX is installed). Then the next time you log in \$PHENIX will be defined.

Running the demo 1J4R-ligand data with LigandFit

To run LigandFit on the demo 1J4R-ligand data, make yourself a tutorials directory and cd into that directory:

```
mkdir tutorials
cd tutorials
```

Now type the phenix command:

```
phenix.run_example --help
```

to list the available examples. Choosing 1J4R-ligand for this tutorial, you can now use the phenix command:

```
phenix.run_example 1J4R-ligand
```

to build the 1J4R-ligand structure with LigandFit. This command will copy the directory \$PHENIX/examples/1J4R-ligand to your current directory (tutorials) and call it tutorials/1J4R-ligand/. Then it will run LigandFit using the command file run.sh that is present in this tutorials/1J4R-ligand/ directory.

This command file run.sh is simple. It says:

```
#!/bin/sh
echo "Running LigandFit on 1J4R data..."
phenix.ligandfit data=1J4R.mtz model=1J4R_no_ligand.pdb ligand=1J4R_random.pdb
```

The first line (#!/bin/sh) tells the system to interpret the remainder of the text in the file using the sh (or bash) -shell (sh).

The command phenix.ligandfit runs the command-line version of LigandFit (see Automated Structure Solution using LigandFit for all the details about LigandFit including a full list of keywords).

The arguments on the command line tell LigandFit about the data file (data=1J4R.mtz), the model without ligand (model=1J4R_no_ligand.pdb), and the ligand (ligand=1J4R_random.pdb). (Note that each of these is specified with an = sign, and that there are no spaces around the = sign.)

The structure factor amplitudes are in the datafile 1J4R.mtz. This is an mtz file which is a binary file that contains summary information about the dataset as well as the reflection data.

Although the phenix.run_example 1J4R-ligand command has just run LigandFit from a script (run.sh), you can run LigandFit yourself from the command line with the same phenix.ligandfit data= ... command. You can also run LigandFit from a GUI, or by putting commands in another type of script file. All these possibilities are described in Using the PHENIX Wizards.

Where are my files?

Once you have started LigandFit or another Wizard, an output directory will be created in your current (working) directory. The first time you run LigandFit in this directory, this output directory will be called LigandFit_run_1_ (or LigandFit_run_1_/, where the slash at the end just indicates that this is a directory). All of the output from run 1 of LigandFit will be in this directory. If you run LigandFit again, a new subdirectory called LigandFit_run_2_ will be created.

Inside the directory LigandFit_run_1_ there will be one or more temporary directories such as TEMP0 created while the Wizard is running. The files in this temporary directory may be useful sometimes in figuring out what the Wizard is doing (or not doing!). By default these directories are emptied when the Wizard finishes (but you can keep their contents with the command clean_up=False if you want.)

What parameters did I use?

Once the LigandFit wizard has started (when run from the command line), a parameters file called ligandfit.eff will be created in your output directory (e.g., LigandFit_run_1_/ligandfit.eff). This parameters file has a header that says what command you used to run LigandFit, and it contains all the starting values of all parameters for this run (including the defaults for all the parameters that you did not set).

The ligandfit.eff file is good for more than just looking at the values of parameters, though. If you copy this file to a new one (for example ligandfit_lores.eff) and edit it to change the values of some of the parameters (resolution=3.0) then you can re-run LigandFit with the new values of your parameters like this:

```
phenix.ligandfit ligandfit_lores.eff
```

This command will do everything just the same as in your first run but use only the data to 3.0 Å.

Reading the log files for your LigandFit run file

While the LigandFit wizard is running, there are several places you can look to see what is going on. The most important one is the overall log file for the LigandFit run. This log file is located in:

```
LigandFit_run_1_/LigandFit_run_1_1.log
```

for run 1 of LigandFit. (The second 1 in this log file name will be incremented if you stop this run in the middle and restart it with a command like phenix.ligandfit run=1).

The LigandFit_run_1_1.log file is a running summary of what the LigandFit Wizard is doing. Here are a few of the key sections of the log files produced for the 1J4R-ligand SAD dataset.

Summary of the command-line arguments

Near the top of the log file you will find:

```
-----  
Starting LigandFit with the command:  
  
phenix.ligandfit data=1J4R.mtz model=1J4R_no_ligand.pdb \  
ligand=1J4R_random.pdb
```

This is just a repeat of how you ran LigandFit; you can copy it and paste it into the command line to repeat this run.

Fitting the ligand into difference density with RESOLVE ligand fitting

The LigandFit Wizard will use the partial model 1J4R_no_ligand.pdb to calculate FC and PHIC (model amplitudes and phases) for the structure factors in 1J4R.mtz. Then these are used to calculate a simple (FO-FC) exp(iPHIC) difference map which is written to resolve_map.mtz.

Then the RESOLVE algorithm for ligand fitting is used to fit your ligand 1J4R_random.pdb into this map. To make the fitting process suitable for parallelization, the fitting is done in a series of tries, each considering more possible rotations of the ligand and each carrying out a more exhaustive search than the previous one. Then depending on how many processors are available, they are run one at a time or in parallel.

Here is the summary of the five attempts at fitting the 1J4R difference map:

```
Fitting ligand #1 from /net/cci-filer1/vol1/tmp/terwill/phenix_examples/1J4R-ligand/1J4R_random.pdb  
Ligand file has 45 atomsSearching for ligand number 1  
Estimated ligand volume: 185.0 A**3  
Starting CC of ligand as input to map: 0.31  
Starting local CC of ligand as input to map: 0.18USING LOCAL CC: 0.18  
Setting background=False as nproc=1  
Try: 1 quickTrying fit with n_indiv_tries_min = 5  
  n_indiv_tries_max = 10  
  n_group_search = 3  
  ligand_file = 1J4R_random.pdb  
Try: 2 thorough  
Trying fit with n_indiv_tries_min = 100  
  n_indiv_tries_max = 100  
  n_group_search = 4  
  ligand_file = 1J4R_random.pdb  
Try: 3 randomize  
Randomizing ligand conformation before starting fit  
Randomized version of /net/cci-filer1/vol1/tmp/terwill/phenix_examples/1J4R-liga  
nd/1J4R_random.pdb placed in random.pdb  
Trying fit with n_indiv_tries_min = 100  
  n_indiv_tries_max = 100  
  n_group_search = 4  
  ligand_file = random.pdb  
Try: 4 extra_thorough  
Trying fit with n_indiv_tries_min = 300  
  n_indiv_tries_max = 300  
  n_group_search = 6  
  ligand_file = 1J4R_random.pdb  
Try: 5 extra_thorough_randomize  
Randomizing ligand conformation before starting fit
```

```

Randomized version of
/net/cci-filer1/vol1/tmp/terwill/phenix_examples/1J4R-ligand/1J4R_random.pdb
placed in random.pdb
Trying fit with n_indiv_tries_min = 300
  n_indiv_tries_max = 300
  n_group_search = 6
  ligand_file = random.pdb
Running 5 jobs on 1 processors
Splitting work into 5 jobs and running 1 at a time with sh in
/net/cci-filer1/vol1/tmp/terwill/phenix_examples/1J4R-ligand/LigandFit_run_1/TEMP0

Starting job 1...
Starting job 2...
Starting job 3...
Starting job 4...
Starting job 5...

Solution for try : 1
  SCORE= 123.868 CC= 0.650 LIGANDS= 1 LIG= ligand_fit_1_1.pdb TEMPLATE=1J4R_random.pdb

Solution for try : 2
  SCORE= 128.528 CC= 0.700 LIGANDS= 1 LIG= ligand_fit_1_1.pdb TEMPLATE=1J4R_random.pdb

Solution for try : 3
  SCORE= 128.170 CC= 0.710 LIGANDS= 1 LIG= ligand_fit_1_1.pdb TEMPLATE=random.pdb

Solution for try : 4
  SCORE= 129.750 CC= 0.720 LIGANDS= 1 LIG= ligand_fit_1_1.pdb TEMPLATE=1J4R_random.pdb

Solution for try : 5
  SCORE= 129.574 CC= 0.720 LIGANDS= 1 LIG= ligand_fit_1_1.pdb TEMPLATE=random.pdb

```

In this case the best fits were found on tries 4 and 5 (the ones with the most thorough fitting attempts).

The log file for the best fit is in `ligand_fit_1_1.pdb` (the first 1 refers to the solution number, and the second 1 refers to the ligand number, as you have the option of finding more than one ligand in the map with `number_of_ligands=n`). This log file will describe the analysis of the ligand and the rigid parts of the ligand, the search for locations of rigid parts of the ligand, and the extension of those rigid parts into density to complete the ligand.

In these runs the parameters `n_indiv_tries_min=5` and `n_indiv_tries_max=10` are the minimum and maximum number of placements of each rigid part of the ligand to test for completing the ligand. The parameter `n_group_search=3` is the number of different rigid parts of the ligand to try. On subsequent tries at fitting these will all be increased to search more thoroughly.

This fitting of the ligand was able to place all 45 atoms in the ligand, with a correlation to the map in the region of the ligand of 0.72. The "LOCAL CC" is the correlation of the model density with map density in the region occupied by the ligand itself, plus any contiguous points with high difference electron density that are connected to the region occupied by the ligand. This local CC is more useful at discriminating between correct and incorrect models than the standard correlation (which only includes the region occupied by the model.)

Note that the fit to the map has improved only slightly in this case over the initial fit found on the first try. Sometimes this is the case and sometimes a considerably better fit may be found by searching more thoroughly.

The LigandFit_summary.dat summary file

A quick summary of the results of your LigandFit run is in the `LigandFit_summary.dat` file in your output directory. This file lists the key files that were produced in your run of LigandFit (all these are in the output directory) and some of the key statistics for the run, including the overall correlation between the model and

the map and the number of copies of the ligand placed. Here is the summary for this 1J4R-ligand fitting run:

```
LIGAND SOLUTIONS FOR RUN 1 SORTED BY OVERALL CC

**** FILES ARE IN THE DIRECTORY: LigandFit_run_1_ ****

RANK SCORE CC COPIES FITTED/TOTAL NAME TEMPLATE
RMSD CC_START LIG_VOLUME

1 129.750 0.720 1 45 / 45 ligand_fit_1_1.pdb 1J4R_random.pdb
0.00 0.18 185.00
```

How do I know if ligand fitting worked?

Here are some of the things to look for to tell if you have obtained a good ligand model:

- What is the overall correlation between the ligand and the difference density map? For a well-fitted ligand this is normally 0.70 or higher. Correlations below 0.5 are poor.
- How well does the ligand fit the difference density? Have a look at the ligand and the difference density map in a graphics program. Does the ligand conformation match the density? Add the model of the macromolecule now. Does the ligand make plausible interactions with the macromolecule?

What to do next

Once you have run LigandFit and have obtained a good ligand model, you will want to refine the model. Add the model to your PDB file containing the remainder of the structure and refine the whole structure. You may need to define the geometry of your ligand. In that case the tool phenix.elbow in the PHENIX package will be most useful (it is quite automatic and generates all the files you need).

Once you have fitted your ligand(s), the next thing you might want to do is to re-run the AutoBuild Wizard, including your ligands with input_lig_file_list=myligand.pdb.

If you do not obtain a good fit of your ligand to the map, there are a few things you should check:

- Does your starting model have solvent molecules where the ligand needs to be placed? If so, then you will need to remove these before LigandFit can fit the ligand there. The LigandFit Wizard excludes all locations that are occupied by atoms in the starting model.
- Have a careful look at all the output files. Work your way through the main log file (e.g., LigandFit_run_1_1.log). Is there anything strange or unusual in any of them that may give you a clue as to what to try next?
- Have a look at the difference electron density map. Does it have clear density for the ligand? If not, you may want to try other types of maps. You can apply phenix.refine to your data file and your model without ligand and it will create a map_coeffs.mtz output file that has sigmaA-weighted map coefficients that should optimally show the placement of your ligand. If this map looks better than the resolve_map.mtz, then you can read in the coefficients like this, where the command lig_map_type=fo-fc_difference_map tells the wizard to read in the map coefficients directly from the data file:

```
phenix.ligandfit data=map_coeffs.mtz lig_map_type=fo-fc_difference_map \
model=partial.pdb ligand=side.pdb
```

Additional information

For details about the LigandFit Wizard, see Automated Ligand Fitting using LigandFit. For help on running Wizards, see Using the PHENIX Wizards.

Tutorial: Solving a structure with MAD data

mad_phasing.html

Introduction

This tutorial will use some moderately good MAD data (3 wavelengths from a gene-5 protein SeMet dataset diffracting to 2.6 Å) as an example of how to solve a MAD dataset with AutoSol. It is designed to be read all the way through, giving pointers for you along the way. Once you have read it all and run the example data and looked at the output files, you will be in a good position to run your own data through AutoSol.

Setting up to run PHENIX

If PHENIX is already installed and your environment is all set, then if you type:

```
echo $PHENIX
```

then you should get back something like this:

```
/xtal/phenix-1.3
```

If instead you get:

```
PHENIX: undefined variable
```

then you need to set up your PHENIX environment. See the PHENIX installation page for details of how to do this. If you are using the C-shell environment (csh) then all you will need to do is add one line to your .cshrc (or equivalent) file that looks like this:

```
source /xtal/phenix-1.3/phenix_env
```

(except that the path in this statement will be where your PHENIX is installed). Then the next time you log in \$PHENIX will be defined.

Running the demo gene-5 data with AutoSol

To run AutoSol on the demo gene-5 data, make yourself a tutorials directory and cd into that directory:

```
mkdir tutorials
cd tutorials
```

Now type the phenix command:

```
phenix.run_example --help
```

to list the available examples. Choosing gene-5-mad for this tutorial, you can now use the phenix command:

```
phenix.run_example gene-5-mad
```

to solve the gene-5 structure with AutoSol. This command will copy the directory \$PHENIX/examples/gene-5-mad to your current directory (tutorials) and call it tutorials/gene-5-mad/. Then it will run AutoSol using the command file run.sh that is present in this tutorials/gene-5-mad/ directory.

This command file run.sh is simple. It says:

```
#!/bin/sh
echo "Running AutoSol on gene-5 protein data..."
phenix.autosol gene-5-mad.eff
```

The first line (`#!/bin/sh`) tells the system to interpret the remainder of the text in the file using the `sh` (or `bash`) -shell (`sh`).

The command `phenix.autosol gene-5-mad.eff` runs the command-line version of AutoSol with the parameters in the file `gene-5-mad.eff` (see Automated Structure Solution using AutoSol for all the details about AutoSol including a full list of keywords).

The file `gene-5-mad.eff` is a typical PHENIX parameters file:

```
# parameters for autosol run with gene-5 3-wavelength MAD data
#
autosol {
  atom_type = Se
  sites = 2
  seq_file = sequence.dat
  crystal_info {
    space_group = C2
    unit_cell = 76.08 27.97 42.36 90 103.2 90
    resolution = 2.6
  }
  wavelength {
    data = high.sca
    lambda = 0.9600
    f_prime = -1.5
    f_double_prime = 3
  }
  wavelength {
    data = peak.sca
    lambda = 0.9792
    f_prime = -3
    f_double_prime = 4
  }
  wavelength {
    data = infl.sca
    lambda = 0.9798
    f_prime = -5
    f_double_prime = 2
  }
}
```

Notice how the brackets (`{` and `}`) work in this file. Everything in this file after the word "autosol" that is between the opening left-bracket (`{`) and the closing right-bracket (`}`) is part of the autosol "scope". The AutoSol wizard looks for "autosol { lots of parameters }" and interprets everything inside these brackets. Everything outside the scope "autosol" is ignored.

Within the autosol scope there are some keywords like "atom_type = Se", these are normally one per line.

There are also additional scopes, with keywords inside them. For example the `space_group` and `unit_cell` information are inside the scope "crystal_info".

The information about each wavelength of data is in a separate scope called "wavelength". You can have as many of these as you like. Within this scope you can define the datafile name, the wavelength (`lambda`), and `f_prime` and `f_double_prime` values for that wavelength.

The MAD data to be used to solve the structure is in the datafiles `peak.sca`, `infl.sca` and `high.sca`. These datafiles are in Scalepack premerged format, which means that there is just one instance of each reflection and the cell parameters are in the file, so we do not need to provide the cell parameters or the space group (unless the ones in the `.sca` files are incorrect!)

The resolution of the data is to about 2.6 Å.

Although the `phenix.run_example gene-5-mad` command has just run AutoSol from a script (`run.sh`), you can run AutoSol yourself from the command line with the same `phenix.autosol gene-5-mad.eff` command. You can also run AutoSol from a GUI. Running a Wizard from a GUI, the command-line.

Where are my files?

Once you have started AutoSol or another Wizard, an output directory will be created in your current (working) directory. The first time you run AutoSol in this directory, this output directory will be called `AutoSol_run_1_` (or `AutoSol_run_1_`, where the slash at the end just indicates that this is a directory). All of the output from run 1 of AutoSol will be in this directory. If you run AutoSol again, a new subdirectory called `AutoSol_run_2_` will be created.

Inside the directory `AutoSol_run_1_` there will be one or more temporary directories such as `TEMP0` created while the Wizard is running. The files in this temporary directory may be useful sometimes in figuring out what the Wizard is doing (or not doing!). By default these directories are emptied when the Wizard finishes (but you can keep their contents with the command `clean_up=False` if you want.)

What parameters did I use?

Once the AutoSol wizard has started (when run from the command line), a parameters file called `autosol.eff` will be created in your output directory (e.g., `AutoSol_run_1_/autosol.eff`). This parameters file has a header that says what command you used to run AutoSol, and it contains all the starting values of all parameters for this run (including the defaults for all the parameters that you did not set).

The `autosol.eff` file is good for more than just looking at the values of parameters, though. If you copy this file to a new one (for example `autosol_lores.eff`) and edit it to change the values of some of the parameters (`resolution=3.0`) then you can re-run AutoSol with the new values of your parameters like this:

```
phenix.autosol autosol_lores.eff
```

This command will do everything just the same as in your first run but use only the data to 3.0 Å.

Reading the log files for your AutoSol run file

While the AutoSol wizard is running, there are several places you can look to see what is going on. The most important one is the overall log file for the AutoSol run. This log file is located in:

```
AutoSol_run_1_/AutoSol_run_1_1.log
```

for run 1 of AutoSol. (The second 1 in this log file name will be incremented if you stop this run in the middle and restart it with a command like `phenix.autosol run=1`).

The `AutoSol_run_1_1.log` file is a running summary of what the AutoSol Wizard is doing. Here are a few of the key sections of the log files produced for the gene-5 MAD dataset.

Summary of the command-line arguments

Near the top of the log file you will find:

```
-----  
Starting AutoSol with the command:  
  
phenix.autosol  
  
Reading effective parameters from gene-5-mad.eff
```

This is just telling you how you ran AutoSol. Next comes a list of the values of all the parameters that were used to run AutoSol. It is the same as the file "autosol.eff" that is written to record these values. The beginning of this file looks like:

```
autosol {
  atom_type = "Se"
  lambda = None
  f_prime = None
  f_double_prime = None
  wavelength_name = peak inf high low remote
  sites = 2
  sites_file = None
  seq_file = "sequence.dat"
```

Here anything that is None was not set. Also anything that has a list of choices (wavelength_name in this case) is not set unless a "*" is next to one of the choices.

This first set of parameters are all keywords that apply to all datasets or are shortcuts for setting keywords that apply to a single dataset. For example "atom_type=Se" will set the atom_type for all wavelengths and all derivatives.

Later in the parameters file comes each wavelength:

```
wavelength {
  wavelength_name = peak inf high low remote
  data = "high.sca"
  labels = None
  atom_type = None
  lambda = 0.96
  f_prime = -1.5
  f_double_prime = 3
  sites = 2
  sites_file = None
  group = 1
  added_wavelength = False
}
```

Here are listed all the values that you have set for this wavelength. The group is a number you can set to identify what group of wavelengths/natives/derivatives this wavelength is a part of. That lets you enter MIR+SAD or any other combinations of datasets. The "added_wavelength" keyword is added by the wizard and just marks whether you specifically added this wavelength or not.

Reading the datafiles.

The AutoSol Wizard will read in your datafiles and check their contents, printing out a summary for each one:

```
HKLIN ENTRY: high.sca
FILE TYPE scalepack_merge
GUESS FILE TYPE MERGE TYPE sca premerged
LABELS['IPLUS', 'SIGIPLUS', 'IMINU', 'SIGIMINU']
Unit cell: (76.08, 27.97, 42.36, 90, 103.2, 90)
Space group: C 1 2 1 (No. 5)
CONTENTS: ['high.sca', 'sca', 'premerged', 'C 1 2 1',
[76.079999999999998, 27.969999999999999, 42.359999999999999, 90.0, 103.2, 90.0],
2.5940784397029653, ['IPLUS', 'SIGIPLUS', 'IMINU', 'SIGIMINU']]
Total of 3 input data files
['peak.sca', 'infl.sca', 'high.sca']
```

Guessing cell contents

The AutoSol Wizard uses the sequence information in your sequence file (sequence.dat) and the cell parameters and space group to guess the number of NCS copies and the solvent fraction, and the number of

total methionines (approximately equal to the number of heavy-atom sites for SeMet proteins):

```
AutoSol_guess_setup_for_scaling AutoSol Run 1 Thu Mar 6 21:43:20 2008
Solvent fraction and resolution and ha types/scatt fact
This is the last dataset to scale
Guessing setup for scaling dataset 1
SG C 1 2 1
cell [76.07999999999998, 27.969999999999999, 42.359999999999999, 90.0, 103.2, 90.0]
Number of residues in unique chains in seq file: 87
Unit cell: (76.08, 27.97, 42.36, 90, 103.2, 90)
Space group: C 1 2 1 (No. 5)
CELL VOLUME :87758.6787391
N_EQUIV:4
GUESS OF NCS COPIES: 1
SOLVENT FRACTION ESTIMATE: 0.46
Total residues:87
Total Met:2
resolution estimate: 2.59
```

Running phenix.xtriage

The AutoSol Wizard automatically runs phenix.xtriage on each of your input datafiles to analyze them for twinning, outliers, translational symmetry, and other special conditions that you should be aware of. You can read more about xtriage in [Data quality assessment with phenix.xtriage](#). Part of the summary output from xtriage for this dataset looks like this:

```
The largest off-origin peak in the Patterson function is 12.60% of the
height of the origin peak. No significant pseudotranslation is detected.
```

```
The results of the L-test indicate that the intensity statistics
behave as expected. No twinning is suspected.
```

Testing for anisotropy in the data

The AutoSol Wizard tests for anisotropy by determining the range of effective anisotropic B values along the principal lattice directions. If this range is large and the ratio of the largest to the smallest value is also large then the data are by default corrected to make the anisotropy small (see [Analyzing and scaling the data in the AutoSol web page](#) for more discussion of the anisotropy correction). In the gene-5 case, the range of anisotropic B values is small and no correction is made:

```
Range of aniso B: 24.58 27.92
Not using aniso-corrected data files as the range of aniso b is
only 3.43 and 'correct_aniso' is not set
```

Note that if any one of the datafiles in a MAD dataset has a high anisotropy, then by default all of them will be corrected for anisotropy.

Scaling MAD data and estimating FA values

The AutoSol Wizard uses SOLVE localscaling to scale MAD data. The procedure is basically to scale all the data to the most complete dataset, ignoring anomalous differences, to create a reference dataset. Then all F+ and F- observations at all wavelengths are scaled to this reference dataset, and then the data are merged to the asymmetric unit, averaging duplicate observations. During this process outliers that deviate from the reference values by more than ratio_out (default=1) standard deviations (using all data in the appropriate resolution shell to estimate the SD) are rejected.

After scaling, the values of f' and f'' are refined based on the relative values of anomalous differences at the various wavelengths and the relative values of dispersive differences among the data at different

wavelengths. Then FA values (estimates of the heavy-atom structure factor) are estimated. These FA values can often be more useful than the anomalous differences at any of the individual wavelengths because they combine the anomalous and dispersive information. At the same time as FA values are calculated, an estimate of the phase difference between the structure factor of the anomalously-scattering atoms and the structure factor corresponding to all other atoms can be estimated. This phase difference is useful later in calculating Fourier maps showing the positions of the anomalously-scattering atoms.

Choosing datafiles with high signal-to-noise

For MAD data the AutoSol Wizard analyzes the correlation of anomalous differences at the various wavelengths. The anomalous difference for a particular reflection is related to the f'' value at each wavelength. Consequently if the data are good then the anomalous differences at different wavelengths (but for the same reflections) are highly correlated. A shell of resolution in which the anomalous differences have a correlation of about 0.3 or greater has some useful information. A strong SeMet dataset will have an overall correlation of 0.6-0.7 for the peak and high energy remote wavelengths. You can see this analysis in the log file `dataset_scale_1.log` for this MAD dataset:

```
Correlation of anomalous differences at different wavelengths.
(You should probably cut your data off at the resolution where
this drops below about 0.3. A good dataset has correlation
between peak and remote of at least 0.7 overall. Data with
correlations below about 0.5 probably are not contributing much.)
```

```
CORRELATION FOR
WAVELENGTH PAIRS
DMIN 1 VS 2 1 VS 3 2 VS 3
```

```
5.18 0.79 0.89 0.73
3.88 0.68 0.75 0.55
3.63 0.68 0.72 0.46
3.43 0.53 0.61 0.41
3.24 0.51 0.58 0.26
3.11 0.51 0.59 0.36
2.98 0.36 0.54 0.13
2.85 0.50 0.45 0.35
2.72 0.28 0.30 0.10
2.59 0.32 0.23 0.14
```

```
ALL 0.55 0.66 0.40
```

During scaling, the AutoSol Wizard estimates the signal-to-noise in each datafile and the resolution where there is significant signal-to-noise (above 0.3:1 signal-to-noise). In this case, the FA's appear to have the highest signal-to-noise (3.1) and the inflection data the lowest (0.5):

```
FILE DATA:FA.sca sn: 3.078877
FILE DATA:peak.sca sn: 2.313605
FILE DATA:high.sca sn: 1.365164
FILE DATA:infl.sca sn: 0.3432194
order of datasets for trying phasing:['FA.sca', 'peak.sca', 'high.sca', 'infl.sca']
```

Running HYSS to find the heavy-atom substructure

The HYSS (hybrid substructure search) procedure for heavy-atom searching uses a combination of a Patterson search for 2-site solutions with direct methods recycling. The search ends when the same solution is found beginning with several different starting points. The HYSS log files are named after the datafile that they are based on and the type of differences (ano, iso) that are being used. In this gene-5 MAD dataset, the HYSS logfile is `peak.sca_ano_1.sca_hyss.log`. The key part of this HYSS log file is:

```
Entering search loop:
```

```

p = peaklist index in Patterson map
f = peaklist index in two-site translation function
cc = correlation coefficient after extrapolation scan
r = number of dual-space recycling cycles
cc = final correlation coefficient

p=000 f=000 cc=0.144 r=015 cc=0.280 [ best cc: 0.280 ]
p=000 f=001 cc=0.134 r=015 cc=0.282 [ best cc: 0.282 0.280 ]
Number of matching sites of top 2 structures: 3
p=000 f=002 cc=0.100 r=015 cc=0.234 [ best cc: 0.282 0.280 ]
p=001 f=000 cc=0.181 r=015 cc=0.263 [ best cc: 0.282 0.280 0.263 ]
Number of matching sites of top 2 structures: 3
Number of matching sites of top 3 structures: 3
p=001 f=001 cc=0.146 r=015 cc=0.277 [ best cc: 0.282 0.280 0.277 0.263 ]
Number of matching sites of top 2 structures: 3
Number of matching sites of top 3 structures: 3
Number of matching sites of top 4 structures: 3
p=001 f=002 cc=0.142 r=015 cc=0.278 [ best cc: 0.282 0.280 0.278 0.277 0.263 ]
Number of matching sites of top 2 structures: 3
Number of matching sites of top 3 structures: 3
Number of matching sites of top 4 structures: 3
Number of matching sites of top 5 structures: 3

```

Here a correlation coefficient of 0.5 is very good (0.1 is hopeless, 0.2 is possible, 0.3 is good) and 2 sites were found that matched in the first two tries. The program continues until 5 structures all have matching sites, then ends and prints out the final correlations, after taking the top 2 sites.

Finding the hand and scoring heavy-atom solutions

Normally either hand of the heavy-atom substructure is a possible solution, and both must be tested by calculating phases and examining the electron density map and by carrying out density modification, as they will give the same statistics for all heavy-atom analysis and phasing steps. Note that in chiral space groups (those that have a handedness such as P61, both hands of the space group must be tested. The AutoSol Wizard will do this for you, inverting the hand of the heavy-atom substructure and the space group at the same time. For example, in space group P61 the hand of the substructure is inverted and then it is placed in space group P65.

The AutoSol Wizard scores heavy-atom solutions based on two criteria by default. The first criterion is the skew of the electron density in the map (SKEW). Good values for the skew are anything greater than 0.1. In a MAD structure determination, the heavy-atom solution with the correct hand may have a more positive skew than the one with the inverse hand. The second criterion is the correlation of local RMS density (CORR_RMS). This is a measure of how contiguous the solvent and non-solvent regions are in the map. (If the local rms is low at one point and also low at neighboring points, then the solvent region must be relatively contiguous, and not split up into small regions.) For MAD datasets, SOLVE is used for calculating phases. For a MAD dataset, a figure of merit of 0.5 is acceptable, 0.6 is fine and anything above 0.7 is very good.

The first three solutions scored are all quite good. Here is the first and best one:

```

SCORING SOLUTION 1: Solution 1 using HYSS on FA.sca. Dataset #1 SG="C 1 2 1", with 2 sites
Number of scoring criteria: 2
Using BAYES-CC (Bayesian estimate of CC of map to perfect) as scores
Evaluating solution 1
FOM found: 0.59
Number of scoring criteria: 2
Using BAYES-CC (Bayesian estimate of CC of map to perfect) as scores
...
Scoring for this solution now...

AutoSol_run_1_/TEMP0/resolve.scores SKEW 0.2597009

```

```
AutoSol_run_1_/TEMP0/resolve.scores CORR_RMS 0.8756687
```

```
CC-EST (BAYES-CC) SKEW : 56.1 +/- 18.3  
CC-EST (BAYES-CC) CORR_RMS : 55.7 +/- 36.1  
ESTIMATED MAP CC x 100: 57.9 +/- 14.1
```

The ESTIMATED MAP CC x 100 is an estimate of the quality of the experimental electron density map (not the density-modified one). A set of real structures was used to calibrate the range of values of each score that were obtained for phases with varying quality. The resulting probability distributions are used above to estimate the correlation between the experimental map and an ideal map for this structure. Then all the estimates are combined to yield an overall Bayesian estimate of the map quality. These are reported as CC x 100 +/- 2SD. These estimated map CC values are usually fairly close, so if the estimate is 57.9 +/- 14.1 then you can be confident that your structure is not only solved but that you will have a good map when it is density modified.

In this case the datasets used to find heavy-atom substructures were the FA values in FA.sca and the peak data in peak.sca_ano_1.sca. For each dataset one solution was found, and that solution and its inverse were scored. The scores were (skipping extra text below):

```
SCORING SOLUTION 1: Solution 1 using HYSS on FA.sca.  
Dataset #1 SG="C 1 2 1", with 2 sites  
ESTIMATED MAP CC x 100: 57.9 +/- 14.1
```

```
SCORING SOLUTION 2: Solution 2 using HYSS on FA.sca and taking inverse.  
Dataset #1 SG="C 1 2 1", with 2 sites  
ESTIMATED MAP CC x 100: 42.0 +/- 24.3
```

```
SCORING SOLUTION 3: Solution 3 using HYSS on peak.sca_ano_1.sca.  
Dataset #1 SG="C 1 2 1", with 2 sites  
ESTIMATED MAP CC x 100: 53.2 +/- 16.7
```

```
SCORING SOLUTION 4: Solution 4 using HYSS on peak.sca_ano_1.sca and taking inverse.  
Dataset #1 SG="C 1 2 1", with 2 sites  
ESTIMATED MAP CC x 100: 53.1 +/- 16.7
```

In this case the best score was using the FA values and taking the original hand (ESTIMATED MAP CC x 100: 57.9 +/- 14.1), and score for the inverted hand of the heavy-atom substructure was worse (ESTIMATED MAP CC x 100: 42.0 +/- 24.3) and so the hand was clear.

Final phasing with SOLVE

Once the best heavy-atom solution or solutions are chosen based on ESTIMATED MAP CC, these are used in a final round of phasing with SOLVE (for MAD phasing). The log file from phasing for solution 1 is in solve_1.prt. This SOLVE log file repeats the correlation analysis of anomalous differences between data at each wavelength. Then it carries out a detailed refinement of the scattering factors at each wavelength. Finally the heavy-atom model is refined and phases are calculated with Bayesian correlated MAD phasing. The final occupancies and coordinates are listed at the end:

```
SITE ATOM OCCUP X Y Z B  
CURRENT VALUES: 1 Se 0.9609 0.4842 0.5199 0.5931 52.0344  
CURRENT VALUES: 2 Se 0.5891 0.4723 0.3052 0.4479 60.0000
```

In this case the occupancy of one site is quite near 1 and the other is lower. The second site is a selenomethionine that is not well ordered (it is the N-terminal residue in the protein).

Statistical density modification with RESOLVE

After MAD phases are calculated with SOLVE, the AutoSol Wizard uses RESOLVE density modification to improve the quality of the electron density map. The statistical density modification in RESOLVE takes advantage of the flatness of the solvent region and the expected distribution of electron density in the region containing the macromolecule, as well as any NCS that can be found from the heavy-atom substructure. The weighted structure factors and phases (FWT, PHWT) from SOLVE are used to calculate the starting map for RESOLVE, and the experimental structure factor amplitudes (FP) and MAD Hendrickson-Lattman coefficients from SOLVE are used in the density modification process. The output from RESOLVE for solution 1 can be found in resolve_1.log. Here are key sections of this output.

First, the plot of how many points in the "protein" region of the map have each possible value of electron density. The plot below is normalized so that a density of zero is the mean of the solvent region, and the standard deviation of the density in the map is 1.0. A perfect map has a lot of points with density slightly less than zero on this scale (the points between atoms) and a few points with very high density (the points near atoms), and no points with very negative density. Such a map has a very high skew (think "skewed off to the right"). This map is good, with a positive skew, though it is not perfect.

```
Plot of Observed (o) and model (x) electron density distributions for protein
region, where the model distribution is given by,
  p_model(beta*(rho+offset)) = p_ideal(rho)
and then convoluted with a gaussian with width of sigma
where sigma, offset and beta are given below under "Error estimate."
```

```
0.03.....
. . .
. xx. .
. xx00xx .
. x0 . xx .
. xx . xx .
. x0 . x0 .
p(rho) . ox . x0 .
. xx . x0 .
. x . xx0 .
. ox . xx .
. xx . xx .
. xx . xxx .
. oxx . oxxx .
. oxx . oxxxxx .
0.0 xxxx.....Oxxxxxxx

-2 -1 0 1 2 3

normalized rho (0 = mean of solvent region)
-----
```

After density modification is complete, this plot becomes much more like one from a perfect structure:

```
0.03.....
. . .
. . .
. xxxx00. .
. xx000xx. .
. xx0 x00 .
. 00 .xx0 .
p(rho) . x . x .
. x . xx .
. ox . oxxx .
. ox . oxxxxx .
. xx . 00xxxxxx .
. ox . 000xxxxxx .
. oxx . 0 0xxxxxx .
xxxx . x0
0.0 x.....x
```

```
-2 -1 0 1 2 3
```

```
normalized rho (0 = mean of solvent region)
```

The key statistic from this RESOLVE density modification is the R-factor for comparison of observed structure factor amplitudes (FP) with those calculated from the density modification procedure (FC). In this gene-5 MAD phasing the R-factor is very low:

```
Overall R-factor for FC vs FP: 0.284 for 2669 reflections
```

An acceptable value is anything below 0.35; below 0.30 is good.

Generation of FreeR flags

The AutoSol Wizard will create a set of free R flags indicating which reflections are not to be used in refinement. By default 5% of reflections (up to a maximum of 2000) are reserved for this test set. If you want to supply a reflection file hires.mtz that has higher resolution than the data used to solve the structure, or has a test set already marked, then you can do this with the keyword `input_refinement_file=hires.mtz`. The files to be used for model-building and refinement are listed in the AutoSol log file:

```
Copying AutoSol_run_1/solve_1.mtz and adding free R flags for refinement
input_data_file_use: AutoSol_run_1/solve_1.mtz
labin_use: labin FP=FP SIGFP=SIGFP PHIB=PHIB FOM=FOM HLA=HLA HLB=HLB HLC=HLC HLD=HLD
Adding FreeR_flag to AutoSol_run_1/TEMP0/solve_1.mtz
...
THE FILE AutoSol_run_1/resolve_1.mtz will be used for model-building
THE FILE exptl_fobs_phases_freer_flags_1.mtz will be used for refinement
```

Model-building with RESOLVE

The AutoSol Wizard by default uses a very quick method to build just the secondary structure of your macromolecule. This is controlled by the keywords `helices_strands_only=True` and `helices_strands_start=True`. If you set `helices_strands_only=True` then only secondary structure will be built. If you instead set `helices_strands_start=True` then a secondary-structure model will be built and then it will be extended with standard RESOLVE model-building. The Wizard will guess from your sequence file whether the structure is protein or RNA or DNA (but you can tell it if you want with `(chain_type=PROTEIN)`).

If the quick model-building does not build a satisfactory model (if the correlation of map and model is less than `acceptable_secondary_structure_cc=0.35`), then model-building is tried again with the standard build procedure, essentially the same as one cycle of model-building with the AutoBuild Wizard (see the web page Automated Model Building and Rebuilding with AutoBuild, except that if you specify `thoroughness=quick` as we have in this example, the model-building is done less comprehensively to speed things up).

In this case the secondary-structure-only model-building produces an initial model with 38 residues built and side chains assigned to 0, and which has a model-map correlation of 0.40

```
Secondary-structure model: AutoSol_run_1/TEMP0/Build_1.pdb
Log file: Build_1.log copied to Build_1.log
Models to combine and extend: ['Build_1.pdb']
Using CC to score in combine_extend
Model 2: Residues built=38 placed=0 Chains=5 Model-map CC=0.40 (Build_combine
_extend_2.pdb)
This is new best model with cc = 0.4
Refining model: Build_combine_extend_2.pdb
Model: AutoSol_run_1/TEMP0/refine_2.pdb R/Rfree=0.50/0.50
```

After several cycles of model completion, the model-map correlation is now reasonably good (0.63), the model-building is considered successful and the refined initial model is written out to `refine_12.pdb` in the

output directory. It is still just a preliminary model, but it is good enough to tell that the structure is solved. For full model-building you will want to go on and use the AutoBuild Wizard (see the web page Automated Model Building and Rebuilding with AutoBuild)

The AutoSol_summary.dat summary file

A quick summary of the results of your AutoSol run is in the AutoSol_summary.dat file in your output directory. This file lists the key files that were produced in your run of AutoSol (all these are in the output directory) and some of the key statistics for the run, including the scores for the heavy-atom substructure and the model-building and refinement statistics. These statistics are listed for all the solutions obtained, with the highest-scoring solutions first. Here is part of the summary for this gene-5 MAD dataset:

```
-----CURRENT SOLUTIONS FOR RUN 1 : -----
*** FILES ARE IN THE DIRECTORY: AutoSol_run_1_ ***

Solution # 1 BAYES-CC: 57.9 +/- 14.1 Dataset #1 FOM: 0.5

Solution 1 using HYSS on FA.sca. Dataset #1 SG="C 1 2 1"
Dataset number: 1
Dataset type: mad
Datafiles used: ['high.sca', 'peak.sca', 'infl.sca']
Sites: 2 (Already used for Phasing at resol of 2.6)
NCS information in: AutoSol_1.ncs_spec
Experimental phases in: solve_1.mtz
Experimental phases plus FreeR_flags for refinement in:
  exptl_fobs_phases_freeR_flags_1.mtz
Density-modified phases in: resolve_1.mtz
HA sites (PDB format) in: ha_1.pdb_formatted.pdb
Sequence file in: sequence.dat
Model in: refine_12.pdb
  Residues built: 65
  Side-chains built: 10
  Chains: 6
  Overall model-map correlation: 0.63
  R/R-free: 0.42/0.48
Scaling logfile in: dataset_1_scale.log
HYSS logfile in: FA.sca_hyss.log
Phasing logfile in: solve_1.prt
Density modification logfile in: resolve_1.log (R=0.28)
Build logfile in: Build_combine_extend_12.log

  Score type: SKEW CORR_RMS
  Raw scores: 0.26 0.88
  100x EST OF CC: 56.10 55.69

  Heavy atom sites (fractional):
  xyz 0.486 -0.480 0.592
  xyz 0.473 0.324 0.450
```

How do I know if I have a good solution?

Here are some of the things to look for to tell if you have obtained a correct solution:

- How much of the model was built? More than 50% is good, particularly if you are using the default of `helices_strands_only=True`. If less than 25% of the model is built, then it may be entirely incorrect. Have a look at the model. If you see clear sets of parallel or antiparallel strands, or if you see helices and strands with the expected relationships, your model is going to be correct. If you see a lot of short fragments everywhere, your model and solution is going to be incorrect. How many side-chains were fitted to density? More than 25% is ok, more than 50% is very good.

- What is the R-factor of the model? This only applies if you are building a full model (not for helices_strands_only=True). For a solution at moderate to high resolution (2.5 Å or better) the R-factor should be in the low 30's to be very good. For lower-resolution data, an R-factor in the low 40's is probably largely correct but the model is not very good.
- What was the overall signal-to-noise in the data? Above 1 is good, below 0.5 is very low.
- What are the individual CC-BAYES estimates of map correlation for your top solution? For a good solution they are all around 50 or more, with 2SD uncertainties that are about 10-20.
- What is the overall "ESTIMATED MAP CC x 100" of your top solution. This should also be 50 or more for a good solution. This is an estimate of the map correlation before density modification, so if you have a lot of solvent or several NCS-related copies in the asymmetric unit, then lower values may still give you a good map.
- What is the difference in "ESTIMATED MAP CC x 100" between the top solution and its inverse? If this is large (more than the 2SD values for each) that is a good sign.

What to do next

Once you have run AutoSol and have obtained a good solution and model, the next thing to do is to run the AutoBuild Wizard. If you run it in the same directory where you ran AutoSol, the AutoBuild Wizard will pick up where the AutoSol Wizard left off and carry out iterative model-building, density modification and refinement to improve your model and map. See the web page Automated Model Building and Rebuilding with AutoBuild for details on how to run AutoBuild.

If you do not obtain a good solution, then it's not time to give up yet. There are a number of standard things to try that may improve the structure determination. Here are a few that you should always try:

- Try setting thoroughness=thorough if it had previously been set to quick. This can make a big difference, though it takes longer.
- Try setting max_choices to a larger number, or desired_coverage to a higher value.
- Have a careful look at all the output files. Work your way through the main log file (e.g., AutoSol_run_1_1.log) and all the other principal log files in order beginning with scaling (dataset_1_scale.log), then looking at heavy-atom searching (FA.sca_hyss.log), phasing (e.g., solve_10.log or solve_xx.log depending on which solution xx was the top solution) and density modification (e.g., resolve_xx.log). Is there anything strange or unusual in any of them that may give you a clue as to what to try next? For example did the phasing work well (high figure of merit) yet the density modification failed? (Perhaps the hand is incorrect). Was the solvent content estimated correctly? (You can specify it yourself if you want). What does the xtriage output say? Is there twinning or strong translational symmetry? Are there problems with reflections near ice rings? Are there many outlier reflections?
- Try a different resolution cutoff. For example 0.5 Å lower resolution than you tried before. Often the highest-resolution shells have little useful information for structure solution (though the data may be useful in refinement and density modification).
- Try a different rejection criterion for outliers. The default is ratio_out=10.0 (toss reflections with delta F more than 10 times the rms delta F of all reflections in the shell). Try instead ratio_out=3 to toss outliers.
- If the heavy-atom substructure search did not yield plausible solutions, try searching with HYSS using the command-line interface, and vary the resolution and number of sites you look for. Can you find a solution that has a higher CC than the one found in AutoSol? If so, you can read your solution in to AutoSol with sites_file=my_sites.pdb.

- Was an anisotropy correction applied in AutoSol? If there is some anisotropy but no correction was applied, you can force AutoSol to apply the correction with `correct_aniso=True`.
- Try related space groups. If you are not positive that your space group is P212121, then try other possibilities with different or no screw axes.

Additional information

For details about the AutoSol Wizard, see Automated structure solution with AutoSol. For help on running Wizards, see Using the PHENIX Wizards.

Tutorial: Solving a structure with MIR data

mir_phasing.html

Introduction

This tutorial will use some very good MIR data (Native and 5 derivatives from a rh-dehalogenase protein MIR dataset analyzed at 2.8 Å) as an example of how to solve a MIR dataset with AutoSol. It is designed to be read all the way through, giving pointers for you along the way. Once you have read it all and run the example data and looked at the output files, you will be in a good position to run your own data through AutoSol.

Setting up to run PHENIX

If PHENIX is already installed and your environment is all set, then if you type:

```
echo $PHENIX
```

then you should get back something like this:

```
/xtal/phenix-1.3
```

If instead you get:

```
PHENIX: undefined variable
```

then you need to set up your PHENIX environment. See the PHENIX installation page for details of how to do this. If you are using the C-shell environment (csh) then all you will need to do is add one line to your .cshrc (or equivalent) file that looks like this:

```
source /xtal/phenix-1.3/phenix_env
```

(except that the path in this statement will be where your PHENIX is installed). Then the next time you log in \$PHENIX will be defined.

Running the demo rh-dehalogenase data with AutoSol

To run AutoSol on the demo rh-dehalogenase data, make yourself a tutorials directory and cd into that directory:

```
mkdir tutorials
cd tutorials
```

Now type the phenix command:

```
phenix.run_example --help
```

to list the available examples. Choosing rh-dehalogenase-mir for this tutorial, you can now use the phenix command:

```
phenix.run_example rh-dehalogenase-mir
```

to solve the rh-dehalogenase structure with AutoSol. This command will copy the directory \$PHENIX/examples/rh-dehalogenase-mir to your current directory (tutorials) and call it tutorials/rh-dehalogenase-mir/. Then it will run AutoSol using the command file run.sh that is present in this tutorials/rh-dehalogenase-mir/ directory.

We are going to run this MIR dataset using a parameters file "rh-dehalogenase-mir.eff". As MIR datasets have a lot of different files and heavy-atom parameters to specify, it is easiest to run MIR by editing a simple file.

This command file run.sh is simple. It says:

```
#!/bin/sh
echo "Running AutoSol on rhodococcus dehalogenase data..."
phenix.autosol rh-dehalogenase-mir.eff
```

The first line (#!/bin/sh) tells the system to interpret the remainder of the text in the file using the sh (or bash) -shell (sh).

The command phenix.autosol runs the command-line version of AutoSol (see Automated Structure Solution using AutoSol for all the details about AutoSol including a full list of keywords).

The second line says to run the AutoSol Wizard, and use the contents of the file rh-dehalogenase-mir.eff as parameters.

Now let's look at the rh-dehalogenase-mir.eff parameters file. Here is the entire file:

```
# parameters for autosol run with rh-dehalogenase-mir Native+5 derivs
#
autosol {
  seq_file = sequence.dat
  crystal_info {
    space_group = p21212
    unit_cell = 93.796 79.849 43.108 90.000 90.000 90.00
  }
  native {
    data = rt_rd_1.sca
  }
  deriv {
    data = auki_rd_1.sca
    atom_type = Au
    sites = 5
    inano = noinano *inano anoonly
    lambda = 1.5418
  }
  deriv {
    data = hgki_rd_1.sca
    atom_type = Hg
    sites = 5
    inano = noinano *inano anoonly
    lambda = 1.5418
  }
  deriv {
    data = ndac_rd_1.sca
    atom_type = Pt
    sites = 5
    inano = noinano *inano anoonly
    lambda = 1.5418
  }
  deriv {
    data = hgi2_rd_1.sca
    atom_type = Hg
    sites = 5
    inano = noinano *inano anoonly
    lambda = 1.5418
  }
  deriv {
    data = smac_1.sca
    atom_type = Sm
    sites = 5
    inano = noinano *inano anoonly
    lambda = 1.5418
  }
}
```

Notice how the brackets (`{` and `}`) work in this file. Everything in this file after the word "autosol" that is between the opening left-bracket (`{`) and the closing right-bracket (`}`) is part of the autosol "scope". The AutoSol wizard looks for "autosol { lots of parameters }" and interprets everything inside these brackets. Everything outside the scope "autosol" is ignored.

Within the autosol scope there are some keywords like "atom_type = Sm", these are normally one per line.

There are also additional scopes, with keywords inside them. For example the space_group and unit_cell information are inside the scope "crystal_info".

The information about the native and each derivative is in a separate scope called "native" or "deriv". You can have one native for an MIR dataset and as many derivatives as you like.

The first part of the script, with the scope "crystal_info" tells AutoSol about the cell and space-group. These values override any values read from the input data files. Next the scope "native" gives the datafile name for the native data. Then a series of "deriv" scopes give information for each of 5 derivatives. Within this "deriv" scope you can define the datafile name, the heavy-atom name, the wavelength (lambda), f_prime and f_double_prime values for that wavelength. If you specify the heavy-atom and wavelength then the AutoSol Wizard will guess the f-prime and f-double-prime values at that wavelength. However if you know these values, then you should enter them.

Note the keyword line " inano = noinano *inano anoonly " for each derivative. This is an example of how choices are specified in a parameters file. The choice with a "*" next to it is the one that is chosen (in this case, "inano" which means include anomalous differences in phasing).

The AutoSol Wizard solves MIR datasets in several steps, and in the first step, the individual derivatives are all solved separately (except using difference Fouriers to phase one derivative using a solution from another). Then when all are finished all the SIR or SIRAS datasets are phased all together with SOLVE Bayesian correlated phasing. This approach works well because a substructure determination is done separately for each derivative, and if any one of them works well, then all the derivatives can be solved.

Although the phenix.run_example rh-dehalogenase-mir command has just run AutoSol from a script (run.sh), you can run AutoSol yourself from this script with the same phenix.autosol rh-dehalogenase-mir.eff command. You can also run AutoSol from a GUI. All these possibilities are described in Using the PHENIX Wizards.

Where are my files?

Once you have started AutoSol or another Wizard, an output directory will be created in your current (working) directory. The first time you run AutoSol in this directory, this output directory will be called AutoSol_run_1_ (or AutoSol_run_1_/, where the slash at the end just indicates that this is a directory). All of the output from run 1 of AutoSol will be in this directory. If you run AutoSol again, a new subdirectory called AutoSol_run_2_ will be created.

Inside the directory AutoSol_run_1_ there will be one or more temporary directories such as TEMP0 created while the Wizard is running. The files in this temporary directory may be useful sometimes in figuring out what the Wizard is doing (or not doing!). By default these directories are emptied when the Wizard finishes (but you can keep their contents with the command clean_up=False if you want.)

What parameters did I use?

Once the AutoSol wizard has started (when run from the command line), a parameters file called autosol.eff will be created in your output directory (e.g., AutoSol_run_1_/autosol.eff). This parameters file has a header that says what command you used to run AutoSol, and it contains all the starting values of all parameters for

this run (including the defaults for all the parameters that you did not set).

The autosol.eff file is good for more than just looking at the values of parameters, though. If you copy this file to a new one (for example autosol_lores.eff) and edit it to change the values of some of the parameters (resolution=3.0) then you can re-run AutoSol with the new values of your parameters like this:

```
phenix.autosol autosol_lores.eff
```

This command will do everything just the same as in your first run but use only the data to 3.0 Å.

Reading the log files for your AutoSol run file

While the AutoSol wizard is running, there are several places you can look to see what is going on. The most important one is the overall log file for the AutoSol run. This log file is located in:

```
AutoSol_run_1_/AutoSol_run_1_1.log
```

for run 1 of AutoSol. (The second 1 in this log file name will be incremented if you stop this run in the middle and restart it with a command like phenix.autosol run=1).

The AutoSol_run_1_1.log file is a running summary of what the AutoSol Wizard is doing. Here are a few of the key sections of the log files produced for the rh-dehalogenase MIR dataset.

Summary of the command-line arguments

Near the top of the log file you will find:

```
Starting AutoSol with the command:
```

```
phenix.autosol
```

```
Reading effective parameters from rh-dehalogenase-mir.eff
```

```
autosol {  
  atom_type = None  
  lambda = None  
  f_prime = None  
  f_double_prime = None  
  wavelength_name = peak inf high low remote  
  sites = None  
  sites_file = None  
  seq_file = "sequence.dat"  
  ...
```

This is just a repeat of the parameters in your rh-dehalogenase-mir.eff parameters file, merged in with all the defaults for the AutoSol wizard.

Reading the datafiles.

The AutoSol Wizard will read in your datafiles and check their contents, printing out a summary for each one. This is done one dataset at a time (each native-derivative pair) until all have been read in. Here is the summary for the first derivative:

```
HKLIN ENTRY: rt_rd_1.sca  
FILE TYPE scalepack_no_merge_original_index  
GUESS FILE TYPE MERGE TYPE sca unmerged  
LABELS['I', 'SIGI']  
CONTENTS: ['rt_rd_1.sca', 'sca', 'unmerged', 'P 21 21 2', None, None, ['I', 'SIGI']]  
Inverse hand of space group: P 21 21 2  
HKLIN ENTRY: auki_rd_1.sca  
FILE TYPE scalepack_no_merge_original_index  
GUESS FILE TYPE MERGE TYPE sca unmerged
```

```

LABELS['I', 'SIGI']
CONTENTS: ['auki_rd_1.sca', 'sca', 'unmerged', 'P 21 21 21', None, None, ['I', 'SIGI']]
Inverse hand of space group: P 21 21 2
Converting the files ['rt_rd_1.sca', 'auki_rd_1.sca'] to sca format before proceeding

```

ImportRawData.

The input data files `rt_rd_1.sca` and `auki_rd_1.sca` are in unmerged Scalepack format. The AutoSol wizard converts everything to premerged Scalepack format before proceeding. Here is where the AutoSol Wizard identifies the format and then calls the ImportRawData Wizard:

```

Running import directly...
WIZARD: ImportRawData

```

followed eventually by...

```

List of output files :
File 1: rt_rd_1_PHX.sca
File 2: auki_rd_1_PHX.sca

```

These output files are in premerged Scalepack format.

After completing the ImportRawData step, the AutoSol Wizard goes back to the beginning, but uses the newly-converted files `rt_rd_1_PHX.sca` and `auki_rd_1_PHX.sca`:

```

HKLIN ENTRY: AutoSol_run_1/rt_rd_1_PHX.sca
FILE TYPE scalepack_merge
GUESS FILE TYPE MERGE TYPE sca premerged
LABELS['IPLUS', 'SIGIPLUS', 'IMINU', 'SIGIMINU']
Unit cell: (93.796, 79.849, 43.108, 90, 90, 90)
Space group: P 21 21 2 (No. 18)
CONTENTS: ['AutoSol_run_1/rt_rd_1_PHX.sca', 'sca', 'premerged', 'P 21 21 2',
[93.796000000000006, 79.849000000000004, 43.107999999999997, 90.0, 90.0, 90.0],
2.4307589843043771, ['IPLUS', 'SIGIPLUS', 'IMINU', 'SIGIMINU']]
Inverse hand of space group: P 21 21 2
Resolution from AutoSol_run_1/rt_rd_1_PHX.sca is 2.43
HKLIN ENTRY: AutoSol_run_1/auki_rd_1_PHX.sca
FILE TYPE scalepack_merge
GUESS FILE TYPE MERGE TYPE sca premerged
LABELS['IPLUS', 'SIGIPLUS', 'IMINU', 'SIGIMINU']
Unit cell: (93.796, 79.849, 43.108, 90, 90, 90)
Space group: P 21 21 2 (No. 18)
CONTENTS: ['AutoSol_run_1/auki_rd_1_PHX.sca', 'sca', 'premerged', 'P 21 21 2',
[93.796000000000006, 79.849000000000004, 43.107999999999997, 90.0, 90.0, 90.0],
2.430806639777233, ['IPLUS', 'SIGIPLUS', 'IMINU', 'SIGIMINU']]
Inverse hand of space group: P 21 21 2
Total of 2 input data files

```

Guessing cell contents and scattering factors

The AutoSol Wizard uses the sequence information in your sequence file (`sequence.dat`) and the cell parameters and space group to guess the number of NCS copies and the solvent fraction. It will use any wavelength information you provide it to guess the values of scattering factors for the heavy-atoms. If you do not give any wavelength then a value of $\lambda=1.5418$ (Cu K alpha) will be used.

```

AutoSol_guess_setup_for_scaling AutoSol Run 1 Thu Dec 18 13:34:29 2008

Setting default value of 0.5 for solvent_fraction
Setting default value of 200 for residues
Solvent fraction and resolution and ha types/scatt fact
Guessing setup for scaling dataset 1
SG P 21 21 2
cell [93.796000000000006, 79.849000000000004, 43.107999999999997, 90.0, 90.0, 90.0]
Number of residues in unique chains in seq file: 294

```



```
Unit cell: (93.796, 79.849, 43.108, 90, 90, 90)
Space group: P 21 21 2 (No. 18)
CELL VOLUME :322858.090387
N_EQUIV:4
GUESS OF NCS COPIES: 1
SOLVENT FRACTION ESTIMATE: 0.51
Total residues:294
Total Met:6
resolution estimate: 2.43
Guessing scattering factors for AU at 1.5418 A

Guesses of scattering factors for Au
Atom Lambda f' f" datafile
Native rt_rd_1_PHX.sca
Au 1.5418 -5.09 7.30 auki_rd_1_PHX.sca
```

Running phenix.xtriage

The AutoSol Wizard automatically runs phenix.xtriage on each of your input datafiles to analyze them for twinning, outliers, translational symmetry, and other special conditions that you should be aware of. You can read more about xtriage in [Data quality assessment with phenix.xtriage](#). Part of the summary output from xtriage for this dataset looks like this:

```
No (pseudo)merohedral twin laws were found.
```

```
Patterson analyses
- Largest peak height : 6.470
  (corresponding p value : 0.60758)
```

```
The largest off-origin peak in the Patterson function is 6.47% of the
height of the origin peak. No significant pseudotranslation is detected.
```

```
The results of the L-test indicate that the intensity statistics
behave as expected. No twinning is suspected.
```

In this space group (P21 21 2) with the cell dimensions in this structure, there are no ways to create a twinned crystal, so you do not have to worry about twinning. There is also no large off-origin peak in the native Patterson, so there does not appear to be any translational pseudo-symmetry.

Testing for anisotropy in the data

After all the SIR datasets are read in, the AutoSol Wizard tests for anisotropy by determining the range of effective anisotropic B values along the principal lattice directions. If this range is large and the ratio of the largest to the smallest value is also large then the data are by default corrected to make the anisotropy small (see the AutoSol web page for more discussion of the anisotropy correction). In the rh-dehalogenase case, the range of anisotropic B values is small and no correction is made:

```
Range of aniso B: 13.21 20.51
Not using aniso-corrected data files as the range of aniso b is only 7.3 and 'correct_aniso' is not set
```

Note that if any one of the datafiles in a MIR dataset has a high anisotropy, then by default all of them will be corrected for anisotropy.

Scaling MIR data

The AutoSol Wizard uses SOLVE localscaling to scale MIR data. The procedure is basically to scale all the data to the native. During this process outliers that deviate from the reference values by more than ratio_out (default=10) standard deviations (using all data in the appropriate resolution shell to estimate the SD) are rejected.

Running HYSS to find the heavy-atom substructure

The HYSS (hybrid substructure search) procedure for heavy-atom searching uses a combination of a Patterson search for 2-site solutions with direct methods recycling. The search ends when the same solution is found beginning with several different starting points. The HYSS log files are named after the datafile that they are based on and the type of differences (ano, iso) that are being used. In this rh-dehalogenase MIR dataset, the HYSS logfile for the HgKI derivative is hgki_rd_1_PHX.sca_iso_2.sca_hyss.log. The key part of this HYSS log file is:

Entering search loop:

```
p = peaklist index in Patterson map
f = peaklist index in two-site translation function
cc = correlation coefficient after extrapolation scan
r = number of dual-space recycling cycles
cc = final correlation coefficient

p=000 f=000 cc=0.186 r=015 cc=0.245 [ best cc: 0.245 ]
p=000 f=001 cc=0.198 r=015 cc=0.240 [ best cc: 0.245 0.240 ]
Number of matching sites of top 2 structures: 3
p=000 f=002 cc=0.174 r=015 cc=0.215 [ best cc: 0.245 0.240 ]
p=001 f=000 cc=0.212 r=015 cc=0.254 [ best cc: 0.254 0.245 0.240 ]
Number of matching sites of top 2 structures: 7
Number of matching sites of top 3 structures: 3
p=001 f=001 cc=0.219 r=015 cc=0.254 [ best cc: 0.254 0.254 0.245 0.240 ]
Number of matching sites of top 2 structures: 8
Number of matching sites of top 3 structures: 7
Number of matching sites of top 4 structures: 3
p=001 f=002 cc=0.163 r=015 cc=0.261 [ best cc: 0.261 0.254 0.254 0.245 ]
Number of matching sites of top 2 structures: 2
Number of matching sites of top 3 structures: 2
Number of matching sites of top 4 structures: 2

...
p=013 f=000 cc=0.184 r=015 cc=0.290 [ best cc: 0.299 0.291 0.290 0.290 ]
Number of matching sites of top 2 structures: 6
Number of matching sites of top 3 structures: 6
Number of matching sites of top 4 structures: 6
```

Here a correlation coefficient of 0.5 is very good (0.1 is hopeless, 0.2 is possible, 0.3 is good) and 8 sites were found that matched in the first two tries. The program continues until 4 structures all have 6 matching sites, then ends and prints out the final correlations, after taking the top 5 sites.

Finding the hand and scoring heavy-atom solutions

Normally either hand of the heavy-atom substructure is a possible solution, and both must be tested by calculating phases and examining the electron density map and by carrying out density modification, as they will give the same statistics for all heavy-atom analysis and phasing steps. Note that in chiral space groups (those that have a handedness such as P61, both hands of the space group must be tested. The AutoSol Wizard will do this for you, inverting the hand of the heavy-atom substructure and the space group at the same time. For example, in space group P61 the hand of the substructure is inverted and then it is placed in space group P65.

The AutoSol Wizard scores heavy-atom solutions based on two criteria. The first criterion is the skew of the electron density in the map (SKEW). Good values for the skew are anything greater than 0.1. In a MIR structure determination, the heavy-atom solution with the correct hand may have a more positive skew than the one with the inverse hand. The second criterion is the correlation of local RMS density (CORR_RMS). This is a measure of how contiguous the solvent and non-solvent regions are in the map. (If the local rms is

low at one point and also low at neighboring points, then the solvent region must be relatively contiguous, and not split up into small regions.) For MIR datasets, SOLVE is used for calculating phases. For a MIR dataset, a figure of merit of 0.5 is acceptable, 0.6 is fine and anything above 0.7 is very good. The scores are listed in the AutoSol log file. Here is the scoring for solution 4 (the best initial map):

```
AutoSol_run_1_/TEMP0/resolve.scores SKEW 0.2369411
AutoSol_run_1_/TEMP0/resolve.scores CORR_RMS 0.9131303
```

```
CC-EST (BAYES-CC) SKEW : 54.1 +/- 19.5
CC-EST (BAYES-CC) CORR_RMS : 61.5 +/- 30.6
ESTIMATED MAP CC x 100: 58.1 +/- 14.0
```

This is a good solution, with a high (and positive) skew (0.24), and a high correlation of local rms density (0.91)

The ESTIMATED MAP CC x 100 is an estimate of the quality of the experimental electron density map (not the density-modified one). A set of real structures was used to calibrate the range of values of each score that were obtained for phases with varying quality. The resulting probability distributions are used above to estimate the correlation between the experimental map and an ideal map for this structure. Then all the estimates are combined to yield an overall Bayesian estimate of the map quality. These are reported as CC x 100 +/- 2SD. These estimated map CC values are usually fairly close, so if the estimate is 58.1 +/- 14.0 then you can be confident that your structure is solved and that the density-modified map will be quite good.

In this case the datasets used to find heavy-atom substructures were the isomorphous differences for each derivative. For each dataset one solution was found, and that solution and its inverse were scored. The scores were (skipping extra text below):

```
SCORING SOLUTION 1: Solution 1 using HYSS on
AutoSol_run_1_/auki_rd_1_PHX.sca_iso_1.sca.
Dataset #1 SG="P 21 21 2", with 5 sites
ESTIMATED MAP CC x 100: 46.8 +/- 20.9
```

```
SCORING SOLUTION 2: Solution 2 using HYSS on
AutoSol_run_1_/auki_rd_1_PHX.sca_iso_1.sca and taking inverse.
Dataset #1 SG="P 21 21 2", with 5 sites
ESTIMATED MAP CC x 100: 32.0 +/- 32.1
```

```
SCORING SOLUTION 3: Solution 3 using HYSS on
AutoSol_run_1_/hgki_rd_1_PHX.sca_iso_2.sca.
Dataset #2 SG="P 21 21 2", with 5 sites
ESTIMATED MAP CC x 100: 33.5 +/- 37.0
```

```
SCORING SOLUTION 4: Solution 4 using HYSS on
AutoSol_run_1_/hgki_rd_1_PHX.sca_iso_2.sca and taking inverse.
Dataset #2 SG="P 21 21 2", with 5 sites
ESTIMATED MAP CC x 100: 58.1 +/- 14.0
```

In this case the best score was solution 4 (as shown above), based on the HGKI derivative and taking the inverse of the heavy-atom sites, with a ESTIMATED MAP CC x 100: 58.1 +/- 14.0 The score from the opposite hand was just 33.5 +/- 37.0 and so the hand was clear.

Finding origin shifts between heavy-atom solutions for different derivatives and combining phases

Depending on the space group, there may be a few or infinitely many totally equivalent heavy-atom substructures for a particular native-derivative pair. These are related to each other by translations that can be thought of as offsets of the origins for the two substructures. The AutoSol Wizard identifies the allowed offsets for the space group. Then it aligns the solutions from different derivatives by finding the origin offset that

maximizes the correlation of electron density in the native Fouriers for the two. Then it combines the phases from the two using addition of Hendrickson-Lattman coefficients. These combined phases are then used to score the phasing obtained by combining the two derivatives. The best combinations are iteratively combined until all available derivatives are considered and combined in an optimal fashion. Once an optimal set of derivatives and sites is found, SOLVE Bayesian correlated phasing is used to calculate a final set of native phases from the native and all the derivatives at once. Here is the best pair of derivatives from this first cycle:

```
Getting origin shift for 1 mapped on to 4
Keeping order of datasets for merge 2.4307589843 2.4307589843
Phases from solution 4:solve_4.mtz
Phases from solution 1:solve_1.mtz
Merged ha files in ha_4_1.pdb
Merged files in merged_4_1.mtz
FOM solution 4: 0.486 FOM solution 1: 0.415 Correlation of maps: 0.247 Ideal map correlation: 0.20169

RESULT: FOM solution 4: 0.486 FOM solution 1: 0.415 Correlation of maps: 0.247 Ideal map correlation: 0.20169
Origin offset of solution 1: [-0.5, 0.0, 0.0]
```

Here solutions 1 and 4 have a map correlation of 0.25, just about the same as expected (0.20) based on the FOM of the two solutions (0.49 and .44) and assuming random errors. The two solutions differ by an origin shift of 0.5 along x.

The two solutions are then phased as a group to use as the basis for density modification:

```
Merging a set of solutions and phasing the group with SOLVE
...
PHASED SOLUTION: Solution 9 based on MIR phasing starting from solutions 4
(dataset #2) and 1 (dataset #1)
...
AutoSol_run_1_/TEMP0/resolve.scores SKEW 0.1159246
AutoSol_run_1_/TEMP0/resolve.scores CORR_RMS 0.8839763

CC-EST (BAYES-CC) SKEW : 33.5 +/- 32.8
CC-EST (BAYES-CC) CORR_RMS : 57.2 +/- 34.9
ESTIMATED MAP CC x 100: 45.4 +/- 22.8
```

Though worse than the HGKI solution by itself, this is reasonably good solution, with a moderate a positive skew (0.12), and a good correlation of local rms density (0.88).

As the original HGKI solution was the best, it is used for density modification and finding additional sites:

```
SOLUTION USED TO START DEN MOD:
Solution 4 using HYSS on AutoSol_run_1_/hgki_rd_1_PHX.sca_iso_2.sca and taking inverse. Dataset #2 SG="P 21 21
HKLIN: solve_4.mtz
Testing density modification with mask_type = histograms
RFACTOR: 0.2655
Best mask type so far is histograms
```

Finding additional sites by density modification and heavy-atom difference Fouriers

Heavy-atom sites are found for derivatives that are not yet solved by phasing using the current model, carrying out density modification to improve the phases, and using the improved phases along with isomorphous differences and the phase difference between the heavy atoms and the non-heavy atoms to calculate Fourier maps showing the positions of the heavy atoms. The top peaks in these maps are used as trial heavy-atom sites (if they are not already part of the heavy-atom model).

In this example solution 4 from derivative 2 is used for this phasing/density modification/Fourier procedure. Sites are found for all the derivatives and new solutions are created and scored using the top sites for each derivative. The combinations are then tested as above, and the highest-scoring ones are kept again. The best solution found is #96:

```

PHASED SOLUTION: Solution 96 based on MIR phasing starting from solutions 4 (dataset #2) and 14 (dataset #1)
...
AutoSol_run_1_/TEMP0/resolve.scores SKEW 0.4449184
AutoSol_run_1_/TEMP0/resolve.scores CORR_RMS 0.9306632

CC-EST (BAYES-CC) SKEW : 71.3 +/- 11.3
CC-EST (BAYES-CC) CORR_RMS : 63.3 +/- 28.2
ESTIMATED MAP CC x 100: 71.9 +/- 10.4

```

This is quite a good solution, with high skew (0.44) and correlation of local rms density (0.93). This solution is the best overall and is used for final phasing and density modification. Notice that it only contains two of the five derivatives. The merging procedure identifies which combinations of derivatives give the best phasing, and all the other derivatives are ignored.

Final phasing with SOLVE

Once the best heavy-atom solution or solutions are chosen based on Z-scores, these are used in a final round of phasing with SOLVE (for MIR phasing). In this case several nearly-equally-good solutions are available, and all are used in phasing, density modification and initial model-building, with the R-factor in density modification and the model-map correlation in model-building being used to identify the best solutions. The log file from phasing for solution 96 is in solve_96.prt. The heavy-atom model is refined and phases are calculated with Bayesian correlated MIR phasing. An important part of this phasing method is a statistical method of taking into account the correlation of non-isomorphism among derivatives. The extent of this correlation is listed in the solve_96.prt summary file:

```

SUMMARY OF CORRELATED ERRORS AMONG DERIVATIVES

DERIVATIVE: 1
CENTRIC REFLECTIONS:
DMIN: ALL 8.91 5.58 4.35 3.68 3.25 2.94 2.71 2.52
RMS errors correlated and uncorrelated with others in group:
  Correlated: 54.6 67.6 58.3 57.0 65.3 58.5 34.7 38.1 37.5
  Uncorrelated: 49.6 64.3 62.8 50.1 46.7 41.4 50.8 32.9 29.2

Correlation of errors with other derivs:
DERIV 2: 0.56 0.60 0.51 0.52 0.61 0.63 0.38 0.49 0.58

```

Here the centric reflections in derivative 1 have non-isomorphism errors related to those in derivative 2, with a correlation coefficient overall of 0.58. another way to look at this is that the RMS correlated error is 54.6 and the RMS uncorrelated (random) error is 49.6. That means that a big part of the errors are correlated, and should be treated as such. The final occupancies and coordinates are listed at the end:

```

SITE ATOM OCCUP X Y Z B
CURRENT VALUES: 1 Hg 0.3744 0.2772 0.2197 0.4194 6.9985
CURRENT VALUES: 2 Hg 0.4444 0.8110 0.3415 0.4388 24.1644
CURRENT VALUES: 3 Hg 0.3327 0.2629 0.2488 0.4174 21.4129
CURRENT VALUES: 4 Hg 0.0684 0.2568 0.1753 0.3437 11.1209
CURRENT VALUES: 5 Hg 0.0918 0.3076 0.2496 0.4639 39.3362

SITE ATOM OCCUP X Y Z B
CURRENT VALUES: 1 Au 0.3856 0.7926 0.3138 0.4669 19.0809
CURRENT VALUES: 2 Au 0.4300 0.2877 0.2163 0.4266 19.5977
CURRENT VALUES: 3 Au 0.3315 0.6380 0.1629 0.4836 15.1735
CURRENT VALUES: 4 Au 0.1238 0.8116 0.3356 0.4366 1.0000
CURRENT VALUES: 5 Au 0.2690 0.2873 0.2161 0.4832 7.4303

```

In this case the occupancies of the top sites are about 1/3, which is fine for MIR (particularly with such heavy atoms as Hg and Au).

Statistical density modification with RESOLVE

After MIR phases are calculated with SOLVE, the AutoSol Wizard uses RESOLVE density modification to improve the quality of the electron density map. The statistical density modification in RESOLVE takes advantage of the flatness of the solvent region and the expected distribution of electron density in the region containing the macromolecule, as well as any NCS that can be found from the heavy-atom substructure. The weighted structure factors and phases (FP, PHIB) from SOLVE are used to calculate the starting map for RESOLVE, and the experimental structure factor amplitudes (FP) and MIR Hendrickson-Lattman coefficients from SOLVE are used in the density modification process. The output from RESOLVE for solution 107 can be found in resolve_96.log. Here are key sections of this output.

First, the plot of how many points in the "protein" region of the map have each possible value of electron density. The plot below is normalized so that a density of zero is the mean of the solvent region, and the standard deviation of the density in the map is 1.0. A perfect map has a lot of points with density slightly less than zero on this scale (the points between atoms) and a few points with very high density (the points near atoms), and no points with very negative density. Such a map has a very high skew (think "skewed off to the right"). This map is good, with a positive skew, though it is not perfect.

```
Plot of Observed (o) and model (x) electron density distributions for protein
region, where the model distribution is given by,
  p_model(beta*(rho+offset)) = p_ideal(rho)
and then convoluted with a gaussian with width of sigma
where sigma, offset and beta are given below under "Error estimate."
```

```
0.03.....
. . .
. . .
. xxxxx .
. x0 oxx .
. x . x0 .
. x . xx .
p(rho) . x . xx .
. x . xx0 .
. xx . x00 .
. ox . xxx0 .
. ox . xx .
. ox . oxxx .
. oxx . xxx .
. xxx . oxxxxx .
0.0 xxxx.....0xxxxx

-2 -1 0 1 2 3

normalized rho (0 = mean of solvent region)
-----
```

After density modification, the curve is more ideal, with a very strong positive skew:

```
0.03.....
. . .
. . .
. xxxxxx . .
. x oxx . .
. x xx .
. x xx .
p(rho) . x . oxx .
. xx . xxx .
. xx . oxxx .
. x . xxxxx .
. xx . xxxxxx 0 .
. x0 . xxxxxx00 .
. xx0 . xxxxxx00
x00 . x0
0.0 0.....x
```

```
-2 -1 0 1 2 3
```

normalized rho (0 = mean of solvent region)

The key statistic from this RESOLVE density modification is the R-factor for comparison of observed structure factor amplitudes (FP) with those calculated from the density modification procedure (FC). In this rh-dehalogenase MIR phasing the R-factor is very low:

Overall R-factor for FC vs FP: 0.253 for 12293 reflections

An acceptable value is anything below 0.35; below 0.30 is good.

Generation of FreeR flags

The AutoSol Wizard will create a set of free R flags indicating which reflections are not to be used in refinement. By default 5% of reflections, (up to a maximum of 2000) are reserved for this test set. If you want to supply a reflection file hires.mtz that has higher resolution than the data used to solve the structure, or has a test set already marked, then you can do this with the keyword input_refinement_file=hires.mtz. The log file tells what file is created:

```
Adding FreeR_flag to AutoSol_run_1_/TEMP0/solve_96.mtz
```

```
Label for column with FP is 'FP' for  
the file AutoSol_run_1_/TEMP0/solve_96.mtz  
Done with adding free R set
```

```
FreeR_flag added to solve_96.mtz
```

```
New file: TEMP0.mtz
```

```
New labin: LABIN FP=FP PHIB=PHIB FOM=FOM HLA=HLA HLB=HLB HLC=HLC HLD=HLD FreeR_flag=FreeR_flag  
Copying TEMP0.mtz to exptl_fobs_phases_freeR_flags_96.mtz  
Columns used: LABIN FP=FP PHIB=PHIB FOM=FOM HLA=HLA HLB=HLB HLC=HLC HLD=HLD FreeR_flag=FreeR_flag
```

```
Checking for HL coeffs in exptl_fobs_phases_freeR_flags_96.mtz True
```

```
Refinement file with freeR flags is in AutoSol_run_1_/exptl_fobs_phases_freeR_flags_96.mtz
```

The files to be used for model-building are listed in the AutoSol log file:

```
THE FILE AutoSol_run_1_/resolve_96.mtz will be used for model-building  
THE FILE exptl_fobs_phases_freeR_flags_96.mtz will be used for refinement
```

Model-building with RESOLVE

The AutoSol Wizard by default uses a very quick method to build just the secondary structure of your macromolecule, and then will try to extend that model with standard model-building. This process is controlled by the keywords helices_strands_start=True. and helices_strands_only=False . The Wizard will guess from your sequence file whether the structure is protein or RNA or DNA (but you can tell it if you want with (chain_type=PROTEIN).

If the quick model-building does not build a satisfactory model (if the correlation of map and model is less than acceptable_secondary_structure_cc=0.35), then model-building is tried again with the standard build procedure, essentially the same as one cycle of model-building with the AutoBuild Wizard (see the web page Automated Model Building and Rebuilding with AutoBuild, except that if you specify thoroughness=quick as we have in this example, the model-building is done less comprehensively to speed things up.

In this case the secondary-structure-only model-building using solution #96 produces an initial model with 179 residues built and side chains assigned to 130, and which has a model-map correlation of 0.53:

```

Secondary-structure model: AutoSol_run_1_/TEMP0/Build_1.pdb
Log file: Build_1.log copied to Build_1.log
Models to combine and extend: ['Build_1.pdb']
Using CC to score in combine_extend
Model 2: Residues built=179 placed=130 Chains=8 Model-map CC=0.53 (Build_combine_extend_2.pdb)
This is new best model with cc = 0.53
Refining model: Build_combine_extend_2.pdb
Model: AutoSol_run_1_/TEMP0/refine_2.pdb R/Rfree=0.41/0.45

```

This is quite an adequate preliminary model. It is then extended in several cycles and quite a good model is produced:

```

Current overall_best model and map Thu Dec 18 16:21:17 2008
Working directory: /net/sunbird/scratch1/terwill/run_121808a/rh-dehalogenase-mir
/AutoSol_run_1_
Model (overall_best.pdb) from: refine_8.pdb
R and R-free: 0.20 0.23
Map-model CC: 0.82
Model-building logfile (overall_best.log) from: model_with_loops_9.log
Model evaluation (overall_best.log_eval) from: refine_8.pdb.log_eval
Map coeffs used for build (overall_best_denmod_map_coeffs.mtz)
from: map_coeffs.mtz
SigmaA map coeffs (overall_best_refine_map_coeffs.mtz)
from: refine_map_coeffs_8.mtz

```

For full model-building you will want to go on and use the AutoBuild Wizard (see the web page Automated Model Building and Rebuilding with AutoBuild)

The AutoSol_summary.dat summary file

A quick summary of the results of your AutoSol run is in the AutoSol_summary.dat file in your output directory. This file lists the key files that were produced in your run of AutoSol (all these are in the output directory) and some of the key statistics for the run, including the scores for the heavy-atom substructure and the model-building and refinement statistics. These statistics are listed for all the solutions obtained, with the highest-scoring solutions first. Here is part of the summary for this rh-dehalogenase MIR dataset:

```

-----CURRENT SOLUTIONS FOR RUN 1 : -----

*** FILES ARE IN THE DIRECTORY: AutoSol_run_1_ ***

Solution # 96 BAYES-CC: 71.9 +/- 10.4 Dataset #0 FOM: 0.6

Solution 96 based on MIR phasing starting from solutions 4 (dataset #2) and 14 (dataset #1)
This solution is a composite of solutions: 4 14 (Already used for Phasing at resol of 2.44) Refined Sites: 5
NCS information in: AutoSol_96.ncs_spec
Experimental phases in: solve_96.mtz
Experimental phases plus FreeR_flags for refinement in: exptl_fobs_phases_freeR_flags_96.mtz
Density-modified phases in: resolve_96.mtz
HA sites (PDB format) in: ha_96.pdb_formatted.pdb
Sequence file in: sequence.dat
Model in: refine_8.pdb
  Residues built: 283
  Side-chains built: 283
  Chains: 0
  Overall model-map correlation: 0.82
  R/R-free: 0.2/0.23
Phasing logfile in: solve_96.prt
Density modification logfile in: resolve_96.log (R=0.25)
Build logfile in: model_with_loops_9.log

Score type: SKEW CORR_RMS
Raw scores: 0.44 0.93
100x EST OF CC: 71.32 63.28

```



```

Refined heavy atom sites (fractional):
deriv 1
xyz 0.277 0.220 0.419
xyz 0.811 0.342 0.439
xyz 0.263 0.249 0.417
xyz 0.257 0.175 0.344
xyz 0.308 0.250 0.464
deriv 2
xyz 0.793 0.314 0.467
xyz 0.288 0.216 0.427
xyz 0.638 0.163 0.484
xyz 0.812 0.336 0.437
xyz 0.287 0.216 0.483

```

How do I know if I have a good solution?

Here are some of the things to look for to tell if you have obtained a correct solution:

- How much of the model was built? More than 50% is good, particularly if you are using the default of `helices_strands_only=True`. If less than 25% of the model is built, then it may be entirely incorrect. Have a look at the model. If you see clear sets of parallel or antiparallel strands, or if you see helices and strands with the expected relationships, your model is going to be correct. If you see a lot of short fragments everywhere, your model and solution is going to be incorrect. How many side-chains were fitted to density? More than 25% is ok, more than 50% is very good.
- What is the R-factor of the model? This only applies if you are building a full model (not for `helices_strands_only=True`). For a solution at moderate to high resolution (2.5 Å or better) the R-factor should be in the low 30's to be very good. For lower-resolution data, an R-factor in the low 40's is probably largely correct but the model is not very good.
- What are the individual CC-BAYES estimates of map correlation for your top solution? For a good solution they are all around 50 or more, with 2SD uncertainties that are about 10-20.
- What is the overall "ESTIMATED MAP CC x 100" of your top solution. This should also be 50 or more for a good solution. This is an estimate of the map correlation before density modification, so if you have a lot of solvent or several NCS-related copies in the asymmetric unit, then lower values may still give you a good map.
- What is the difference in "ESTIMATED MAP CC x 100" between the top solution and its inverse? If this is large (more than the 2SD values for each) that is a good sign.

What to do next

Once you have run AutoSol and have obtained a good solution and model, the next thing to do is to run the AutoBuild Wizard. If you run it in the same directory where you ran AutoSol, the AutoBuild Wizard will pick up where the AutoSol Wizard left off and carry out iterative model-building, density modification and refinement to improve your model and map. See the web page [Automated Model Building and Rebuilding with AutoBuild](#) for details on how to run AutoBuild.

If you do not obtain a good solution, then it's not time to give up yet. There are a number of standard things to try that may improve the structure determination. Here are a few that you should always try:

- Try setting `thoroughness=thorough` if it had previously been set to `quick`. This can make a big difference, though it takes longer.
- Try setting `max_choices` to a larger number, or `desired_coverage` to a higher value.

- Have a careful look at all the output files. Work your way through the main log file (e.g., `AutoSol_run_1_1.log`) and all the other principal log files in order beginning with scaling (`dataset_1_scale.log`), then looking at heavy-atom searching (e.g., `auki_rd_1_PHX.sca_iso_1.sca_hyss.log`), phasing (e.g., `solve_96.log` or `solve_xx.log` depending on which solution xx was the top solution) and density modification (e.g., `resolve_xx.log`). Is there anything strange or unusual in any of them that may give you a clue as to what to try next? For example did the phasing work well (high figure of merit) yet the density modification failed? (Perhaps the hand is incorrect). Was the solvent content estimated correctly? (You can specify it yourself if you want). What does the xtriage output say? Is there twinning or strong translational symmetry? Are there problems with reflections near ice rings? Are there many outlier reflections?
- Try `thoroughness=thorough` instead of `quick`.
- Try a different resolution cutoff. For example 0.5 Å lower resolution than you tried before. Often the highest-resolution shells have little useful information for structure solution (though the data may be useful in refinement and density modification).
- Try a different rejection criterion for outliers. The default is `ratio_out=10.0` (toss reflections with delta F more than 10 times the rms delta F of all reflections in the shell). Try instead `ratio_out=3` to toss outliers.
- If the heavy-atom substructure search did not yield plausible solutions, try searching with HYSS using the command-line interface, and vary the resolution and number of sites you look for. Can you find a solution that has a higher CC than the one found in AutoSol? If so, you can read your solution in to AutoSol with `sites_file=my_sites.pdb`.
- Was an anisotropy correction applied in AutoSol? If there is some anisotropy but no correction was applied, you can force AutoSol to apply the correction with `correct_aniso=True`.
- Try including more phased solutions from each derivative with the keyword `min_phased_each_deriv=8` instead of the default 1.
- Try including more combinations of solutions with the keyword `max_composite_choices=16` instead of the default 8.
- Try related space groups. If you are not positive that your space group is P212121, then try other possibilities with different or no screw axes.

Additional information

For details about the AutoSol Wizard, see Automated structure solution with AutoSol. For help on running Wizards, see Using the PHENIX Wizards.

Tutorial: Validation with MolProbity

molprobity.html

- Overview
- Setup
- File inputs
- Validation tools overview Polygon Opening Coot MolProbity Validation

Validation tools overview

Polygon Opening Coot MolProbity Validation

- Polygon
- Opening Coot
- MolProbity Validation
- Making general corrections Rotamers C-Beta Deviations N/Q/H Flips Clashes Notes

Making general corrections

Rotamers C-Beta Deviations N/Q/H Flips Clashes Notes

- Rotamers
- C-Beta Deviations
- N/Q/H Flips
- Clashes
- Notes
- Cis-nonProlines overview
- Making cis-peptide corrections Justified cis conformations Mistaken cis conformations Testing both cis and trans

Making cis-peptide corrections

Justified cis conformations Mistaken cis conformations Testing both cis and trans

- Justified cis conformations
- Mistaken cis conformations
- Testing both cis and trans
- References

Overview

This tutorial will show you how to do comprehensive model validation within the PHENIX graphical user interface (GUI). (Validation in the PHENIX GUI reviews many of the aspects of this tutorial.)

Setup

The example used for this tutorial is called Protein Kinase A (3dnd). To get the requisite files open the GUI and set up the tutorial, as described here, called Protein Kinase A (validation). This should setup the main/GUI window. On the right-hand side, under Crystals, click Validation and map-based comparisons and

then select Comprehensive Validation (MolProbity).

File inputs

In the "Comprehensive Validation" window, browse to the tutorial directory that you specified above, and select 3dnd.pdb as the "Input model", 3dnd.mtz as the "Reflections file", and 3dnd.ligands.cif as the "Restraints (CIF)" file. This CIF file defines restraints for ligands in input PDB file.

For this model, adding certain validations to the Polygon representation is useful. Scroll down to find the button labeled "Polygon options" and click it. In the window that pops up, turn on the checkboxes for "Ramachandran favored" and "Rotamer outliers". Then hit OK.

Now you are ready to run the program, select Run from the top menu bar.

Validation tools overview

Polygon

When validation has run, click on the "Compare statistics" button. This will launch a tool called "Polygon" (it may take several seconds), which is used to simultaneously compare R-work, R-free, RMSD-bonds, RMSD-angles, Clashscore, and Average B factor. Well-built models will usually have a small, fairly equilateral polygon, whereas larger or significantly asymmetric deviations are indicative of model problems.

As you can see for 3dnd, both RMSD-bonds and RMSD-angles are a bit large, beyond the expected peaks for structures of similar resolution. This could indicate that misfit areas of the structure are causing geometric strain. We will visit some of these areas in the next sections.

From the measures we added, you can also see that Rotamer outliers are quite high. Ramachandran favored looks unusual, but this measure should have a high value, unlike the others. More importantly, this structure's Ramachandran favored score falls around the peak of Polygon's histogram for structures of similar resolution. This is important context for choosing which validations to focus on.

You may close the "POLYGON" window and return to the comprehensive validation window.

Opening Coot

Click the "Open in Coot" button. When Coot loads, note that it says "Connected to PHENIX" in the toolbar - this allows the validation GUI to communicate with Coot.

If Coot does not load, go to Preferences->Graphics in the main Phenix window to specify a path to your Coot installation.

MolProbity Validation

Now select the "MolProbity" tab in the Phenix Comprehensive validation window. Under this tab, you will see sub-tabs for "Summary", "Basic geometry", "Protein", and "Clashes". If your model contained RNA, there would also be an "RNA" tab.

Summary

Under the "Summary" tab are most of the same overall statistics that you would find when running the MolProbity webserver. These statistics are colored stoplight-style (green, yellow, and red) as a rough guide the severity of a given result.

Notice the high percentage of rotamer outliers, and large number of C-beta deviations (those combine geometry problems around the C-alpha into a single measure, as explained in Lovell 2003). The Ramachandran favored percent appears a little low, and is colored yellow. However, as we saw in the Polygon above, this value is typical of structures around 2.26Å. The Ramachandran statistics might be improvable, but other validations take higher priority in this context.

This tab also displays Rama-Z score validation. This is a whole-model assessment of how realistic the Ramachandran distribution is and guards against overfitting to Ramachandran criteria. This structure has a reasonable Ramachandran distribution, with all its Rama-Z scores falling within +2 to -2.

Basic Geometry

From the MolProbity tab, click on the "Basic geometry" tab. Here you find a summary of all bond, angle, dihedral, chirality, and planarity outliers. Outliers will be listed in the associated lists, and each item is clickable, which will center in Coot. Note the bond outlier for Ile A 163 - we'll be seeing this residue again later.

Protein-specific validations

Next, go to the "Protein" tab. This section contains validation information for Ramachandran and rotamer outliers, C-beta deviations, recommended Asn/Gln/His sidechain flips (these have NOT already been done for you, as you can tell if you click on His 39 in the flip list to see it in Coot), and non-trans peptide bonds.

RNA-specific validations

If this structure contained any RNA molecules, there would be an additional tab named "RNA". This section contains validation information for RNA nucleotides with covalent geometry outliers (also found in the Basic geometry tab), RNA nucleotides with sugar pucker outliers, and RNA suites with unexpected backbone conformations.

All-atom Contacts

Go to the "Clashes" tab. This section contains a list of all the steric overlaps $>0.4\text{\AA}$ (i.e. "clashes") in the structure, sorted by severity. Scroll down the list and note that almost all the clashes involve at least one Hydrogen atom. Hydrogens form most of the steric contacts in a structure and are crucial to contact analysis! Phenix, MolProbity, and Coot will generally ensure that hydrogens are added when you need them, but make sure to use phenix.reduce to add hydrogens manually if you are working outside these systems.

Making General Corrections

Refinement programs are excellent at optimizing details, but may struggle with large changes across energy barriers. Our corrections are therefore focused on getting residues over energy barriers and into the right local minimum.

Rotamers

Select the Protein tab and find the list of rotamer outliers. Click on Leu 27 from the A chain, which will center on this residue's CA in the Coot window.

As you can see in Coot, this orientation is not a terrible fit to the density - but it is a rotamer outlier and energetically unfavorable due to an eclipsed Chi angle. You can see the eclipsed conformation by looking down the CG-CB bond and noting that the CD1 atom nearly overlaps with the backbone CA. Recentering on CB may help you arrange this viewpoint. The residue also has a suggestive positive difference peak near the CD1.

We'll use the tools in Coot to fix this sidechain. Select the "Auto Fit Rotamer" tool. You can find it in the default toolbar or in Calculate-> Model/Fit/Refine. It looks like a sidechain surrounded by green. Then click on an

atom in the Leu 27 sidechain. Did you see it rotate ~180 degrees?

To confirm the correctness of this change, use the "Undo" and "Redo" buttons to observe the change again. Look down the CG-CB bond as before. You should see that the sidechain has a favorable, staggered conformation after the change. Also note that the CD2 atom is now pointed towards the positive difference density peak - you've got the atom close enough to the correct position that refinement should be able to sort out the details.

C-Beta Deviations

Return to the Validation GUI window, and find the list of C-beta position outliers. Select Ile 163 from the A chain, which will center on this residue's CB atom in the Coot window. This sidechain also appears in the rotamer outliers list on this tab and on the bond-length outliers list on the Basic Geometry tab. Misfit sidechains will often have multiple diagnostic indicators of a problem, which is useful in identifying the worst offenders. Also note the large blob of positive density near to the sidechain, further evidence that it may not be in an optimal position.

Select the "Auto Fit Rotamer" and then click an atom in the Ile 163 sidechain. Alternatively, you can define a Real Space Refine Zone around this sidechain, and drag the CG2 atom of the short arm into the large difference density peak. Use the "Undo" and "Redo" to toggle back and forth across the change. Note that the CA and especially the CB atoms have moved significantly.

Correcting misfit sidechains such as these can be tricky, as many rounds of refinement within the wrong local minimum have caused distortions in the model to accommodate the misfit. It would be important to revisit this region after another refinement. In this case, you can expect to find position of this Ile further improved, as the neighboring atoms are able to recover from the strain caused by the initial outlier.

N/Q/H Flips

Return to the Validation GUI window, and find the list of Backwards Asn/Gln/His sidechains.

Asparagine, Glutamine, and Histidine sidechains are pseudo symmetric. They can easily be fit into electron density backwards, eliminating hydrogen bonds and introducing clashes. Hydrogen addition via Reduce (in Phenix, Coot, or MolProbity) will usually "flip" these N/Q/H residues to the positions with best steric contacts. Here, you can use Coot to force a necessary Histidine flip.

Select HIS 39 from the A chain, which will center on this residue in the Coot window. Note that the CD2 atom of His 39 and the N atom of Asp 41 are involved in a serious clash. If we flipped the His 180°, then the ND1 atom would be in a position to form a hydrogen bond with the backbone NH where this clash is.

Define a Real Space Refine Zone that includes HIS 39. In the refinement menu that pops up, look under "Active Refinement" and click the "Side-chain Flip 180°" button. (This flip only works on N/Q/H residues; use the Auto Fit Rotamer tool for other tasks.)

Clashes

Return to the Validation GUI window, and navigate to the Clash tab.

Steric clashes usually serve as emphasis on other model problems. However, there are some errors that are clearly diagnosed by clashes alone, such as N/Q/H flips and certain misplaced water molecules. There are many tools for identifying misplaced waters, and clashes are excellent for finding the most problematic ones.

Select "A 177 GLN HG3 ____ A 554 HOH O" from the list of clashes, which will center on this clash in the Coot window. Scroll the 2Fo-Fc map contour down, and note that this water has only marginal density even at a low contour. There's no justification for keeping a water in this position.

To remove this water, select the "Delete item" tool. In the Coot sidebar, it looks like a trash can. In the Delete item tool, toggle the Water mode, then click on the offending water.

The next water clash on the list, "A 275 VAL HG21 ____ A 577 HOH O", is another good example of an unjustified water marked by a clash. Try using Coot to remove it as well.

Notes

The kind of sidechain fixups you've done in KiNG or Coot can mostly be accomplished using real-space refinement in phenix.refine (which includes a rotamer correction component), up to somewhere between 2 and 2.5 Å (and of course NQH flips are done automatically by Reduce in either MolProbity or Phenix). At lower resolution, however, real-space refinement with rotamer correction cannot reliably discern between correct and incorrect rotamers. That's a hard job for people as well, but can often be done if you have the interactive information on clashes and H-bonds from the non-pairwise, H-aware all-atom-contact dots.

You can correct many of the other outliers in the GUI lists, especially the sidechains. Try a few!

Cis-nonProlines Overview

You will need another set of tutorial data for this section. In the main Phenix window, go to New Project -> Set up tutorial data -> "Validation - xyloglucanase cis peptides".

If you are starting from the main Phenix window, select Comprehensive validation (X-ray/Neutron) as shown above. If you still have the comprehensive validation window open, return to the "Configure" tab. Select 2cn3.pdb as the input model and 2cn3.mtz as the reflections file. We will not use a restraints file this time, so clear that field if necessary. Now run the validation; this will take a few moments.

A type of outlier that can be very difficult for real-space refinement alone to resolve is an incorrect cis-peptide. The peptide bond between carbon and nitrogen joins adjacent amino acids. Due to the nearby carbonyl, the peptide bond has partial double bond character and does not rotate freely. Most peptides are trans, but genuine cis peptides occur preceding 5% of prolines and preceding about 1 in 3000 non-proline residues. Despite its rarity, a cis conformation can be tempting to model at lower resolutions because it may appear to fit constricted or patchy density. Once modeled, cis peptides are difficult to escape, since doing so would involve rotating 180 degrees through a high energetic barrier. Fortunately, there is a tool in Coot to simplify human-directed corrections.

Once the validation completes, open the model in Coot. Then move to the MolProbity tab of the validation output, and then the Protein tab within MolProbity. Scroll down to the bottom of the Protein tab to find Cis and Twisted peptides validation. Note the overall model statistics and the suspiciously high incidence of non-trans peptides in this model.

Peptide bonds necessarily span two residues, so both are listed in the validation output table. However, in shorthand, peptide bonds are properly associated with their following residue, due to the special relationship cis peptide bonds have with a following proline. Thus, "chain A GLY 161 to GLU 162" is a cis-Glu, and "chain A, GLY 293 to PRO 294" is a cis-Pro.

Making Cis-peptide Corrections

This structure is a xyloglucanase. Real cis-nonProline conformations are more common in carbohydrate-active enzymes like this one than elsewhere. As a result, this structure contains both real and incorrect cis peptides.

Justified cis conformations

We'll look at a real example first. Click on "GLY 161 to GLU 162" in the table, which will center Coot on the N atom of the peptide bond of interest. Note that Coot marks cis peptides with a colored trapezoid (green for cis-Proline, red for cis-nonProline, yellow for twisted). Scroll the electron density to a higher contour to convince yourself that cis truly is the best conformation to fit this region. You need this kind of strong support - from experimental data, homology, or chemistry - to justify cis-nonProlines.

Mistaken cis conformations

Now we will fix a mistake. Click on "LYS 269 to GLY 270" to center the Coot window on this peptide bond. In addition to the red trapezoid marking the cis-peptide, there are large areas of difference density around the peptide bond. Also note the clash between the GLY 270 CA and the LYS 247 O.

To perform the correction: Go to Calculate -> Other Modeling Tools from the Coot menu bar, and select Cis <-> Trans. Click on one of the atoms in the offending peptide bond, and Coot will flip the bond. The bond conformation has now been corrected, but the atoms have moved out of the density and geometry distortions have been introduced. Select Real Space Refine Zone, and then select atoms on either side of the peptide bond. This correction involved a large change, so be sure to give Coot at least two full residues on either side of the bond to work with. Once you have obtained a satisfactory local refinement, note that the GLY 270 N is now in position to form a hydrogen bond with the LYS 247 O, where the clash was.

ASN 298 to GLY 299 is another good example of a region with marginal electron density and a mistaken cis-peptide. Select it from the list and try fixing it. As a bonus, you can then move the ASN 298 sidechain into some highly likely positive difference density.

Testing both cis and trans

Finally, select "MET 348 to ASN 349" from the list. This region has no significant difference density, and the 2Fo-Fc map is fairly convincing. Nevertheless, let's test it. Try changing this peptide to trans and do a local refinement.

It's not a good fit to the density! Convince yourself that there's no way to get even a marginal fit from the trans conformation, then hit "Undo" until you get the cis-Asn back. Trying both possibilities like this can be a useful strategy.

References

Structure validation by Calpha geometry: phi,psi and Cbeta deviation. S.C. Lovell, I.W. Davis, W.B. Arendall, P.I. de Bakker, J.M. Word, M.G. Prisant, J.S. Richardson, and D.C. Richardson. Proteins 50, 437-50 (2003).

- Structure validation by Calpha geometry: phi,psi and Cbeta deviation. S.C. Lovell, I.W. Davis, W.B. Arendall, P.I. de Bakker, J.M. Word, M.G. Prisant, J.S. Richardson, and D.C. Richardson. Proteins 50, 437-50 (2003).

MolProbity: More and better reference data for improved all-atom structure validation. C.J. Williams, B.J. Hintze, J.J. Headd, N.W. Moriarty, V.B. Chen, S. Jain, S.M. Prisant MG Lewis, L.L. Videau, D.A. Keedy, L.N. Deis, I.I.I. Arendall WB, V. Verma, J.S. Snoeyink, P.D. Adams, S.C. Lovell, J.S. Richardson, and D.C. Richardson. Protein Science 27, 293-315 (2018).

- MolProbity: More and better reference data for improved all-atom structure validation. C.J. Williams, B.J. Hintze, J.J. Headd, N.W. Moriarty, V.B. Chen, S. Jain, S.M. Prisant MG Lewis, L.L. Videau, D.A. Keedy, L.N. Deis, I.I.I. Arendall WB, V. Verma, J.S. Snoeyink, P.D. Adams, S.C. Lovell, J.S. Richardson, and D.C. Richardson. Protein Science 27, 293-315 (2018).

Crystallographic model quality at a glance. L. Urzhumtseva, P.V. Afonine, P.D. Adams, and A. Urzhumtsev. Acta Cryst. D65, 297-300 (2009).

- Crystallographic model quality at a glance. L. Urzhumtseva, P.V. Afonine, P.D. Adams, and A. Urzhumtsev. *Acta Cryst. D* 65, 297-300 (2009).

Tutorial: solving a structure with Phaser-MR

mr.html

Overview

This tutorial shows how to use the Phaser-MR GUIs to solve structures by molecular replacement, using example data included with the Phenix distribution. We recommend reading the MR overview before running these tutorials (and for adventurous users, the primary Phaser reference as well).

A brief word about terminology: the search models used in Phaser are customarily named "ensembles", and in fact Phaser will often work best in difficult cases when a set of superposed models (generated using Ensembler or similar tools) is used. However, this is not required and for easily solved structures, such as the tutorial datasets used here, a single PDB file will be sufficient. Furthermore, while Phaser can search for multiple alternative ensembles and output only the best solution, here we will use just one candidate per component. Also note that the search models used here are single proteins, but this is not a requirement if the relative orientation of multiple chains are known (such as physiological dimers, multi-subunit complexes, or DNA helices).

An essential feature of Phaser is that it scores possible solutions based on the expected asymmetric unit cell (ASU) contents (that is, the scattering mass). Therefore, for the examples shown here, we will be supplying sequence files for the crystallized proteins. We could, however, specify the molecular weight or number of residues instead. Importantly, the manner in which the ASU contents is specified is completely decoupled from the "ensemble" (search model) input; where multiple proteins have been crystallized, we could supply separate sequence files (as is done here for the beta-blip example) or a single file with multiple sequences, and the result will be identical. While it is technically possible to run Phaser with just a search model and extrapolate the molecular weight based on the desired number of copies, this is not recommended.

Full automation: phaser.MRage

The separate MRage program (which uses Phaser as its core) can automatically discover and prepare search models, and search in parallel on a multi-core workstation or cluster. A separate tutorial is available on the Phaser home page.

Single-component searches

For homogeneous structures where only one component needs to be placed, a streamlined interface is available. The tutorial data for this example is the A2U-globulin structure, which includes one each of data, PDB, and sequence files. In this interface, it is assumed that multiple PDB files are part of a single ensemble; if you instead want to try multiple equally likely ensembles and let Phaser pick the best one, use the advanced interface (described below).

Start the GUI by clicking "Phaser-MR (simple one-component interface)", the second item in the list under "Molecular replacement." This interface has a single input field for all file types, and on most operating systems you can simply drag the three files from the project folder into the input field. Alternatively, click "Add file" to open a file dialog.

The symmetry information and labels list will automatically be populated when the reflections are loaded; you do not need to change the defaults. You must enter either the sequence identity or RMSD for each model in the ensemble; in this case an RMSD of 1.0 is appropriate for the single model used. If you have no specific idea of the accuracy of the model, it is best to simply provide the sequence identity to the target and let

Phaser provide the corresponding RMSD. Because we already entered a sequence, no molecular weight is required, but the value for "Component copies" should be set to 4.

The next tab contains basic search options; you should change "Number of copies to search for" to 4. (While it may seem inefficient to specify the number of component copies and the number of search copies separately, this would allow us to search for a tetramer, while still only supplying the sequence of one chain.)

Usually it is best not to specify a resolution limit. Phaser chooses a resolution limit based on the particular problem (accuracy and completeness of the model, number of reflections). If the problem is expected to be straightforward, the resolution will automatically be limited to reduce runtime while preserving an excellent chance of solving the structure on the first pass.

The other controls on this tab do not need to be modified, but the option for alternative space groups deserves special mention. If there is any ambiguity about the precise space group, for instance if the data were processed as P222 but the exact combination of screw axes is unknown, or the space group could be either P6(1) or P6(5), Phaser can try multiple space groups and identify the best one at the translation function stage. The default is to try only the original space group and its enantiomer (if any); this will have no effect for the examples used here (the MTZ files already contain the correct symmetry), but it would be appropriate for the P6(1) versus P6(5) case. The most permissive setting is "All possible in the same pointgroup".

Now we are ready to run Phaser: click the Run button on the toolbar. A new tab will appear with a status panel, a list of warnings, and summary output (basically excerpted from the log file). For many structures (including this one) there will be one or more warnings, but these do not necessarily mean that Phaser will not be successful. This example should finish in several minutes.

Once the run is complete, a results tab will appear, showing a summary of the solution and output files, with buttons to launch other programs. The two statistics reported are the log-likelihood gain (LLG), which expresses the probability of this solution compared to a random-atom model, and the translation function Z-score, which denotes the signal-to-noise ratio of the solution. In this case both are excellent, indicating that Phaser found an unambiguous solution.

At this point we recommend inspecting the maps and crystal packing in Coot. Because Phaser only performs rigid-body and group B-factor refinement, the map quality will most likely be relatively poor compared to a refined map, with many unexplained difference peaks, but the map should still be interpretable for a correct solution.

The appropriate next step would be either rebuilding or refinement, both of which can be launched directly from the results tab. For this example we could go straight to refinement; if the structure was instead solved using homologous proteins, running AutoBuild would be a better choice.

Multi-component searches

The interface for multi-component and other more difficult searches is considerably more complicated, especially since the ensemble and ASU contents inputs are split up. The tutorial data for this example is "TEM-1 Beta-lactamase/beta-lactamase inhibitor complex" (or "beta-blip"), and includes five files: an MTZ file and two each of search models and sequence files.

Start the standard Phaser-MR GUI by clicking "Phaser-MR" under the "Molecular replacement" section. The first tab that appears defines the experimental data; click "Browse..." next to the "Data file" field and select the file named "beta_blip_P3221.mtz". The unit cell and space group will be automatically filled in, and the "Data labels" menu will be populated with suitable choices (only one in this case).

The other options in this window are equivalent to those in the simple interface, and do not need to be modified, though it is always a good idea to provide a title for the job.

The second tab, "Ensembles", is where the search models are entered. First we will load the beta-lactamase model: for "Model ID", enter "beta", then click "Add file" below the list widget and select "beta.pdb". This will add it to the list with the "Variance" field blank. Click the "Change variance" button, and enter "100" for the sequence identity and click "OK".

The window should now look like this:

At the bottom of the window, click "Add PDB ensemble". A second ensemble sub-tab will appear. Now repeat the procedure using "blip.pdb" instead. (The order in which these ensembles are input is technically not important, because Phaser will normally determine the appropriate search order automatically. However, in this case Phaser will indeed search for beta first, because it is a significantly larger protein.)

The third configuration tab, "ASU contents", is where we tell Phaser what scattering mass to expect in the crystal. If we know the exact molecular weight (or number of residues) of the entire structure, we could simply enter that here, but we will use sequences instead. This has the advantage that the sequences are then available if we launch AutoBuild from the Results tab. The files "beta.seq" and "blip.seq" should be input in the "Sequence file" field in successive tabs; click the button "Add macromolecule(s)" at the bottom of the window to add a tab. The number of copies is 1 in both cases. (Here the order in which the ASU contents is input really is unimportant.)

The final configuration tab, "Search procedure", is where we tell Phaser how to use the ensembles we input. Initially only the "Component 1" branch is shown. Click on the triple dotted button to its right and a menu will appear allowing us to select which of the two ensembles defined earlier Phaser should search for. The up-down spin button in the rightmost column lets us specify how many copies of this component we want Phaser to attempt to place in the ASU. We select the beta ensemble for the first component and choose to search for one copy. To specify the blip component we press the "Add Search" button, which will add another Component branch to the Search GUI. As before we instruct Phaser to search for one component but with the blip ensemble.

Note: Selecting both ensembles to be searched for in one component would mean that Phaser would try them in turn as alternative choices of model for the same component, and then select the ensemble yielding the best result.

At this point we are ready to run Phaser, which should again take only a few minutes to complete successfully.

Note that the Results tab that appears at the end of the job allows us easily to launch either the phenix.refine or AutoBuild GUI, with many of the fields pre-filled. If the data provided to Phaser had included Friedel pairs (i.e. anomalous data), then a button would also be available to launch the AutoSol GUI for MR-SAD phasing.

Tutorial: Molecular replacement, refinement, and rebuilding (RanGDP mutant)

mr_refine.html

We are grateful for Murray Stewart (MRC LMB, Cambridge) for this example.

This tutorial also requires Coot, which is not distributed with Phenix; see [here](#) for details on obtaining Coot and using it with the Phenix GUI.

Overview

This tutorial is intended for users who are completely unfamiliar with macromolecular crystallography and/or Phenix. It outlines the procedure for solving a structure by molecular replacement, performing initial refinement, and rebuilding the structure in Coot. Because the structure involved (Ran-GDP R76E mutant) is relatively small and moderate resolution, it can be significantly improved in a reasonable amount of time without much difficulty. You should first read the documentation for the Phenix GUI before running this tutorial. We also recommend reading the molecular replacement overview and the documentation for the Xtriage and phenix.refine GUIs.

As with other Phenix tutorial datasets, you can automatically set up a project with the necessary files from the GUI; the example will be listed in the "Molecular replacement" category of the example menu.

There will be three input files for this project: the experimental data and R-free flags (MTZ format), the wild-type RanGDP used as a search model for MR (PDB format), and the sequence (FASTA format).

Step 0: Xtriage

This step is not strictly necessary to solve the structure, but is strongly recommended for any dataset that you are unfamiliar with, especially if you are unsure whether and/or how the structure can be solved. Launch Xtriage by clicking the icon under "Reflection tools" in the main GUI. Only the MTZ and FASTA files are used as input for this example (the "Reference PDB file" option only applies for models which have already been placed correctly and refined against the data).

Once you have entered the files, click the Run button to start Xtriage. The program should finish in a few seconds. Take some time to look through the result tabs and familiarize yourself with the statistics reported [here](#). Since this is high-quality data, there is nothing that requires special handling. (We usually recommend using the original intensities - ideally unmerged - for Xtriage, but since only amplitudes are available for this dataset those can be used instead.) Note that based on the sequence, Xtriage has (correctly) suggested that only one copy is present in the asymmetric unit (ASU) of the crystal.

Step 1: Phaser-MR

To solve the structure we will use the simple Phaser-MR GUI, which should be the first program in the "Molecular replacement" category. This interface is suitable for structures where we only need to use a single search model. All three files (MTZ, PDB, and FASTA) should be used as input. (The sequence is optional; we could instead enter the expected molecular weight or let Phaser guess based on the ensemble, but it is safer and simpler to supply detailed information.)

Note that any time you have a field for a file name, there will be a magnifying glass icon on the same line (in GUIs like Xtriage this will be a separate button). Click this icon to see basic information about the contents of

the file (or for text files, open in an editor). The panel for the PDB file will look like this:

For a reflection file, each array of data will have a separate panel, and you can select which array to view using the drop-down menu:

Right-clicking on the magnifying glass will open a menu with additional options; later in the tutorial you will edit a PDB file this way.

In the "Sequence identity" field, enter "99" (the search model differs in sequence by only a single residue, although some local conformational differences exist.) All other settings can be left on default for this run. Again, click "Run" to start Phaser; this should take several minutes. You can watch the status window to see how molecular replacement is proceeding; we recommend familiarizing yourself with the log output.

Phaser will open a new tab displaying the results when the job is finished. There are several obvious clues that model placement and phasing was successful:

The LLG (log-likelihood gain) is high and positive. This is essentially a measure of the probability of the solution. (Negative scores almost always mean that MR was not successful.) The TFZ (translation function Z-score) is well above 8 (the threshold above which the large majority of solutions are correct). This indicates the signal-to-noise ratio of the solution. Although not displayed in this tab, the R-factor after refinement is well below the threshold for a random agreement between F-obs and F-calc. You can find this in the log output on the status tab if you scroll up (look for "RVAL" in the section labeled "Refinement table"). Note that a higher-than-random R-factor cannot be taken as an indicator that MR failed, since correct solutions will very often have this property (but immediately improve upon running full refinement).

- The LLG (log-likelihood gain) is high and positive. This is essentially a measure of the probability of the solution. (Negative scores almost always mean that MR was not successful.)
- The TFZ (translation function Z-score) is well above 8 (the threshold above which the large majority of solutions are correct). This indicates the signal-to-noise ratio of the solution.
- Although not displayed in this tab, the R-factor after refinement is well below the threshold for a random agreement between F-obs and F-calc. You can find this in the log output on the status tab if you scroll up (look for "RVAL" in the section labeled "Refinement table"). Note that a higher-than-random R-factor cannot be taken as an indicator that MR failed, since correct solutions will very often have this property (but immediately improve upon running full refinement).

At this time you can click "Open in Coot" to view the model and maps calculated by Phaser. However, the map quality will be poor at this stage, so it is not worth making any changes to the model until after refinement.

Step 2: Refinement

You can launch phenix.refine directly from Phaser by clicking the button labeled "Run phenix.refine". This will load the output model and the MTZ file; you should also input the sequence (this is not actually used in refinement, but the post-refinement validation will check for sequence errors in the model).

The appropriate column labels and symmetry information should be loaded for you automatically. You should look at the options on the "Refinement settings" tab to get an idea of what is possible in phenix.refine; however, for this structure the default settings will be fine.

Once refinement is started, a new tab will be added with two sub-tabs for log output and overall status. You should familiarize yourself with the log output, especially as it provides some insight into how refinement is performed in Phenix, but for routine use the graphical view is sufficient.

If you are running Phenix with default settings and have Coot installed, the GUI will start Coot with the current model and maps continuously updated. (You can disable this in the Refinement settings of the preferences.) The R-factors should start to improve immediately upon refinement; both R-work and R-free will drop significantly. (Note that R-free is initially lower than R-work because the structure was not previously refined against this dataset; this is quite common for structures solved by MR.) The geometry should also improve slightly. By the third cycle there is less improvement being made and the R-free will converge near 0.30; the structure could probably benefit from a couple more cycles but it is clear that manual intervention will be needed, and the maps should be much better now.

(Note that depending on the operating system and version of Phenix you are using, the statistics you obtain may not exactly match those shown here; this is normal and not cause for concern unless the differences are large.)

After refinement several more tabs will appear displaying a summary of the results plus a number of validation metrics. A list of output files is shown on the first of these tabs, along with buttons to start Coot or PyMOL. You should now click the Coot button to view the final refined model and maps.

Step 3: Rebuilding

For novice crystallographers, it is worth looking through the structure residue-by-residue, but most of the structure will already be correctly built. The validation pages give us some indication which regions are most problematic, both in terms of geometry and properties of the atoms and maps. In particular, the real-space correlation page lists a number of residues with a poor correlation coefficient (CC) to the electron density:

In general a CC below 0.8 is worrisome; unlike other uses of this metric, even the worst-fit residues will never have a negative correlation. In this model the suspicious residues are clustered in several key regions, which can be visualized with the multi-criterion plot (which also displays geometry outliers):

Clicking on any part of this chart, or any item in the list, will recenter Coot on the indicated residue. For now we will focus on the loop from Val 137 through Arg 140, which has obviously undergone a conformational change:

Although we could attempt to drag the residues into the correct location, it is often easier (and more instructive!) to simply delete them and rebuild from scratch. This has the added advantage that we can calculate a new omit map, which should be clearer with the incorrect residues removed. Delete the four residues, and save the edited model. Return to Phenix, and under the "Maps" category of the main GUI, click "Calculate maps". Load the MTZ file and the edited model from Coot. You should specify a file prefix that clearly indicates the purpose of the maps. All other options can be left alone, since we only need the default map coefficients.

Calculating the maps should only take a few seconds; when complete, a new tab will appear with a button to load the results in Coot (along with the model used for phasing). You should first delete the currently loaded objects from Coot, since these are now redundant. (Alternately, you can close Coot entirely after saving the edited model, since it will be restarted by Phenix if necessary.) The density for the deleted loop should be much clearer now. To rebuild, follow this general outline:

Click the "Add residue" button on the toolbar (it can be found below the mutate buttons; the icon is an alanine residue with a plus sign), then click the C-terminal residue that you want to add to (in this case, Ile 136). Coot will place a new Ala residue; the initial location may be incorrect, but you can use the "Rotate/Translate Zone/Chain/Molecule" button to drag it into approximately the correct position. You can then use the "Mutate and AutoFit" button to change it to the correct residue type (in this case Val; the rest of the loop is FHR). Very important: every time you add a residue, you should run the "Real space refine zone" function, selecting

approximately four residues adjacent to the target. This will correct any geometry errors, and usually improve the fit to the map as well. If you do not run this but instead continue adding residues, distortions in the model will make it more difficult to place and refine each successive residue. The geometry does not need to be ideal at this point, since you will always run another round of full refinement after rebuilding, but it should not show any obvious errors.

- Click the "Add residue" button on the toolbar (it can be found below the mutate buttons; the icon is an alanine residue with a plus sign), then click the C-terminal residue that you want to add to (in this case, Ile 136).
- Coot will place a new Ala residue; the initial location may be incorrect, but you can use the "Rotate/Translate Zone/Chain/Molecule" button to drag it into approximately the correct position. You can then use the "Mutate and AutoFit" button to change it to the correct residue type (in this case Val; the rest of the loop is FHR).
- Very important: every time you add a residue, you should run the "Real space refine zone" function, selecting approximately four residues adjacent to the target. This will correct any geometry errors, and usually improve the fit to the map as well. If you do not run this but instead continue adding residues, distortions in the model will make it more difficult to place and refine each successive residue. The geometry does not need to be ideal at this point, since you will always run another round of full refinement after rebuilding, but it should not show any obvious errors.

Once you are finished the model should look approximately like this:

There are a few other problems with the model, but we will skip some of these to save time. First, however, the mutation of interest in this structure needs to be made; in Phenix, if you included the sequence file the validation output will include a sequence check which flags Arg76 as a mismatch. This can be corrected with Coot's "Mutate and AutoFit" function.

At this point you should again save the current model and return to Phenix.

Step 4: More refinement

Open phenix.refine again and load the original MTZ and FASTA files, and the latest PDB file from Coot. One other problem that we did not fix yet is that the occupancy of the GDP and magnesium atoms have been reset to zero by Phaser (this will be flagged in the "Atomic properties" section of the previous validation results). Phenix provides a simple editor for manipulating the contents of PDB files; right-click on the magnifying glass next to the PDB file in the phenix.refine input file list, and select "Edit structure".

In the editor, click the arrow buttons to open the tree view, and select the magnesium and GDP atom groups (hold down the Shift key to select more than one). Clicking the "Edit" button on the toolbar, or right-clicking the selected atoms, will open a menu with an option to set the occupancy. Select this and enter an occupancy of 1 in the window that appears, then click "Okay". Save the model and exit the editor; the modified PDB file should now have replaced the model from Coot.

We can still use the default refinement settings, but this time it will also be helpful to place solvent atoms, so you should check the "Update waters" button. The R-factors at the start of refinement should already be significantly lower due to the corrected occupancies and rebuilt residues, but refinement will again drop them several more percent.

It is worth trying to complete the structure and correct all of the remaining errors, but if you have followed all of these steps correctly the R-free may already be comparable to the originally published structure.

Revisting completed jobs

Phenix tracks a variety of information about each job run through the GUI, which can be accessed through the job history panel in the main window. If you have completed step 4 it should look something like this:

Select a job and click the button labeled "Show details" to view statistics and input and output files for the job. You may also input additional notes if you wish. The job history interface also allows you to flag a job as a failure or delete it entirely (including the output directory, if any). Double-clicking a job (or clicking the restore button in either the list view or the job info panel) will reload the results and configuration.

Variations

There are several other ways you can use this dataset to learn about Phenix:

Advanced Phaser: you can use the full Phaser-MR GUI instead of the simple one; this is the GUI you would need if the structure contained multiple different components and/or you needed more control over how the search is performed. (Time: several minutes) Ligand fitting: instead of resetting the occupancy for GDP and MG, you could instead delete these residues and run LigandFit to place GDP in the difference map. (Time: approx. 10 minutes, less if multiple processors are available.) AutoBuild: you can let Phenix rebuild the structure for you in AutoBuild. Try setting the "rebuild in place" both ways; True or Auto will move the existing atoms without adding or removing any, False will build an entirely new model. (Time: several hours, depending on number of processors.) Advanced refinement: try experimenting with the settings in phenix.refine to get a feel for how these affect refinement. Simulated annealing may be particularly interesting. (Time: several minutes to ~1 hour, depending on options and number of processors.) If you continue rebuilding the structure after the second refinement, you should use a more conservative strategy, plus hydrogen atoms and TLS.

- Advanced Phaser: you can use the full Phaser-MR GUI instead of the simple one; this is the GUI you would need if the structure contained multiple different components and/or you needed more control over how the search is performed. (Time: several minutes)
- Ligand fitting: instead of resetting the occupancy for GDP and MG, you could instead delete these residues and run LigandFit to place GDP in the difference map. (Time: approx. 10 minutes, less if multiple processors are available.)
- AutoBuild: you can let Phenix rebuild the structure for you in AutoBuild. Try setting the "rebuild in place" both ways; True or Auto will move the existing atoms without adding or removing any, False will build an entirely new model. (Time: several hours, depending on number of processors.)
- Advanced refinement: try experimenting with the settings in phenix.refine to get a feel for how these affect refinement. Simulated annealing may be particularly interesting. (Time: several minutes to ~1 hour, depending on options and number of processors.) If you continue rebuilding the structure after the second refinement, you should use a more conservative strategy, plus hydrogen atoms and TLS.

References

Kent, H.M., Moore, M.S., Quimby, B.B., Baker, A.M., McCoy, A.J., Murphy, G.A., Corbett, A.H., Stewart, M. Engineered mutants in the switch II loop of Ran define the contribution made by key residues to the interaction with nuclear transport factor 2 (NTF2) and the role of this interaction in nuclear protein import. *J Mol Biol.* 1999 289:565-77.

Features and development of Coot. P. Emsley, B. Lohkamp, W.G. Scott, and K.. Cowtan. *Acta Cryst. D*66, 486-501 (2010).

- Features and development of Coot. P. Emsley, B. Lohkamp, W.G. Scott, and K.. Cowtan. *Acta Cryst. D*66, 486-501 (2010).

Towards automated crystallographic structure refinement with phenix.refine. P.V. Afonine, R.W. Grosse-Kunstleve, N. Echols, J.J. Headd, N.W. Moriarty, M. Mustyakimov, T.C. Terwilliger, A. Urzhumtsev, P.H. Zwart, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 68, 352-67 (2012).

- Towards automated crystallographic structure refinement with phenix.refine. P.V. Afonine, R.W. Grosse-Kunstleve, N. Echols, J.J. Headd, N.W. Moriarty, M. Mustyakimov, T.C. Terwilliger, A. Urzhumtsev, P.H. Zwart, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 68, 352-67 (2012).

MolProbity: More and better reference data for improved all-atom structure validation. C.J. Williams, B.J. Hintze, J.J. Headd, N.W. Moriarty, V.B. Chen, S. Jain, S.M. Prisant MG Lewis, L.L. Videau, D.A. Keedy, L.N. Deis, I.I.I. Arendall WB, V. Verma, J.S. Snoeyink, P.D. Adams, S.C. Lovell, J.S. Richardson, and D.C. Richardson. *Protein Science* 27, 293-315 (2018).

- MolProbity: More and better reference data for improved all-atom structure validation. C.J. Williams, B.J. Hintze, J.J. Headd, N.W. Moriarty, V.B. Chen, S. Jain, S.M. Prisant MG Lewis, L.L. Videau, D.A. Keedy, L.N. Deis, I.I.I. Arendall WB, V. Verma, J.S. Snoeyink, P.D. Adams, S.C. Lovell, J.S. Richardson, and D.C. Richardson. *Protein Science* 27, 293-315 (2018).

Tutorial: Automatically rebuilding a model solved by molecular replacement

rebuild.html

Introduction

This tutorial will start with a molecular replacement model for a2u-globulin, and carry out the process of rebuilding this model using the rebuild-in-place option of the AutoBuild Wizard. The tutorial is designed to be read all the way through, giving pointers for you along the way. Once you have read it all and run the example data and looked at the output files, you will be in a good position to run your own data through AutoBuild.

Setting up to run PHENIX

If PHENIX is already installed and your environment is all set, then if you type:

```
echo $PHENIX
```

then you should get back something like this:

```
/xtal/phenix-1.3
```

If instead you get:

```
PHENIX: undefined variable
```

then you need to set up your PHENIX environment. See the PHENIX installation page for details of how to do this. If you are using the C-shell environment (csh) then all you will need to do is add one line to your .cshrc (or equivalent) file that looks like this:

```
source /xtal/phenix-1.3/phenix_env
```

(except that the path in this statement will be where your PHENIX is installed). Then the next time you log in \$PHENIX will be defined.

Running the demo a2u-globulin-rebuild data with AutoBuild

To run AutoBuild on the demo a2u-globulin-rebuild data, make yourself a tutorials directory and cd into that directory:

```
mkdir tutorials
cd tutorials
```

Now type the phenix command:

```
phenix.run_example --help
```

to list the available examples. Choosing a2u-globulin-rebuild for this tutorial, you can now use the phenix command:

```
phenix.run_example a2u-globulin-rebuild
```

to build the a2u-globulin-rebuild structure with AutoBuild. This command will copy the directory \$PHENIX/examples/a2u-globulin-rebuild to your current directory (tutorials) and call it tutorials/a2u-globulin-rebuild/. Then it will run AutoBuild using the command file run.sh that is present in this tutorials/a2u-globulin-rebuild/ directory.

This command file run.sh is simple. It says:

```
#!/bin/sh
echo "Running AutoBuild on a2u-globulin MR data "
phenix.autobuild mup_mr_solution.pdb a2u-sigmaa.mtz a2u-globulin.seq \
input_map_file=a2u-sigmaa.mtz input_map_labels="FWT PHIC"
```

The first line (#!/bin/sh) tells the system to interpret the remainder of the text in the file using the sh (or bash) -shell (sh).

The command phenix.autobuild runs the command-line version of AutoBuild (see Automated Structure Solution using AutoBuild for all the details about AutoBuild including a full list of keywords).

The arguments on the command line tell AutoBuild about the molecular replacement model (mup_mr_solution.pdb the data file (data=a2u-sigmaa.mtz), sequence file (seq_file=a2u-globulin.seq), the input map file with sigmaA-weighted phases (same as the input data file, but using different data columns: input_map_file=a2u-sigmaa.mtz), and the data columns for the map calculation: input_map_labels="FWT PHIC"). (Note that each of these is specified with an = sign, and that there are no spaces around the = sign.)

Note the backslash "\" at the end of some of the lines in the phenix.autobuild command. This tells the C-shell (which interprets everything in this file) that the next line is a continuation of the current line. There must be no characters (not even a space) after the backslash for this to work.

The structure factor amplitudes and experimental phase information are in the datafile a2u-sigmaa.mtz. This is an mtz file which is a binary file that contains summary information about the dataset as well as the reflection data.

Although the phenix.run_example a2u-globulin-rebuild command has just run AutoBuild from a script (run.sh), you can run AutoBuild yourself from the command line with the same phenix.autobuild ... command. You can also run AutoBuild from a GUI, or by putting commands in another type of script file. All these possibilities are described in Using the PHENIX Wizards.

Where are my files?

Once you have started AutoBuild or another Wizard, an output directory will be created in your current (working) directory. The first time you run AutoBuild in this directory, this output directory will be called AutoBuild_run_1_ (or AutoBuild_run_1_/, where the slash at the end just indicates that this is a directory). All of the output from run 1 of AutoBuild will be in this directory. If you run AutoBuild again, a new subdirectory called AutoBuild_run_2_ will be created.

Inside the directory AutoBuild_run_1_ there will be one or more temporary directories such as TEMPO created while the Wizard is running. The files in this temporary directory may be useful sometimes in figuring out what the Wizard is doing (or not doing!). By default these directories are emptied when the Wizard finishes (but you can keep their contents with the command clean_up=False if you want.)

What parameters did I use?

Once the AutoBuild wizard has started (when run from the command line), a parameters file called autobuild.eff will be created in your output directory (e.g., AutoBuild_run_1_/autobuild.eff). This parameters file has a header that says what command you used to run AutoBuild, and it contains all the starting values of all parameters for this run (including the defaults for all the parameters that you did not set).

The autobuild.eff file is good for more than just looking at the values of parameters, though. If you copy this file to a new one (for example autobuild_lores.eff) and edit it to change the values of some of the parameters (resolution=3.0) then you can re-run AutoBuild with the new values of your parameters like this:

```
phenix.autobuild autobuild_lores.eff
```

This command will do everything just the same as in your first run but use only the data to 3.0 Å.

Reading the log files for your AutoBuild run file

While the AutoBuild wizard is running, there are several places you can look to see what is going on. The most important one is the overall log file for the AutoBuild run. This log file is located in:

```
AutoBuild_run_1_/AutoBuild_run_1_1.log
```

for run 1 of AutoBuild. (The second 1 in this log file name will be incremented if you stop this run in the middle and restart it with a command like `phenix.autobuild run=1`).

The `AutoBuild_run_1_1.log` file is a running summary of what the AutoBuild Wizard is doing. Here are a few of the key sections of the log files produced for the a2u-globulin-rebuild SAD dataset.

Summary of the command-line arguments

Near the top of the log file you will find:

```
Starting AutoBuild with the command:
```

```
phenix.autobuild input_pdb_file=mup_mr_solution.pdb data=a2u-sigmaa.mtz \  
seq_file=a2u-globulin.seq input_map_file=a2u-sigmaa.mtz \  
input_map_labels='FWT PHIC'
```

```
Guessing mup_mr_solution.pdb is a starting model.  
Guessing a2u-sigmaa.mtz is a datafile.  
Guessing a2u-globulin.seq is sequence file.
```

The first couple lines are just a repeat of how you ran AutoBuild; you can copy it and paste them into the command line to repeat this run.

The last 3 lines point out that you didn't specify exactly what these three files are...and is confirming what the Wizard is going to use them for.

Guessing the chain type

The AutoBuild Wizard will read in your sequence file and guess whether this is PROTEIN, DNA, or RNA from the sequence:

```
Guessing chain type from a2u-globulin.seq  
Setting chain type to PROTEIN
```

If you want to tell the Wizard what the chain type is, you can say, `chain_type=PROTEIN`.

Deciding on rebuild-in-place and editing input PDB file

The AutoBuild Wizard has two main options for rebuilding a model: rebuild-in-place and not rebuilding in place. Rebuilding in place is a method for rebuilding a model without changing the overall positioning of residues or the alignment of the sequence in the model. It is carried out by sequentially removing a short segment and rebuilding just that segment, and keeping the sequence of the segment the same. The alternative method for model rebuilding is to start from scratch and build a model directly into the electron density map. (Optionally you can also re-use fragments of your original model.).

The AutoBuild Wizard by default will decide whether to use rebuild-in-place based on whether an alignment of the sequence you input with the model you input can be made. If the sequence alignment yields at least 50% identical residues, then rebuild-in-place will be recommended. (You can choose not to use rebuild in place

with the keyword `rebuild_in_place=No`).

Rebuilding in place is generally a good idea if you just want to fix up a model that is almost correct; it is not a good way to make a lot of big changes in a model. It will not fill in loops or otherwise correct connectivity.

In this example the sequence identity is high and rebuilding in place is recommended. The Wizard then edits your PDB file to give it the sequence you have provided (if you provide no sequence then the sequence in the PDB file will be kept), deleting any segments that do not match your sequence file at all, and inserting gaps where there are missing residues:

```
Deciding if we want to use rebuild-in-place...

Rebuild in place recommended as identity * fraction_aligned is 63.06%

-----
Percent identity of mup_mr_solution.pdb to sequence in a2u-globulin.seq is 63.06
Finished with creating edited pdb file: AutoBuild_run_1_/edited_pdb.pdb
```

Guessing column labels

The AutoBuild Wizard will need to know which columns in your input data file and your input map file to use. It guesses which column labels to use and lists them out:

```
Getting column labels from a2u-sigmaa.mtz for input data file
Resolution from datafile: 2.38339035134
SG: P 21 21 21
Cell: [106.81999969482422, 62.340000152587891, 114.19000244140625, 90.0, 90.0, 90.0]
Input labels: ['FP', 'SIGFP', 'None', 'FOM', 'None', 'None', 'None', 'None', 'None']

Getting column labels from a2u-sigmaa.mtz for input map file
SG: P 21 21 21
Cell: [106.81999969482422, 62.340000152587891, 114.19000244140625, 90.0, 90.0, 90.0]
Map input labels: ['FWT', 'PHIC']
```

These are indeed the appropriate columns to use for experimental phases and for map coefficients, respectively. Note the "None" in the input labels for a2u-sigmaa.mtz. The input labels in this list correspond to FP SIGFP PHIB FOM HLA HLB HLC HLC FreeR_flag and there is no Free R data in the input data file. All the data that is expected for each input file in AutoBuild can be seen in the AutoBuild web page under "Specifying which columns of data to use from input data files".

Guessing cell contents

The AutoBuild Wizard uses the sequence information in your sequence file (sequence.dat) and the cell parameters and space group to guess the number of NCS copies and the solvent fraction:

```
Number of residues in unique chains in seq file: 181
Unit cell: (106.82, 62.34, 114.19, 90, 90, 90)
Space group: P 21 21 21 (No. 19)
CELL VOLUME :760409.359319
N_EQUIV:4
GUESS OF NCS COPIES: 4
SOLVENT FRACTION ESTIMATE: 0.48
Data file (for everything including refinement): a2u-sigmaa.mtz
```

Running phenix.xtriage

The AutoBuild Wizard automatically runs phenix.xtriage on your input datafile to analyze it for twinning, outliers, translational symmetry, and other special conditions that you should be aware of. You can read more about xtriage in Data quality assessment with phenix.xtriage. The xtriage output is in the file

a2u-sigmaa.mtz_xtriage.log. Part of the summary output from xtriage for this dataset looks like this:

The largest off-origin peak in the Patterson function is 5.10% of the height of the origin peak. No significant pseudotranslation is detected.

The results of the L-test indicate that the intensity statistics behave as expected. No twinning is suspected.

Generation of FreeR flags

The AutoBuild Wizard will create a set of free R flags indicating which reflections are not to be used in refinement. By default 5% of reflections (up to a maximum of 2000) are reserved for this test set. If you supply a reflection file with free R flags already set, then they will be used. If you want to supply a file ref.mtz specifically for refinement, you can do that with input_refinement_file=ref.mtz. Also if you want to supply a high-resolution datafile hires.mtz that has then you can do this with the keywords input_hires_file=hires.mtz. After generation of free R flags if necessary, and any merging of data files, the file to be used for refinement is called exptl_fobs_phases_freeR_flags.mtz.

Rebuild-in-place model-building with RESOLVE by building multiple models and combining the best parts

The AutoBuild Wizard by default uses RESOLVE to build an atomic model of your structure. In this example the rebuild-in-place option is used. To rebuild a model in place means to rebuild the model without adding or removing any residues or changing the connectivity of the chain. The AutoBuild Wizard uses the multiple-models algorithm to rebuild your model several times and then to recombine them together to form a single very good model. (The multiple-models algorithm can also be used to form several very good models if you want).

Here is where this is done:

```
Setting up to build 5 models to be combined into final model #1
```

```
Setting background=False as nproc=1
```

```
Building 5 models ...Splitting work into 5 jobs and running 1 at a time with sh in  
/net/idle/scratch1/terwill/run_072908a/a2u-globulin-rebuild/AutoBuild_run_1/TEMP0
```

```
Starting job 1...
```

```
Starting job 2...
```

```
Starting job 3...
```

```
Starting job 4...
```

```
Starting job 5...
```

```
Collecting multiple_model runs to form a single final model
```

```
Solution for try : 1 cycle_best_3.pdb
```

```
Solution 2 from rebuild cycle 3 R= 0.2
```

```
Saving /net/idle/scratch1/terwill/run_072908a/a2u-globulin-rebuild/  
AutoBuild_run_1/TEMP0/AutoBuild_run_1/cycle_best_3.pdb as  
/net/idle/scratch1/terwill/run_072908a/a2u-globulin-rebuild/  
AutoBuild_run_1/MULTIPLE_MODELS/initial_model.pdb_1_1
```

```
Saving /net/idle/scratch1/terwill/run_072908a/a2u-globulin-rebuild/  
AutoBuild_run_1/TEMP0/AutoBuild_run_1/cycle_best_3.mtz as  
/net/idle/scratch1/terwill/run_072908a/a2u-globulin-rebuild/  
AutoBuild_run_1/MULTIPLE_MODELS/initial_model.mtz_1_1  
...
```

```
Solution 4 from rebuild cycle 5 R= 0.21
```

```
Solution 2 from rebuild cycle 3 R= 0.21
```

```
Solution 3 from rebuild cycle 4 R= 0.21
```

```
Solution 3 from rebuild cycle 4 R= 0.21
```

In this case each of the individually rebuilt models is pretty good, with an R from 0.20 to 0.21. These models are then combined together, with the best parts of each model (based on correlation to the density-modified map) kept:

```
AutoBuild_combine AutoBuild Run 2 Mon May 19 12:06:11 2008

Current combine number: 1
Combining initial models # 1
...
NOTE: only keeping merged models if they improve R

Merging model 1
Removing waters and ligands (if any): AutoBuild_run_1_/TEMP0/current_model.pdb
R/Rfree for model composite_model_refined_1.pdb 0.21 0.23

...

Merging model 5
combining current_model.pdb initial_model.pdb_1_5
Removing waters and ligands (if any): AutoBuild_run_1_/TEMP0/current_model.pdb
Removing waters and ligands (if any): AutoBuild_run_1_/TEMP0/initial_model.pdb_1_5
R/Rfree for model composite_model_refined_5.pdb 0.2 0.23

...

Merging model 2
combining current_model.pdb initial_model.pdb_1_2
Removing waters and ligands (if any): AutoBuild_run_1_/TEMP0/current_model.pdb
Removing waters and ligands (if any): AutoBuild_run_1_/TEMP0/initial_model.pdb_1_2
R/Rfree for model composite_model_refined_2.pdb 0.21 0.23
Not keeping this merged model as it is no better than current best

...

New overall best saved:

Current overall_best model and map Mon Jun 30 01:08:22 2008
Working directory:
/net/idle/scratch1/terwill/run_072908a/a2u-globulin-rebuild/AutoBuild_run_1_
Model (overall_best.pdb) from: overall_best.pdb
Cycle: 1
R and R-free: 0.20 0.23
Map coeffs used for build (overall_best_denmod_map_coeffs.mtz)
from: overall_best_denmod_map_coeffs.mtz

Final composite model # 1 : AutoBuild_run_1_/MULTIPLE_MODELS/model.pdb_1
Final composite mtz # 1 : AutoBuild_run_1_/MULTIPLE_MODELS/resolve_composite_map.mtz_1
```

Here there were four models built, with R values from 0.2 to 0.22. The one with the lowest R (model 4) was used as the starting model, and each of the others was merged with this model, taking the best-fitting parts. An improved model was obtained by merging model 3 with model 4, yielding an R just under 0.20.

Individual rebuild-in-place log files

If you want to see the details of the individual runs of rebuild_in_place on this model, you will want to look in the directory that was listed above:

```
AutoBuild_run_1_/TEMP0
```

In this TEMP0 directory you will find subdirectories like:

```
AutoBuild_run_1_/
AutoBuild_run_2_/
```


which in turn contain the log files for the runs of these individual rebuild_in_place rebuilding steps. Here is a summary of the run in

```
AutoBuild_run_1_/TEMP0/AutoBuild_run_1_/AutoBuild_run_1_1.log
```

After reading in the data in this sub-process, the starting model is refined:

```
Refining model: unrefined.pdb_1
Model: refine.pdb_1 R/Rfree=0.31/0.32
```

This starting model is rather good, with an R of 0.32. This is about the quality of model that works best with rebuild-in-place. The NCS in this model is analyzed automatically:

```
GROUP 1
Summary of NCS group with 4 operators:
ID of chain/residue where these apply: [['A', 'B', 'C', 'D'],
[[[1, 157]], [[1, 157]], [[1, 157]], [[1, 157]]]]
RMSD (A) from chain A: 0.0 0.0 0.0 0.0
Number of residues matching chain A:[157, 157, 157, 157]
Source of NCS info: AutoBuild_run_1_/TEMP0/overall_best.pdb
Correlation of NCS: 0.76
```

There is good correlation of NCS-related density (as expected as all the density is model-based so far) and the 4 chains have a 0.0 RMSD after application of NCS (also expected as we have simply placed the same molecule in 4 different orientations and locations in the asymmetric unit).

The rebuild-in place procedure is then carried out:

```
Running standard build
Starting with current best model of refine.pdb_1 ...setting it to refine.pdb_1
Including parts of this model if best...
Starting best model from starting_model.pdb
Rebuilding in place: starting_model.pdb
New total residues built: 562

Warning: some residues not rebuilt successfully on cycle 2:
Chain 'A': 1- 10, 156- 157,
Chain 'C': 1- 10, 61- 65, 151- 157,
Chain 'B': 1- 5, 61- 65,
Chain 'D': 1- 10, 61- 65, 151- 157
Refining model: Build_rebuild_in_place_1.pdb
Model: AutoBuild_run_1_/TEMP0/refine_1.pdb R/Rfree=0.23/0.26
```

Notice the warning about some of the residues that were not rebuilt. For any residues that the rebuilding process simply could not fit, the original coordinates are kept. In favorable cases, by the end of iterative model-building all the coordinates can be fit. In cases where they cannot, you should be cautious about using those parts of the model that do not get rebuilt. In this case the R has decreased substantially by rebuilding to 0.23.

The side chains are then rebuilt, and the best-fitting parts of the current models are recombined to yield a new model with a similar R of 0.24:

```
Rebuilding side chains from Build_composite_refined_3.pdb
Refining model: edit_model.pdb
Model: AutoBuild_run_1_/TEMP0/Build_composite_refined_side_3.pdb R/Rfree=0.24/0.26
```

This is the end of the first cycle of rebuilding. Next the rebuilt model is used to calculate phases and the resulting map is used in statistical density modification, including not just solvent flattening and histograms of density, but also the 4-fold NCS in this structure. This whole process is repeated 2 more times, yielding the best model on the second cycle, with a final R/Rfree of 0.20/0.23. On the final cycle of model-building a few residues are still not rebuilt:

```
Warning: some residues not rebuilt successfully on cycle 3:
```

```
Chain 'A': 156- 157,  
Chain 'C': 1- 10, 61- 65, 156- 157,  
Chain 'B': 1- 10,  
Chain 'D': 1- 10, 61- 65, 156- 157
```

and these residues (the N-terminal 10 residues of chains B, C, and D) and residues 61-65 and the C-terminal 2 residues of chain C) should probably not be used in the model.

The final refined model is written out to AutoBuild_run_1_/TEMP0/AutoBuild_run_1_/cycle_best_3.pdb. Note that this run is carried out entirely within the TEMP0/ subdirectory because this is a sub-process. This model is then combined with the models from the other sub-processes in this multiple-model run to form a final model.

Note that the quality of the model improves a lot in the first cycle and not very much after that. This is common in the rebuild-in-place procedure.

The AutoBuild_summary.dat summary file

A quick summary of the results of your AutoBuild run is in the AutoBuild_summary.dat file in your output directory. This file lists the key files that were produced in your run of AutoBuild (all these are in the output directory) and some of the key statistics for the run. Here is the summary for this a2u-globulin-rebuild model-building run:

```
Summary of model-building for run 1 Mon Jun 30 01:08:54 2008  
Files are in the directory:  
/net/idle/scratch1/terwill/run_072908a/a2u-globulin-rebuild/AutoBuild_run_1_  
  
Starting mtz file:  
/net/idle/scratch1/terwill/run_072908a/a2u-globulin-rebuild/a2u-sigmaa.mtz  
Sequence file:  
/net/idle/scratch1/terwill/run_072908a/a2u-globulin-rebuild/a2u-globulin.seq  
One composite model generated with 'multiple_models'  
  
Best solution on cycle: 1 R/Rfree=0.2/0.23  
  
Summary of output files for Solution 1 from multiple_models cycle 1  
  
--- Model (PDB file) ---  
pdb_file: AutoBuild_run_1_/cycle_best_1.pdb  
  
--- Model-map correlation log file ---  
log_eval: AutoBuild_run_1_/cycle_best_1.log_eval  
  
--- Data for refinement FP SIGFP PHIM FOMM HLAM HLBM HLCM HLDM FreeR_flag ---  
hklout_ref: AutoBuild_run_1_/exptl_fobs_phases_freeR_flags.mtz  
  
--- Density-modified map coefficients FP PHIM FOM ---  
hklout_denmod: AutoBuild_run_1_/cycle_best_1.mtz  
  
SOLUTION CYCLE R RFREE BUILT PLACED  
1 1 0.20 0.23 0 0
```

Your best model is in AutoBuild_run_3_/cycle_best_1.pdb, and it has an R/Rfree of 0.20/0.23, which is quite reasonable. The best density-modified map coefficients are in AutoBuild_run_3_/cycle_best_1.mtz. The values of "BUILT" and "PLACED" are zero because no new model is being built in this process (just rebuilding in place). There is a residue-by-residue analysis of the correlation between density calculated from this model and this map in AutoBuild_run_3_/cycle_best_1.log_eval. If you want to refine your model further, you should use the data in AutoBuild_run_3_/exptl_fobs_phases_freeR_flags.mtz, which includes a set of freeR flags.

How do I know if iterative model-building, density modification and refinement with rebuild-in-place worked?

Here are some of the things to look for to tell if you have obtained a good model:

- How much of the model failed to be rebuilt with the rebuild-in-place procedure? This should be a very small part of the model at most. If 25% of your model cannot be rebuilt, then the density is probably very weak (or the resolution may be too low).
- What is the R-factor and the free R-factor of the model? For a solution at moderate to high resolution (2.5 Å or better) the R-factor should be in the low 20's and the free R-factor about 0.05 higher to be very good. For lower-resolution data, an R-factor in the low 30's is probably largely correct but the model is not very good.
- Does the model show NCS if there are more than one copies in the asymmetric unit? You can look at your log file which will tell you about any NCS in your model.

What to do next

Once you have run AutoBuild and have obtained a good model, you will want to inspect and touch up the model carefully, rebuilding any parts of the model that do not agree well with the final map. You should also have a close look at all the solvent molecules in your model, making sure that they all have reasonable relationships to the macromolecule and to each other, and that they are not simply filling up density where a ligand or the macromolecule really goes.

The next thing to do is to add in any ligands (metals, cofactors) if there is density for them. You can use the LigandFit Wizard (see Automated Ligand Fitting using LigandFit) to help you fit ligands into your map automatically.

If you do not obtain a good model, then it's not quite time to give up yet. There are a number of standard things to try that may improve the model building. Here are a few that you should try:

- Have a careful look at all the output files. Work your way through the main log file (e.g., AutoBuild_run_1_1.log). Is there anything strange or unusual in any of them that may give you a clue as to what to try next? What does the xtriage output say? Is there twinning or strong translational symmetry? Are there problems with reflections near ice rings? Are there many outlier reflections?
- Have a look at the electron density maps at different stages in model building. How does the starting map look? How about the maps produced during the iterative building process (cycle_best_xx.mtz)? If the maps look poor, then the rebuilding process will not work well. However if the maps look good and the rebuilding is not working well then it is worth trying to identify where the problem is.
- Try a different resolution cutoff. You may be able to use higher-resolution data in AutoBuild than you used in structure solution.

Additional information

For details about the AutoBuild Wizard, see Automated Model building and Rebuilding using AutoBuild. For help on running Wizards, see Using the PHENIX Wizards.

Tutorial: RNase-S Refinement

refine.html

Overview

This tutorial will show you how to refine a simple structure using the PHENIX graphical user interface (GUI). Please read the `phenix.refine` documentation before running this tutorial, as it will familiarize you with the GUI and the options available for refinement. You may also find the dictionary of terms helpful.

Setup

The example used for this tutorial is called `rnase-s`, and can be set up in the GUI as described here. In the tutorial setup window, select the dataset labeled "S. aureofaciens ribonuclease Sa" under "Refinement", and click "Okay".

Once you have created the project, click the button labeled "phenix.refine" in the main GUI under "Refinement".

Input files

Only two input files are required for running `phenix.refine` with this dataset, the starting model and the reflections. These can be added either by dragging and dropping them into the input file list from the desktop, or by clicking "Add file" and selecting them from the dialog window.

The space group and unit cell will be loaded automatically; these will always be taken from the last symmetry-containing file to be added (in this case, the PDB file and reflections file both contain the symmetry information). The available data arrays for refinement will be listed in the "Data labels" drop-down menu; the first of these is what we want to use, and will be selected automatically. Because this structure predates the use of the R-free statistic in refinement, no R-free flags are available; these will be generated once refinement starts.

If you wish you may also input the file `1sar.fa`, which is the protein sequence; this is not necessary for refinement itself but will be used to validate the model sequence at the end of the run.

Refinement options

The second configuration tab, Refinement settings contains most of the essential options for running `phenix.refine`. The first section controls the model parameters to be refined. For the `rnase-s` example, the XYZ coordinates, Real-space, Individual B-factors, and TLS strategies are appropriate. The Occupancies strategy is also selected by default, but since the default behavior will not affect any of the atoms in the `rnase-s` input model, it can be left on without negatively impacting runtime or the result quality.

All strategies can be applied to specific atoms by defining atom selections. For most of the strategies the default of refining all atoms is appropriate, but for TLS a more fine-grained division of groups is appropriate. Select "TLS parameters" from the drop-down menu labeled "Modify selections for". A new window will appear for defining TLS groups (the same window can also be opened by clicking "TLS" on the toolbar). Since PHENIX includes a built-in program to identify suitable TLS groups based on the input B-factors, we will use this. Click "Find TLS groups" on the toolbar to start this program; it will typically take 15 seconds to several minutes depending on the size of the structure and the number of processors used (on Windows this is limited to one).

When the program is finished, atom selections for the TLS groups will be loaded into the list. Select one of these and click the button labeled "View/pick" to display the selection visually; you can modify the selection manually if you want, but in this case the automatic selections are suitable. You can now save the TLS selections by clicking "Update and exit" on the toolbar of the TLS editor window.

In the "Restraints and weighting" section, you should activate the NCS restraints option, as the RNase-S structure contains two copies of the protein chain. For "NCS type", select "torsion-angle"; this will restrain the dihedral angles to be similar, instead of the older global-superposition based method, which does not allow for the small local deformations between chains which are present in this model.

For this particular structure, optimizing the X-ray/stereochemistry weight will be helpful, producing a model with a lower R-free and a smaller gap between R-work and R-free. However, since this takes significantly longer to finish, the optimization may be left off for the purposes of running the tutorial. The remainder of the options can also be left in the default state, although we recommend experimenting with Automatically add hydrogens to model, Update waters, Simulated annealing, and Fix bad sidechain rotamers to get a feel for how these options affect performance.

At this point the Refinement settings tab should look approximately like the image below:

The final tab controls output settings; these can also be left alone, but you may wish to set the run title, as this will appear in the job history in the main GUI. The output map coefficients will include a 2mFo-DFc and mFo-DFc map.

Running the program

Click the "Run" button on the toolbar to start the program. Depending on the operating system and local setup, you may have multiple options to run the job: by default it will be start in the same process, which will allow faster communication with the GUI, but this also means that closing the GUI will kill the job. On Mac and Linux you will also have the option to run the job as "detached", which allows you to close the GUI, but lacks some features. Queuing systems are also supported on Linux. You can also view or edit the complete parameters file before running if desired. In this case, you should choose the first item in the menu ("Run now").

Because no R-free flags were present in the input file, a dialog will appear asking if you want to create new flags, with instructions for how to use these flags after refinement:

You should click "Okay" to proceed; if you want to adjust the parameters used for creating the flags, click "Cancel" and then edit settings in the dialog opened by clicking "Options" under "X-ray data" in the first configuration tab, then re-run. Immediately after the job starts, another window will appear warning you that new flags have been created:

This is not an error message, but prevents against unknowingly generating new R-free flags by mistake.

When the process starts, a new tab will appear, which contains two inner tabs. The second tab will be visible at first, and displays the basic refinement statistics (including R-factors, bond and angle deviations, and validation scores), both for the current model and plotted over the course of refinement:

The appearance of the R-factor plot is an important indicator of whether the selected refinement options are appropriate or not. In this case, since R-free flags were not previously used in refinement, the R-work and R-free will start out the same and immediately diverge. This is expected; however, the gap between the two values should stay relatively low, and R-free should still decrease over the course of refinement. A large increase in the gap indicates overfitting, and additional restraints and/or a more conservative optimization strategy may be required. If R-free increases during refinement, the changes in the model are not justified and

the result should be discarded.

The first tab shows the complete log output:

You can also view the log file itself by clicking "View log". Experienced users may not find it necessary to inspect the log, but we recommend that you familiarize yourself with its contents, as it contains additional information that may be useful for debugging, as well as providing an overview of the program logic and workflow.

While refinement is running, if Coot is available on your system, it will be started and the current model and maps will be automatically loaded each time they change. This behavior can be disabled in the preferences (under "Graphics", but Coot can also be started during refinement by clicking "Show graphics" in the "Run status" tab. PyMOL is also supported. During refinement, the current model will be colored in cyan; at the end of refinement, the final model will be colored yellow.

After refinement

Once the job completes, additional tabs will appear reporting the output files, final statistics and a variety of validation criteria:

The output files will mostly take the form `${PREFIX}_${RUN_ID}.${EXTENSION}`, for instance "rnase-s_refine_19.mtz" in the image above; the prefix can be modified in the Output configuration tab. The run ID will be set automatically by the GUI. Files named using this convention include:

.log: the same log output that appears in the GUI. .eff: the final effective parameters, including any automatic modifications performed internally. .geo: a complete listing of geometry restraints at the start of refinement, including both target and actual values, sorted by deviation from ideal. .pdb: the final refined model. .mtz: all reflection data associated with the run, including the R-free flags, the input data, the filtered data used in refinement (excluding outliers), the final "F(model)" (the calculated structure factors, including bulk solvent contribution), and map coefficients (including, by default, the 2mFo-DFc map with and without missing F-obs substituted by F-calc, the mFo-DFc map, and if anomalous data were used in refinement, the anomalous difference map).

- .log: the same log output that appears in the GUI.
- .eff: the final effective parameters, including any automatic modifications performed internally.
- .geo: a complete listing of geometry restraints at the start of refinement, including both target and actual values, sorted by deviation from ideal.
- .pdb: the final refined model.
- .mtz: all reflection data associated with the run, including the R-free flags, the input data, the filtered data used in refinement (excluding outliers), the final "F(model)" (the calculated structure factors, including bulk solvent contribution), and map coefficients (including, by default, the 2mFo-DFc map with and without missing F-obs substituted by F-calc, the mFo-DFc map, and if anomalous data were used in refinement, the anomalous difference map).

Additionally, if modifications were made to the input data, including the generation of R-free flags, a file named `${PREFIX}_data.mtz` will be included. If you created a new set of flags, as was done for this tutorial, you should always use this MTZ file for future refinements.

Guidelines for interpreting the validation results can be found in the documentation for the validation GUI or the external Molprobtutorial from the Cold Spring Harbor Laboratory crystallography course.

Command-line usage

This tutorial can also be run from the command line, although the final validation is only available from the GUI. The example files can be found here:

```
$PHENIX/examples/rnase-s
```

The equivalent command line arguments for the run described above are:

```
phenix.refine rnase-s.pdb rnase25.mtz xray_data.labels=FNAT,SIGFNAT,merged \
xray_data.r_free_flags.generate=True ncs=True ncs.type=torsion \
refine.strategy=individual_sites+individual_sites_real_space+individual_adp+tls+occupancies \
tls.find_automatically=True
```

Note that in the GUI, the identification of TLS groups was run interactively as a separate step prior to starting the job, rather than automatically during refinement. On the command line, you can run this command to obtain the TLS parameters:

```
phenix.find_tls rnase-s.pdb
```

References

Towards automated crystallographic structure refinement with phenix.refine. P.V. Afonine, R.W. Grosse-Kunstleve, N. Echols, J.J. Headd, N.W. Moriarty, M. Mustyakimov, T.C. Terwilliger, A. Urzhumtsev, P.H. Zwart, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 68, 352-67 (2012).

- Towards automated crystallographic structure refinement with phenix.refine. P.V. Afonine, R.W. Grosse-Kunstleve, N. Echols, J.J. Headd, N.W. Moriarty, M. Mustyakimov, T.C. Terwilliger, A. Urzhumtsev, P.H. Zwart, and P.D. Adams. *Acta Crystallogr D Biol Crystallogr* 68, 352-67 (2012).

Sevcik J, Dodson EJ, Dodson GG. (1991) Determination and restrained least-squares refinement of the structures of ribonuclease Sa and its complex with 3'-guanylic acid at 1.8 Å resolution. *Acta Crystallogr., Sect. B* 47:240-253. PMID: 1654932

Tutorial: Structure refinement

refinement_examples.html

A set of structure refinement examples using phenix.refine.

Authors

- Examples compilation: Pavel Afonine (PAfonine@lbl.gov), Ralf Grosse-Kunstleve
- Atomic models and data: CCI Structure Library (Paul Adams, PAdams@lbl.gov), PDB

Quick Facts

- All necessary files to run structure refinement examples with phenix.refine are located in refinement_examples directory;
- Each sub-directory refinement_examples/example_xx contains all task files (command and parameter files) necessary to run an example;
- Data and model files are located in refinement_examples/data sub-directory;
- Some examples are quick to run and some could take an hour or more (depending on computer);
- The actual R-factors resulted from running these examples may slightly vary from listed below due to potential differences in hardware and software (different versions of compilers or PHENIX).

Run examples

There are two different ways to run the examples:

- Type the command: `% phenix.run_refinement_example 5a` this will create the directory example_5a and run the example 5a in this directory. Running the command above without arguments will run all examples (will take a long time). One can also run several examples like this: `% phenix.run_refinement_example 1a 2b 3a 3c` this will create 4 directories: example_1a, ..., example_3c and run the examples 1a, 2b, 3a and 3c in these directories.

Type the command:

```
% phenix.run_refinement_example 5a
```

this will create the directory example_5a and run the example 5a in this directory. Running the command above without arguments will run all examples (will take a long time). One can also run several examples like this:

```
% phenix.run_refinement_example 1a 2b 3a 3c
```

this will create 4 directories: example_1a, ..., example_3c and run the examples 1a, 2b, 3a and 3c in these directories.

- From PHENIX distribution copy the entire refinement_examples directory to where you plan to run the examples. Then go to any of sub-directories refinement_examples/example_xx and execute a run file. This will start the selected refinement example, wait until it's finished and inspect the output files.

From PHENIX distribution copy the entire refinement_examples directory to where you plan to run the examples. Then go to any of sub-directories refinement_examples/example_xx and execute a run file. This will start the selected refinement example, wait until it's finished and inspect the output files.

Examples

Refinement of 1akg structure at 1.1 Å resolution

In this series of examples we review the simple refinement with all default parameters, automatic picking and refinement of solvent molecules, switching from isotropic to anisotropic ADP refinement, practice monitoring of Rfactors change during refinement and experience the effect of adding and refinement of hydrogen atoms.

example_1a: The simplest possible refinement mode: refinement with all default parameters. This includes 3 macro-cycles of bulk-solvent and anisotropic scaling, individual coordinates and isotropic B-factors

refinement, refinement of occupancies for atoms in alternative conformations. Final: R-work = 0.2264 R-free = 0.2423 The start model does not contain any water molecules placed into it. Next example demonstrates how

to add and refine ordered solvent. example_1b: Same refinement run as in example_1a with automatic water picking and refinement. Final: R-work = 0.1771 R-free = 0.1855 Automatic water picking and refinement

dropped the R-factors by ~5%. At 1.1 Å resolution all well ordered non-hydrogen atoms should be refined as anisotropic. example_1c: Same refinement run as in example_1b with all non-water atoms refined

anisotropically. Final: R-work = 0.1534 R-free = 0.1849 Refinement of non-solvent atoms as anisotropic dropped both Rwork and Rfree (as expected at this resolution). example_1d: Same refinement run as in

example_1c performed with hydrogen atoms added to the start model. Final: R-work = 0.1426 R-free = 0.1703 Using the hydrogens in refinement may further improve the model. Indeed, adding the hydrogen atoms and

refining them as riding model decreased R-factors by approximately 1%. example_1e: Same refinement run as in example_1d performed with optimized X-ray target weights for both coordinate and ADP refinement.

Final: R-work = 0.1340 R-free = 0.1658 Although phenix.refine uses automatic procedure for relative xray/geometry weight calculation and finds good weights in most of cases, it may be a good idea to try to optimize the weights. A drop in R-factors by ~1% in this example clearly demonstrates this. Since it is very slow, this option is recommended to run overnight for a final model tune up.

In this series of examples we review the simple refinement with all default parameters, automatic picking and refinement of solvent molecules, switching from isotropic to anisotropic ADP refinement, practice monitoring of Rfactors change during refinement and experience the effect of adding and refinement of hydrogen atoms.

example_1a: The simplest possible refinement mode: refinement with all default parameters. This includes 3 macro-cycles of bulk-solvent and anisotropic scaling, individual coordinates and isotropic B-factors refinement, refinement of occupancies for atoms in alternative conformations.

Final: R-work = 0.2264 R-free = 0.2423

The start model does not contain any water molecules placed into it. Next example demonstrates how to add and refine ordered solvent.

example_1b: Same refinement run as in example_1a with automatic water picking and refinement.

Final: R-work = 0.1771 R-free = 0.1855

Automatic water picking and refinement dropped the R-factors by ~5%. At 1.1 Å resolution all well ordered non-hydrogen atoms should be refined as anisotropic.

example_1c: Same refinement run as in example_1b with all non-water atoms refined anisotropically.

Final: R-work = 0.1534 R-free = 0.1849

Refinement of non-solvent atoms as anisotropic dropped both Rwork and Rfree (as expected at this resolution).

example_1d: Same refinement run as in example_1c performed with hydrogen atoms added to the start model.

Final: R-work = 0.1426 R-free = 0.1703

Using the hydrogens in refinement may further improve the model. Indeed, adding the hydrogen atoms and refining them as riding model decreased R-factors by approximately 1%.

example_1e: Same refinement run as in example_1d performed with optimized X-ray target weights for both coordinate and ADP refinement.

Final: R-work = 0.1340 R-free = 0.1658

Although phenix.refine uses automatic procedure for relative xray/geometry weight calculation and finds good weights in most of cases, it may be a good idea to try to optimize the weights. A drop in R-factors by ~1% in this example clearly demonstrates this. Since it is very slow, this option is recommended to run overnight for a final model tune up.

Refinement of 1071B at 3.0Å resolution, 2 NCS groups with 3 components in each group.

The aim of these four examples below is to practice determining and specifying of NCS groups (using atom selection syntax). Also it underlines the importance of checking NCS groups that determined automatically by phenix.refine. The last example demonstrates the use of TLS. example_2a: Refinement with all default parameters. This includes 3 macro-cycles of bulk-solvent and anisotropic scaling, coordinates and individual isotropic B-factors refinement. Final: R-work = 0.2082 R-free = 0.2729 The number of parameters and relatively low resolution produce fairly large gap between R-work and R-free. Hopefully we can add some more "observations" by using NCS restraints. example_2b: Same refinement run as in example_2a with using the NCS restraints (NCS operators detected automatically). Final: R-work = 0.2404 R-free = 0.2839 phenix.refine can automatically determine NCS groups and set up the selections for NCS restraints. It does it by simple analysis of chains in input PDB. Although it does the good job in many cases, in this particular one the found selections are not optimal and their use in refinement is counterproductive: both R-work and R-free jumped up. Visual inspection of the model on graphics suggests more smart selections as demonstrated below. example_2c: Same refinement run as in example_2a with using the NCS restraints (NCS operators provided manually). Final: R-work = 0.2147 R-free = 0.2566 The correct selections for NCS groups and their use in refinement produce nice R-factors. example_2d: Same refinement run as in example_2c with using TLS to model ADP of selected domains. The use of TLS model for ADP of selected chains results in some further model improvement. Final: R-work = 0.2079 R-free = 0.2540

The aim of these four examples below is to practice determining and specifying of NCS groups (using atom selection syntax). Also it underlines the importance of checking NCS groups that determined automatically by phenix.refine. The last example demonstrates the use of TLS.

example_2a: Refinement with all default parameters. This includes 3 macro-cycles of bulk-solvent and anisotropic scaling, coordinates and individual isotropic B-factors refinement.

Final: R-work = 0.2082 R-free = 0.2729

The number of parameters and relatively low resolution produce fairly large gap between R-work and R-free. Hopefully we can add some more "observations" by using NCS restraints.

example_2b: Same refinement run as in example_2a with using the NCS restraints (NCS operators detected automatically).

Final: R-work = 0.2404 R-free = 0.2839

phenix.refine can automatically determine NCS groups and set up the selections for NCS restraints. It does it by simple analysis of chains in input PDB. Although it does the good job in many cases, in this particular one the found selections are not optimal and their use in refinement is counterproductive: both R-work and R-free

jumped up. Visual inspection of the model on graphics suggests more smart selections as demonstrated below.

example_2c: Same refinement run as in example_2a with using the NCS restraints (NCS operators provided manually).

Final: R-work = 0.2147 R-free = 0.2566

The correct selections for NCS groups and their use in refinement produce nice R-factors.

example_2d: Same refinement run as in example_2c with using TLS to model ADP of selected domains.

The use of TLS model for ADP of selected chains results in some further model improvement.

Final: R-work = 0.2079 R-free = 0.2540

Refinement of 1038B at 3.0Å resolution, 10 fold NCS.

These three examples below are similar to previous set of examples with two main differences: the automatic determination of NCS groups worked well and the use of TLS modeling resulted in more significant improvement. example_3a: Refinement with all default parameters. This includes 3 macro-cycles of bulk-solvent and anisotropic scaling, coordinates and individual isotropic B-factors refinement. Final: R-work = 0.2043 R-free = 0.2743 The gap between R-work and R-free clearly indicated the overfitting. The NCS restraints will help as shown in the next example. example_3b: Same refinement run as in example_3a with using the NCS restraints (NCS operators detected automatically). Final: R-work = 0.2207 R-free = 0.2527 Applying NCS restrains decreased R-free and narrowed the gap R-work - R-free. No manual work was necessary to specify NCS groups. example_3c: Same refinement run as in example_3b with using TLS to model ADP of selected domains. Final: r_work = 0.2008 r_free = 0.2410 Selecting TLS groups required a closer look at the model on graphics. This is rewarded by ~2% drop in R. Note that the refinement is not fully converged as indicated by continuous drop in R-factors between the last two macro-cycles (see PDB file header: in refinement statistics step-by-step). This suggests that adding a few more refinement macro-cycles may further improve the model.

These three examples below are similar to previous set of examples with two main differences: the automatic determination of NCS groups worked well and the use of TLS modeling resulted in more significant improvement.

example_3a: Refinement with all default parameters. This includes 3 macro-cycles of bulk-solvent and anisotropic scaling, coordinates and individual isotropic B-factors refinement.

Final: R-work = 0.2043 R-free = 0.2743

The gap between R-work and R-free clearly indicated the overfitting. The NCS restraints will help as shown in the next example.

example_3b: Same refinement run as in example_3a with using the NCS restraints (NCS operators detected automatically).

Final: R-work = 0.2207 R-free = 0.2527

Applying NCS restrains decreased R-free and narrowed the gap R-work - R-free. No manual work was necessary to specify NCS groups.

example_3c: Same refinement run as in example_3b with using TLS to model ADP of selected domains.

Final: r_work = 0.2008 r_free = 0.2410

Selecting TLS groups required a closer look at the model on graphics. This is rewarded by ~2% drop in R. Note that the refinement is not fully converged as indicated by continuous drop in R-factors between the last two macro-cycles (see PDB file header: in refinement statistics step-by-step). This suggests that adding a few more refinement macro-cycles may further improve the model.

Refinement using Simulated Annealing (SA)

In this couple of examples we demonstrate the power of SA refinement to improve a poor starting model. Combining it with automatic water picking is a good idea at the cost of just one parameter added to the command line. example_4a: Refinement with all default parameters. This includes 3 macro-cycles of bulk-solvent and anisotropic scaling, coordinates and individual isotropic B-factors refinement. Start: $r_{\text{work}} = 0.4166$ $r_{\text{free}} = 0.4678$ Final: $r_{\text{work}} = 0.3621$ $r_{\text{free}} = 0.4188$ The default refinement run (plus water picking) did a good progress in model improvement. example_4b: Same refinement run as in example_4a with using SA. Start: $r_{\text{work}} = 0.4166$ $r_{\text{free}} = 0.4678$ Final: $r_{\text{work}} = 0.2771$ $r_{\text{free}} = 0.3414$ Adding a SA refinement to the previous refinement run results in big model improvement as indicated by drop in both R-work and R-free factors.

In this couple of examples we demonstrate the power of SA refinement to improve a poor starting model. Combining it with automatic water picking is a good idea at the cost of just one parameter added to the command line.

example_4a: Refinement with all default parameters. This includes 3 macro-cycles of bulk-solvent and anisotropic scaling, coordinates and individual isotropic B-factors refinement.

Start: $r_{\text{work}} = 0.4166$ $r_{\text{free}} = 0.4678$ Final: $r_{\text{work}} = 0.3621$ $r_{\text{free}} = 0.4188$

The default refinement run (plus water picking) did a good progress in model improvement.

example_4b: Same refinement run as in example_4a with using SA.

Start: $r_{\text{work}} = 0.4166$ $r_{\text{free}} = 0.4678$ Final: $r_{\text{work}} = 0.2771$ $r_{\text{free}} = 0.3414$

Adding a SA refinement to the previous refinement run results in big model improvement as indicated by drop in both R-work and R-free factors.

Refinement of f' and f'' for a structure with unknown (to the dictionary) molecule in it

The goal of this example is to show that in case of refinement against anomalous data the refinement of f' and f'' to model the anomalous differences in F^+ and F^- may deliver another drop in R-factors from small fractions to ~1% or more. Also it shows how to use eLBOW program to generate a CIF (stereochemistry dictionary) file for the unknown item in the input PDB. A PDB entry with the code 1mdg is used in this example. example_5a: Refinement with all default parameters. This includes 3 macro-cycles of bulk-solvent and anisotropic scaling, coordinates and individual isotropic B-factors refinement. Final: R-work = 0.1944 R-free = 0.2368 example_5b: Same refinement run as in example_5a with f' and f'' refinement for anomalous scatterer. Final: R-work = 0.1900 R-free = 0.2287 Note the drop in both R and R-free.

The goal of this example is to show that in case of refinement against anomalous data the refinement of f' and f'' to model the anomalous differences in F^+ and F^- may deliver another drop in R-factors from small fractions to ~1% or more. Also it shows how to use eLBOW program to generate a CIF (stereochemistry dictionary) file for the unknown item in the input PDB. A PDB entry with the code 1mdg is used in this example.

example_5a: Refinement with all default parameters. This includes 3 macro-cycles of bulk-solvent and anisotropic scaling, coordinates and individual isotropic B-factors refinement.

Final: R-work = 0.1944 R-free = 0.2368

example_5b: Same refinement run as in example_5a with f' and f'' refinement for anomalous scatterer.

Final: R-work = 0.1900 R-free = 0.2287

Note the drop in both R and R-free.

Refinement using twinned data

This example shows how to do the refinement if the data is twinned. phenix.xtriage was used to obtain the twinning operator. For more information about twinning analysis and using in refinement look separate paragraph in PHENIX documentation. example_6a: Refinement with all default parameters. This includes 3 macro-cycles of bulk-solvent and anisotropic scaling, coordinates and individual isotropic B-factors refinement. A PDB entry with the code 1l2h is used in this example. Final: R-work = 0.2476 R-free = 0.2755 example_6b: Same refinement run as in example_6a with the twinning taken into account. Final: R-work = 0.1640 R-free = 0.2069

This example shows how to do the refinement if the data is twinned. phenix.xtriage was used to obtain the twinning operator. For more information about twinning analysis and using in refinement look separate paragraph in PHENIX documentation.

example_6a: Refinement with all default parameters. This includes 3 macro-cycles of bulk-solvent and anisotropic scaling, coordinates and individual isotropic B-factors refinement. A PDB entry with the code 1l2h is used in this example.

Final: R-work = 0.2476 R-free = 0.2755

example_6b: Same refinement run as in example_6a with the twinning taken into account.

Final: R-work = 0.1640 R-free = 0.2069

Questions, comments, more examples

- [phenix.refine documentation](#)
- PAfonine@lbl.gov or phenixbb@phenix-online.org

Tutorial: Solving a structure with SAD data

sad_phasing.html

Introduction

This tutorial will use some good SAD data (peak wavelength from an IF5A dataset diffracting to 1.7 Å) as an example of how to solve a SAD dataset with AutoSol. It is designed to be read all the way through, giving pointers for you along the way. Once you have read it all and run the example data and looked at the output files, you will be in a good position to run your own data through AutoSol.

Setting up to run PHENIX

If PHENIX is already installed and your environment is all set, then if you type:

```
echo $PHENIX
```

then you should get back something like this:

```
/xtal/phenix-1.3
```

If instead you get:

```
PHENIX: undefined variable
```

then you need to set up your PHENIX environment. See the PHENIX installation page for details of how to do this. If you are using the C-shell environment (csh) then all you will need to do is add one line to your .cshrc (or equivalent) file that looks like this:

```
source /xtal/phenix-1.3/phenix_env
```

(except that the path in this statement will be where your PHENIX is installed). Then the next time you log in \$PHENIX will be defined.

Running the demo p9 data with AutoSol

To run AutoSol on the demo p9 data, make yourself a tutorials directory and cd into that directory:

```
mkdir tutorials
cd tutorials
```

Now type the phenix command:

```
phenix.run_example --help
```

to list the available examples. Choosing p9-sad for this tutorial, you can now use the phenix command:

```
phenix.run_example p9-sad
```

to solve the p9 structure with AutoSol. This command will copy the directory \$PHENIX/examples/p9-sad to your current directory (tutorials) and call it tutorials/p9-sad/. Then it will run AutoSol using the command file run.sh that is present in this tutorials/p9-sad/ directory.

This command file run.sh is simple. It says:

```
#!/bin/sh
phenix.autosol seq_file=seq.dat sites=4 atom_type=Se data=p9_se_w2.sca \
space_group="I4" unit_cell="113.949 113.949 32.474 90.000 90.000 90.00" \
resolution=2.4 thoroughness=quick
```

The first line (`#!/bin/sh`) tells the system to interpret the remainder of the text in the file using the `sh` (or `bash`) -shell (`sh`).

The command `phenix.autosol` runs the command-line version of AutoSol (see Automated Structure Solution using AutoSol for all the details about AutoSol including a full list of keywords). The arguments on the command line tell AutoSol about the sequence file (`seq_file=seq.dat`), the number of sites to look for (`sites=4`), and the atom type (`atom_type=Se`). (Note that each of these is specified with an `=` sign, and that there are no spaces around the `=` sign.) The Phaser heavy-atom refinement and model completion algorithm used in the AutoSol SAD phasing will add additional sites if warranted.

Note the backslash `"\"` at the end of some of the lines in the `phenix.autosol` command. This tells the C-shell (which interprets everything in this file) that the next line is a continuation of the current line. There must be no characters (not even a space) after the backslash for this to work.

The SAD data to be used to solve the structure is in the datafile `p9_se_w2.sca`. This datafile is in Scalepack unmerged format, which means that there may be multiple instances of each reflection and the cell parameters are not in the file, so we need to provide the cell parameters with the command, `unit_cell="113.949 113.949 32.474 90.000 90.000 90.00"`. (Note that the cell parameters are surrounded by quotation marks. That tells the parser that these are all together.) In this example, the space group in the `p9_se_w2.sca` file is `I41`, but the correct space group is `I4`, so we need to tell AutoSol the correct space group with `space_group="I4"`.

The resolution of the data in `p9_se_w2.sca` is to 1.74 Å, but in this example we would like to solve the structure quickly, so we have cut the resolution back with the commands `resolution=2.4` and `thoroughness=quick`. The `quick` command sets several defaults to give a less comprehensive search for heavy-atom sites and a less thorough model-building than if you use the default of `thoroughness=thorough`.

Although the `phenix.run_example p9-sad` command has just run AutoSol from a script (`run.sh`), you can run AutoSol yourself from the command line with the same `phenix.autosol seq_file= ...` command. You can also run AutoSol from a GUI, or by putting commands in another type of script file. All these possibilities are described in Using the PHENIX Wizards.

Where are my files?

Once you have started AutoSol or another Wizard, an output directory will be created in your current (working) directory. The first time you run AutoSol in this directory, this output directory will be called `AutoSol_run_1_` (or `AutoSol_run_1_`, where the slash at the end just indicates that this is a directory). All of the output from run 1 of AutoSol will be in this directory. If you run AutoSol again, a new subdirectory called `AutoSol_run_2_` will be created.

Inside the directory `AutoSol_run_1_` there will be one or more temporary directories such as `TEMP0` created while the Wizard is running. The files in this temporary directory may be useful sometimes in figuring out what the Wizard is doing (or not doing!). By default these directories are emptied when the Wizard finishes (but you can keep their contents with the command `clean_up=False` if you want.)

What parameters did I use?

Once the AutoSol wizard has started (when run from the command line), a parameters file called `autosol.eff` will be created in your output directory (e.g., `AutoSol_run_1_/autosol.eff`). This parameters file has a header that says what command you used to run AutoSol, and it contains all the starting values of all parameters for this run (including the defaults for all the parameters that you did not set).

The autosol.eff file is good for more than just looking at the values of parameters, though. If you copy this file to a new one (for example autosol_hires.eff) and edit it to change the values of some of the parameters (resolution=1.74) then you can re-run AutoSol with the new values of your parameters like this:

```
phenix.autosol autosol_hires.eff
```

This command will do everything just the same as in your first run but use all the data to 1.74 Å.

Reading the log files for your AutoSol run file

While the AutoSol wizard is running, there are several places you can look to see what is going on. The most important one is the overall log file for the AutoSol run. This log file is located in:

```
AutoSol_run_1_/AutoSol_run_1_1.log
```

for run 1 of AutoSol. (The second 1 in this log file name will be incremented if you stop this run in the middle and restart it with a command like phenix.autosol run=1).

The AutoSol_run_1_1.log file is a running summary of what the AutoSol Wizard is doing. Here are a few of the key sections of the log files produced for the p9 SAD dataset.

Summary of the command-line arguments

Near the top of the log file you will find:

```
-----  
Starting AutoSol with the command:  
  
phenix.autosol seq_file=seq.dat sites=4 atom_type=Se data=p9_se_w2.sca space_group=I4 \  
unit_cell='113.949 113.949 32.474 90.000 90.000 90.00' resolution=2.4 \  
thoroughness=quick
```

This is just a repeat of how you ran AutoSol; you can copy it and paste it into the command line to repeat this run.

ImportRawData.

The input data file p9_se_w2.sca is in unmerged Scalepack format. The AutoSol wizard converts everything to premerged Scalepack format before proceeding. Here is where the AutoSol Wizard identifies the format and then calls the ImportRawData Wizard:

```
HKLIN ENTRY: p9_se_w2.sca  
GUESS FILE TYPE MERGE TYPE sca unmerged  
LABELS['I', 'SIGI']  
CONTENTS: ['p9_se_w2.sca', 'sca', 'unmerged', 'I 41', None, None, ['I', 'SIGI']]  
Converting the files ['p9_se_w2.sca'] to sca format before proceeding  
Running import directly...  
WIZARD: ImportRawData
```

Using the datafiles converted to premerged format.

After completing the ImportRawData step, the AutoSol Wizard goes back to the beginning, but uses the newly-converted file p9_se_w2_PHX.sca:

```
HKLIN ENTRY: AutoSol_run_1_/p9_se_w2_PHX.sca  
FILE TYPE scalepack_merge  
GUESS FILE TYPE MERGE TYPE sca premerged  
LABELS['IPLUS', 'SIGIPLUS', 'IMINU', 'SIGIMINU']  
Unit cell: (113.949, 113.949, 32.474, 90, 90, 90)  
Space group: I 4 (No. 79)
```



```
CONTENTS: ['AutoSol_run_1_/p9_se_w2_PHX.sca', 'sca', 'premerged', 'I 4',
[113.949, 113.949, 32.473999999999997, 90.0, 90.0, 90.0],
1.7443432606877809, ['IPLUS', 'SIGIPLUS', 'IMINU', 'SIGIMINU']]
Total of 1 input data files
```

Guessing cell contents

The AutoSol Wizard uses the sequence information in your sequence file (seq.dat) and the cell parameters and space group to guess the number of NCS copies and the solvent fraction, and the number of total methionines (approximately equal to the number of heavy-atom sites for SeMet proteins):

```
AutoSol_guess_setup_for_scaling AutoSol Run 1 Fri Mar 7 00:53:48 2008
```

```
Solvent fraction and resolution and ha types/scatt fact
This is the last dataset to scale
Guessing setup for scaling dataset 1
SG I 4
cell [113.949, 113.949, 32.473999999999997, 90.0, 90.0, 90.0]
Number of residues in unique chains in seq file: 139
Unit cell: (113.949, 113.949, 32.474, 90, 90, 90)
Space group: I 4 (No. 79)
CELL VOLUME :421654.580793
N_EQUIV:8
GUESS OF NCS COPIES: 1
SOLVENT FRACTION ESTIMATE: 0.64
Total residues:139
Total Met:4
resolution estimate: 2.4
```

Running phenix.xtriage

The AutoSol Wizard automatically runs phenix.xtriage on each of your input datafiles to analyze them for twinning, outliers, translational symmetry, and other special conditions that you should be aware of. You can read more about xtriage in Data quality assessment with phenix.xtriage. Part of the summary output from xtriage for this dataset looks like this:

```
The largest off-origin peak in the Patterson function is 6.49% of the
height of the origin peak. No significant pseudotranslation is detected.
```

```
The results of the L-test indicate that the intensity statistics
behave as expected. No twinning is suspected.
```

Testing for anisotropy in the data

The AutoSol Wizard tests for anisotropy by determining the range of effective anisotropic B values along the principal lattice directions. If this range is large and the ratio of the largest to the smallest value is also large then the data are by default corrected to make the anisotropy small (see Analyzing and scaling the data in the AutoSol web page for more discussion of the anisotropy correction). In the p9 case, the range of anisotropic B values is small and no correction is made:

```
Range of aniso B: 15.67 26.14
Not using aniso-corrected data files as the range of aniso b is only
10.47 and 'remove_aniso' is not set
```

Choosing datafiles with high signal-to-noise

During scaling, the AutoSol Wizard estimates the signal-to-noise in each datafile and the resolution where there is significant signal-to-noise (above 0.3:1 signal-to-noise). You can see this analysis in the log file dataset_scale_1.log for dataset 1. In this case, the signal-to-noise is 1.4 to a resolution of 2.4 Å:

Running HYSS to find the heavy-atom substructure

The HYSS (hybrid substructure search) procedure for heavy-atom searching uses a combination of a Patterson search for 2-site solutions with direct methods recycling. The search ends when the same solution is found beginning with several different starting points. The HYSS log files are named after the datafile that they are based on and the type of differences (ano, iso) that are being used. In this p9 SAD dataset, the HYSS logfile is p9_se_w2_PHX.sca_ano_1.sca_hyss.log. The key part of this HYSS log file is:

Entering search loop:

```
p = peaklist index in Patterson map
f = peaklist index in two-site translation function
cc = correlation coefficient after extrapolation scan
r = number of dual-space recycling cycles
cc = final correlation coefficient

p=000 f=000 cc=0.392 r=015 cc=0.532 [ best cc: 0.532 ]
p=000 f=001 cc=0.381 r=015 cc=0.532 [ best cc: 0.532 0.532 ]
Number of matching sites of top 2 structures: 6
```

Here a correlation coefficient of 0.5 is very good (0.1 is hopeless, 0.2 is possible, 0.3 is good) and 6 sites were found that matched in the first two tries. The program continues until 5 structures all have matching sites, then ends and prints out the final correlations, after taking the top 4 sites.

Finding the hand and scoring heavy-atom solutions

Normally either hand of the heavy-atom substructure is a possible solution, and both must be tested by calculating phases and examining the electron density map and by carrying out density modification, as they will give the same statistics for all heavy-atom analysis and phasing steps. Note that in chiral space groups (those that have a handedness such as P61, both hands of the space group must be tested. The AutoSol Wizard will do this for you, inverting the hand of the heavy-atom substructure and the space group at the same time. For example, in space group P61 the hand of the substructure is inverted and then it is placed in space group P65.

Scoring heavy-atom solutions

The AutoSol Wizard scores heavy-atom solutions based on two criteria by default. The first criterion is the skew of the electron density in the map (SKEW). Good values for the skew are anything greater than 0.1. In a SAD structure determination, the heavy-atom solution with the correct hand may have a much more positive skew than the one with the inverse hand. The second criterion is the correlation of local RMS density (CORR_RMS). This is a measure of how contiguous the solvent and non-solvent regions are in the map. (If the local rms is low at one point and also low at neighboring points, then the solvent region must be relatively contiguous, and not split up into small regions.) For SAD datasets, Phaser is used for calculating phases. For a SAD dataset, a figure of merit of 0.3 is acceptable, 0.4 is fine and anything above 0.5 is very good. The scores for solution #1 are listed in the AutoSol log file:

Scoring for this solution now...

```
AutoSol_run_1_/TEMP0/resolve.scores SKEW -0.047612928
AutoSol_run_1_/TEMP0/resolve.scores CORR_RMS 0.8755398
```

```
CC-EST (BAYES-CC) SKEW : 10.0 +/- 26.1
CC-EST (BAYES-CC) CORR_RMS : 55.7 +/- 36.1
```

Resetting sigma of quality estimate due to wide range of estimated values:

```

Overall quality: 14.7
Highest lower bound of quality for individual estimates: 37.6
Current 2*sigma: 37.5 New 2*sigma: 45.7
ESTIMATED MAP CC x 100: 14.7 +/- 45.7

```

The ESTIMATED MAP CC x 100 is an estimate of the quality of the experimental electron density map (not the density-modified one). A set of real structures was used to calibrate the range of values of each score that were obtained for phases with varying quality. The resulting probability distributions are used above to estimate the correlation between the experimental map and an ideal map for this structure. Then all the estimates are combined to yield an overall Bayesian estimate of the map quality. These are reported as CC x 100 +/- 2SD. These estimated map CC values are usually fairly close, so as the estimate is 14.7 +/- 45.7, you can be quite confident that this solution is not the right one.

The wizard then tries the inverse solution...

```

Scoring for this solution now...

```

```

AutoSol_run_1_/TEMP0/resolve.scores SKEW 0.2644597
AutoSol_run_1_/TEMP0/resolve.scores CORR_RMS 0.9274329

```

```

CC-EST (BAYES-CC) SKEW : 56.5 +/- 18.1
CC-EST (BAYES-CC) CORR_RMS : 63.1 +/- 28.5
ESTIMATED MAP CC x 100: 60.0 +/- 13.6

```

```

Reading NCS information from: AutoSol_run_1_/TEMP0/resolve.log
  based on [ha_2.pdb,phaser_2.mtz]
Reformatting ha_2.pdb and putting it in ha_2.pdb_formatted.pdb
RANGE to KEEP :1.28
Confident of the hand (Quality diff from opp hand is 1.9 sigma)

```

This solution looks a lot better. The overall estimated map CC value is 60.0 +/- 13.6. This means that your structure is not only solved but that you will have a good map when it is density modified.

Final phasing with Phaser

Once the best heavy-atom solution or solutions are chosen based on ESTIMATED MAP CC x 100, these are used in a final round of phasing with Phaser (for SAD phasing). The log file from phasing for solution 2 is in phaser_2.log. Here is the final part of the output from this log file, showing the refined coordinates, occupancies, thermal (B) factors for the 4 sites, along with the refined scattering factors (in this case only f' is refined), and the final figure of merit of phasing (0.544):

```

Atom Parameters: 4 atoms in list
  X Y Z O B (AnisoB) M Atomtype
#1 0.180 -0.113 -0.681 1.135 22.8 ( ---- ) 1 SE
#2 0.686 -0.238 -0.710 0.980 23.0 (+22.40) 1 SE
#3 0.665 -0.206 -0.774 1.020 28.2 (+26.14) 1 SE
#5 0.027 0.758 0.905 0.176 23.9 ( ---- ) 1 SE

Scattering Parameters:
  Atom f" (f')
  SE 5.5196 -8.0000

Figures of Merit
-----
Bin Resolution Acentric Centric Single Total
  Number FOM Number FOM Number FOM Number FOM
ALL 28.49- 2.40 7502 0.594 874 0.140 51 0.057 8427 0.544

log-likelihood gain -90088

```

Statistical density modification with RESOLVE

After SAD phases are calculated with Phaser, the AutoSol Wizard uses RESOLVE density modification to improve the quality of the electron density map. The statistical density modification in RESOLVE takes advantage of the flatness of the solvent region and the expected distribution of electron density in the region containing the macromolecule, as well as any NCS that can be found from the heavy-atom substructure. The weighted structure factors and phases (FWT, PHWT) from Phaser are used to calculate the starting map for RESOLVE, and the experimental structure factor amplitudes (FP) and SAD Hendrickson-Lattman coefficients from Phaser are used in the density modification process. The output from RESOLVE for solution 1 can be found in resolve_2.log. Here are key sections of this output.

First, the plot of how many points in the "protein" region of the map have each possible value of electron density. The plot below is normalized so that a density of zero is the mean of the solvent region, and the standard deviation of the density in the map is 1.0. A perfect map has a lot of points with density slightly less than zero on this scale (the points between atoms) and a few points with very high density (the points near atoms), and no points with very negative density. Such a map has a very high skew (think "skewed off to the right"). This map is good, with a positive skew, though it is not perfect.

```
Plot of Observed (o) and model (x) electron density distributions for protein
region, where the model distribution is given by,
  p_model(beta*(rho+offset)) = p_ideal(rho)
and then convoluted with a gaussian with width of sigma
where sigma, offset and beta are given below under "Error estimate."
```

```
0.03.....
. . .
. . .
. xxxxxx .
. 00x000000xx0 .
. xx0 . x00 .
. x0 . xx0 .
p(rho) . xx . xx00 .
. 0x . xx000 .
. x0 . xx0 .
. xx . xxxx .
. x0 . xxxx .
. xx . xxxx .
xxx . xxxxx .
x . 00xxxx
0.0 x.....X

-2 -1 0 1 2 3
```

normalized rho (0 = mean of solvent region)

After density modification is complete, this plot becomes much more like one from a perfect structure:

```
0.03.....
. . .
. xxxxx . .
. x0000xx0 . .
. 0x0 0x0 . .
. xx x0 . .
. 0x xx00 .
p(rho) . 0x .x00 .
. x . x0 .
. x . xxxx .
. xx . xxxx0 .
. xx . xxxxxxxx .
. xx . 00xxxxxx .
xx . 0 0xxxxx00
x . x0
0.0 0.....X

-2 -1 0 1 2 3
```

normalized rho (0 = mean of solvent region)

The key statistic from this RESOLVE density modification is the R-factor for comparison of observed structure factor amplitudes (FP) with those calculated from the density modification procedure (FC). In this p9 SAD phasing the R-factor is very low:

Overall R-factor for FC vs FP: 0.239 for 8422 reflections

An acceptable value is anything below 0.35; below 0.30 is good.

Generation of FreeR flags

The AutoSol Wizard will create a set of free R flags indicating which reflections are not to be used in refinement. By default 5% of reflections (up to a maximum of 2000) are reserved for this test set. If you want to supply a reflection file hires.mtz that has higher resolution than the data used to solve the structure, or has a test set already marked, then you can do this with the keyword `input_refinement_file=hires.mtz`. The files to be used for model-building and refinement are listed in the AutoSol log file:

```
FreeR_flag added to phaser_2.mtz
...
Saving exptl_fobs_phases_freeR_flags_2.mtz for refinement
THE FILE AutoSol_run_1/resolve_2.mtz will be used for model-building
```

Model-building with RESOLVE

The AutoSol Wizard by default uses a very quick method to build just the secondary structure of your macromolecule. This is controlled by the keyword `helices_strands_only=True`. The Wizard will guess from your sequence file whether the structure is protein or RNA or DNA (but you can tell it if you want with `chain_type=PROTEIN`).

If the quick model-building does not build a satisfactory model (if the correlation of map and model is less than `acceptable_secondary_structure_cc=0.35`), then model-building is tried again with the standard build procedure, essentially the same as one cycle of model-building with the AutoBuild Wizard (see the web page Automated Model Building and Rebuilding with AutoBuild, except that if you specify `thoroughness=quick` as we have in this example, the model-building is done less comprehensively to speed things up).

In this case the secondary-structure-only model-building produces an initial model with 61 residues built and side chains assigned to 0, and which has a model-map correlation of 0.33:

```
Model with helices and strands is in Build_1.pdb
Log for helices and strands is in Build_1.log
Final file: AutoSol_run_1/TEMP0/Build_1.pdb
Log file: Build_1.log copied to Build_1.log
Model 1: Residues built=61 placed=0 Chains=9 Model-map CC=0.33
This is new best model with cc = 0.33
Getting R for model: Build_1.pdb
Model: AutoSol_run_1/TEMP0/refine_1.pdb R/Rfree=0.55/0.58
```

As the model-map correlation is only 0.33, the Wizard decides that this is not good enough and tries again with regular model-building, yielding a better model with 86 residues built and a map correlation of 0.55:

```
Model 2: Residues built=86 placed=7 Chains=15 Model-map CC=0.55
This is new best model with cc = 0.55
Refining model: Build_2.pdb
Model: AutoSol_run_1/TEMP0/refine_2.pdb R/Rfree=0.46/0.49
```

After one model completion cycle (including extending ends of chains, fitting loops, and building outside the region already built, the best model built has 77 residues built, 22 assigned to sequence and a map correlation of 0.61:

```

Model completion cycle 1
Models to combine and extend: ['Build_2.pdb', 'refine_2.pdb']
Model 3: Residues built=77 placed=22 Chains=10 Model-map CC=0.61
This is new best model with cc = 0.61
Refining model: Build_combine_extend_3.pdb
Model: AutoSol_run_1_/TEMP0/refine_3.pdb R/Rfree=0.45/0.47

```

This initial model is written out to refine_3.pdb in the output directory. It is still just a preliminary model, but it is good enough to tell that the structure is solved. For full model-building you will want to go on and use the AutoBuild Wizard (see the web page Automated Model Building and Rebuilding with AutoBuild)

The AutoSol_summary.dat summary file

A quick summary of the results of your AutoSol run is in the AutoSol_summary.dat file in your output directory. This file lists the key files that were produced in your run of AutoSol (all these are in the output directory) and some of the key statistics for the run, including the scores for the heavy-atom substructure and the model-building and refinement statistics. These statistics are listed for all the solutions obtained, with the highest-scoring solutions first. Here is part of the summary for this p9 SAD dataset:

```

-----CURRENT SOLUTIONS FOR RUN 1 : -----
*** FILES ARE IN THE DIRECTORY: AutoSol_run_1_ ****

Solution # 2 BAYES-CC: 60.0 +/- 13.6 Dataset #1 FOM: 0.54 -----

Solution 2 using HYSS on
/net/firebird/scratch1/terwill/run_072908a/p9-sad/AutoSol_run_1_/
p9_se_w2_PHX.sca_ano_1.sca and taking inverse. Dataset #1
Dataset number: 1
Dataset type: sad
Datafiles used: [
'/net/firebird/scratch1/terwill/run_072908a/p9-sad/AutoSol_run_1_/p9_se_w2_PHX.sca']
Sites: 4 (Already used for Phasing at resol of 2.4) Refined Sites: 4
NCS information in: AutoSol_2.ncs_spec
Experimental phases in: phaser_2.mtz
Experimental phases plus FreeR_flags for refinement in:
exptl_fobs_phases_freeR_flags_2.mtz
Density-modified phases in: resolve_2.mtz
HA sites (PDB format) in: ha_2.pdb_formatted.pdb
Sequence file in: seq.dat
Model in: refine_3.pdb
Residues built: 77
Side-chains built: 22
Chains: 10
Overall model-map correlation: 0.61
R/R-free: 0.45/0.47
Scaling logfile in: dataset_1_scale.log
HYSS logfile in: p9_se_w2_PHX.sca_ano_1.sca_hyss.log
Phasing logfile in: phaser_2.log
Density modification logfile in: resolve_2.log (R=0.24)
Build logfile in: Build_combine_extend_3.log

Score type: SKEW CORR_RMS
Raw scores: 0.26 0.93
BAYES-CC: 56.50 63.07

Refined heavy atom sites (fractional):
xyz 0.180 -0.113 -0.681
xyz 0.686 -0.238 -0.710
xyz 0.665 -0.206 -0.774
xyz 0.027 0.758 0.905

```

How do I know if I have a good solution?

Here are some of the things to look for to tell if you have obtained a correct solution:

- How much of the model was built? More than 50% is good, particularly if you are using the default of `helices_strands_only=True`. If less than 25% of the model is built, then it may be entirely incorrect. Have a look at the model. If you see clear sets of parallel or antiparallel strands, or if you see helices and strands with the expected relationships, your model is going to be correct. If you see a lot of short fragments everywhere, your model and solution is going to be incorrect. How many side-chains were fitted to density? More than 25% is ok, more than 50% is very good.
- What is the R-factor of the model? This only applies if you are building a full model (not for `helices_strands_only=True`). For a solution at moderate to high resolution (2.5 Å or better) the R-factor should be in the low 30's to be very good. For lower-resolution data, an R-factor in the low 40's is probably largely correct but the model is not very good.
- What was the overall signal-to-noise in the data? Above 1 is good, below 0.5 is very low.
- What are the individual CC-BAYES estimates of map correlation for your top solution? For a good solution they are all around 50 or more, with 2SD uncertainties that are about 10-20.
- What is the overall "ESTIMATED MAP CC x 100" of your top solution. This should also be 50 or more for a good solution. This is an estimate of the map correlation before density modification, so if you have a lot of solvent or several NCS-related copies in the asymmetric unit, then lower values may still give you a good map.
- What is the difference in "ESTIMATED MAP CC x 100" between the top solution and its inverse? If this is large (more than the 2SD values for each) that is a good sign.

What to do next

Once you have run AutoSol and have obtained a good solution and model, the next thing to do is to run the AutoBuild Wizard. If you run it in the same directory where you ran AutoSol, the AutoBuild Wizard will pick up where the AutoSol Wizard left off and carry out iterative model-building, density modification and refinement to improve your model and map. See the web page Automated Model Building and Rebuilding with AutoBuild for details on how to run AutoBuild.

If you do not obtain a good solution, then it's not time to give up yet. There are a number of standard things to try that may improve the structure determination. Here are a few that you should always try:

- Try setting `thoroughness=thorough` if it had previously been set to `quick`. This can make a big difference, though it takes longer.
- Try setting `max_choices` to a larger number, or `desired_coverage` to a higher value.
- Have a careful look at all the output files. Work your way through the main log file (e.g., `AutoSol_run_1_1.log`) and all the other principal log files in order beginning with scaling (`dataset_1_scale.log`), then looking at heavy-atom searching (`p9_se_w2_PHX.sca_ano_1.sca_hyss.log`), phasing (e.g., `phaser_1.log` or `phaser_xx.log` depending on which solution xx was the top solution) and density modification (e.g., `resolve_xx.log`). Is there anything strange or unusual in any of them that may give you a clue as to what to try next? For example did the phasing work well (high figure of merit) yet the density modification failed? (Perhaps the hand is incorrect). Was the solvent content estimated correctly? (You can specify it yourself if you want). What does the xtriage output say? Is there twinning or strong translational symmetry? Are there problems with reflections near ice rings? Are there many outlier reflections?
- Try a different resolution cutoff. For example 0.5 Å lower resolution than you tried before. Often the highest-resolution shells have little useful information for structure solution (though the data may be

useful in refinement and density modification).

- Try a different rejection criterion for outliers. The default is `ratio_out=10.0` (toss reflections with delta F more than 10 times the rms delta F of all reflections in the shell). Try instead `ratio_out=3` to toss outliers.
- If the heavy-atom substructure search did not yield plausible solutions, try searching with HYSS using the command-line interface, and vary the resolution and number of sites you look for. Can you find a solution that has a higher CC than the one found in AutoSol? If so, you can read your solution in to AutoSol with `sites_file=my_sites.pdb`.
- Was an anisotropy correction applied in AutoSol? If there is some anisotropy but no correction was applied, you can force AutoSol to apply the correction with `remove_aniso=True`.

Additional information

For details about the AutoSol Wizard, see Automated structure solution with AutoSol. For help on running Wizards, see Using the PHENIX Wizards.

Tutorial: Refinement against twinned data

twinning.html

Introduction

This tutorial describes the detection of twinning with phenix.xtriage and subsequent refinement with phenix.refine

Background and nomenclature

Twinning is a phenomenon in which the crystal used in data collection is a composition of several distinct domains whose orientation differs, but are related by known (and predictable) operators. The net effect of the presence of multiple lattices is that the recorded data is the sum of a number of diffraction patterns. This tutorial deals with the case when the twinned crystal consists of two domains. This type of twinning is known as hemihedral twinning. The twinning can be either merohedral (M; the twin related lattices overlap exactly) or pseudo-merohedral (PM; the twin related lattices overlap almost exactly). Classification of the type of twinning (M or PM) can be performed on the basis of group theoretical arguments and is done in phenix.xtriage. X-ray data collected from a hemihedrally twinned specimen is effectively the sum of two diffraction patterns. The relative size of the smallest crystal domain to the whole crystal is known as the twin fraction, often denoted by α . The operator that relates overlapping miller indices, is known as the twin law. As mentioned in the previous section, the effect of twinning on the X-ray data, is that the intensity for a given miller index as seen on the detector has two non-equal contributors:

$$J(H) = (1-\alpha)I(H) + (\alpha)I(RH) \text{ [eq. 1a]}$$

$$J(RH) = (1-\alpha)I(RH) + (\alpha)I(H) \text{ [eq. 1b]}$$

In the previous expressions, the intensities I of the miller index H and its twin mate RH build up a single intensity J . The twin law is denoted by R and is used to find the RH , twin related index of H . The twin law R is usually written down in algebraic terms: $(k,h,-l)$.

Detection of twinning with phenix.xtriage

The presence of twinning usually reveals itself by intensity statistics that do not fall within the range expected for untwinned data. phenix.xtriage reports a number of intensity statistics:

- $\langle I^2 \rangle / \langle I \rangle^2$
- $\langle F^2 \rangle / \langle F \rangle^2$
- $\langle |E^2 - 1| \rangle$
- an NZ plot
- $\langle |L| \rangle$

Expected values for the above statistics and their 'allowed' ranges for untwinned data are known from a data bases analysis. The results of the L-test are used in xtriage to auto-detect twinning.

Examples

Typing

```
phenix.xtriage porin.cv xray_data.unit_cell=104.4,104.4,124.25,90,90,120 xray_data.space_group=R3
```

gives (parts omitted):

Determining possible twin laws.

The following twin laws have been found:

Type	Axis	R metric (%)	delta (le Page)	delta (Lebedev)	Twin law
M	2-fold	0.000	0.000	0.000	-h-k,k,-l
PM	2-fold	2.476	1.548	0.022	-h,2/3*h+1/3*k-2/3*l,-2/3*h-4/3*k-1/3*l
PM	4-fold	2.476	1.548	0.022	h+k,-2/3*h-1/3*k+2/3*l,2/3*h-2/3*k+1/3*l
PM	2-fold	2.476	1.548	0.022	-h,1/3*h-1/3*k+2/3*l,2/3*h+4/3*k+1/3*l
PM	3-fold	2.476	1.032	0.022	h+k,-1/3*h-2/3*k-2/3*l,-2/3*h+2/3*k-1/3*l
PM	4-fold	2.476	1.548	0.022	-k,1/3*h+2/3*k+2/3*l,-4/3*h-2/3*k+1/3*l
PM	3-fold	2.476	1.032	0.022	-k,-1/3*h+1/3*k-2/3*l,4/3*h+2/3*k-1/3*l

M: Merohedral twin law

PM: Pseudomerohedral twin law

1 merohedral twin operators found

6 pseudo-merohedral twin operators found

In total, 7 twin operator were found

.
. .
. .
. .

Twinning and intensity statistics summary (acentric data):

Statistics independent of twin laws

- $\langle I^2 \rangle / \langle I \rangle^2$: 1.667
- $\langle F \rangle^2 / \langle F^2 \rangle$: 0.857
- $\langle |E^2 - 1| \rangle$: 0.609
- $\langle |L| \rangle$, $\langle L^2 \rangle$: 0.401, 0.225

Multivariate Z score L-test: 8.174

The multivariate Z score is a quality measure of the given spread in intensities. Good to reasonable data is expected to have a Z score lower than 3.5.

Large values can indicate twinning, but small values do not necessarily exclude it.

Statistics depending on twin laws

Operator	type	R obs.	Britton alpha	H alpha	ML alpha
-h-k,k,-l	M	0.195	0.292	0.315	0.304
-h,2/3*h+1/3*k-2/3*l,-2/3*h-4/3*k-1/3*l	PM	0.420	0.068	0.069	0.022
h+k,-2/3*h-1/3*k+2/3*l,2/3*h-2/3*k+1/3*l	PM	0.410	0.079	0.084	0.022
-h,1/3*h-1/3*k+2/3*l,2/3*h+4/3*k+1/3*l	PM	0.419	0.070	0.069	0.022
h+k,-1/3*h-2/3*k-2/3*l,-2/3*h+2/3*k-1/3*l	PM	0.413	0.078	0.079	0.022
-k,1/3*h+2/3*k+2/3*l,-4/3*h-2/3*k+1/3*l	PM	0.415	0.070	0.075	0.022
-k,-1/3*h+1/3*k-2/3*l,4/3*h+2/3*k-1/3*l	PM	0.415	0.074	0.077	0.022

Patterson analysis

- Largest peak height : 5.689
- (corresponding p value : 7.822e-01)

The largest off-origin peak in the Patterson function is 5.69% of the height of the origin peak. No significant pseudo translation is detected.

The results of the L-test indicate that the intensity statistics are significantly different then is expected from good to reasonable, untwinned data.

As there are twin laws possible given the crystal symmetry, twinning could be the reason for the departure of the intensity statistics from normality. It might be worthwhile carrying out refinement with a twin specific target function.

As listed clearly in the summary states above, the data is suspected to be twinned. Given the relatively large tolerances in finding twin laws, seven twin laws are found. Six of them are pseudo merohedral, one of them is merohedral. Looking at the R-value analysis in the last table (column R-obs), the merohedral twin law is the most likely in this case, as this merging R-value is lower than any of the other R values reported.

A quick model based twinning analysis

In most cases, the presence of twinning does not impede structure solution via molecular replacement (or sometimes even S/MAD). If a molecular replacement solution is available testing which twin law is the most likely can be done via

```
phenix.twin_map_utils xray_data.file=porin.cv model=porin.pdb unit_cell=104.4,104.4,124.25,90,90,120 space_group=R3
```

The latter tool performs bulk solvent scaling and R-value calculation for all possible twin laws in the given crystal setting. In this particular case, the twin law (-h-k,k,-l) is the most likely, as it produces the R-value and lowest refinement target value out of the 7 listed twin listed (0.19 vs 0.25).

Selection of a cross validation set

Although the given reflection file in this tutorial already has an assigned test set for cross validation purposes, assigning it properly is relatively important. When the data is twinned, each observed reflection has a contribution of two (with hemihedral twinning) non-twinned components. When a test set is designed, care must be taken that free and work reflections are not related by a twin law. The R-free set assignment in phenix.refine and phenix.reflection_file_converter is designed with this in mind: the free reflections are chosen to obey the highest possible symmetry of the lattice. Choosing a free set with phenix.refine is as simple as including the keywords

```
xray_data.r_free_flags.generate=True
```

on the command line

Refinement protocols

Within phenix, various refinement protocols can be followed. A few typical examples will be shown below.

Restrainted refinement of positional and atomic displacement parameters

Standard restrained refinement of positional and atomic displacement parameters is invoked via

```
phenix.refine porin.cv porin.pdb twin_law="-h-k,k,-l" unit_cell=104.4,104.4,124.25,90,90,120 space_group=R3
```

Refinement is performed in macro cycles, in which either positions, atomic displacement parameters or twin and bulk solvent parameters are refined.

Rigid body refinement

The command

```
phenix.refine porin.cv porin.pdb twin_law="-h-k,k,-l" strategy=rigid_body unit_cell=104.4,104.4,124.25,90,90,120
```

will perform rigid body refinement using the twin target function.

TLS

For tls refinement, it is advisable to construct a small parameter file that contains TLS definitions:

```
refinement.refine {
  strategy = *individual_sites rigid_body *individual_adp group_adp *tls \
  occupancies group_anomalous none
  adp {
    tls = "chain A"
  }
}
refinement.twinning {
  twin_law = "-h-k,k,-l"
}
```

Saving these parameters as tls.def one can run

```
phenix.refine porin.cv porin.pdb tls.def unit_cell=104.4,104.4,124.25,90,90,120 space_group=R3
```

Water picking

Ordered solvent can be picked as a part of the refined procedure, more details are available from the phenix.refine manual. Note that the (difference) map used by phenix.refine, is constructed using detwinned data (see below). Including water picking in refinement can be carried out as follows:

```
phenix.refine porin.cv porin.pdb twin_law="-h-k,k,-l" ordered_solvent=True unit_cell=104.4,104.4,124.25,90,90,120
```

Map inspection

Electron density maps during twin refinement are constructed using detwinned data. Choice of map coefficients and detwinning mode, is controlled by a set of parameters as shown below:

```
refinement.twinning {
  twin_law = "-h-k,k,-l"
  detwin {
    mode = algebraic proportional *auto
    local_scaling = False
    map_types {
      twofofc = *two_m_dtfo_d_fc two_dtfo_fc
      fofc = *m_dtfo_d_fc gradient m_gradient
      aniso_correct = False
    }
  }
}
```

By default, data is detwinned using algebraic techniques, unless the twin fraction is above 45%, in which case detwinning is performed using proportionality of twin related I_{calc} values. Detwinning using the proportionality option, results in maps that are more biased towards the model, resulting in seemingly cleaner, but in the end less informative maps. The 2mFo-dFc map coefficients can be chosen to have sigmaA weighting (two_m_dtfo_d_fc) or not (two_dtfo_fc). IN both cases, the map coefficients correspond to the 'untwinned' data. A difference map can be constructed using either sigmaA weighted detwinned data (m_dtfo_d_fc), a sigmaA weighted gradient map (m_gradient) or a plain gradient map (gradient). The default is m_dtfo_d_fc but can be changed to gradient or m_gradient if desired.

Phenix Video Tutorials

tutorial_channel.html

The Phenix video tutorials explain how to run Phenix programs and how to interpret the results. All tutorial videos are available on the Phenix tutorials YouTube channel: www.youtube.com/c/phenixtutorials.

A list of all videos is also below (click on the panels to expand the view).

Overview

Map-tools

Refinement

Reflection data

Data analysis

Molecular replacement

Experimental Phasing

Model Building

Other tools

Ligands

Validation

Mixed Tools

Mini-lectures

Other

Full length lectures (45 min - 1 hour)